เอกสารพกเข้าห้องสอน · แบบเปิดตำรา (Open Book)

📘 เอกสารพกเข้าห้องสอน Final — Operating Systems (ระบบปฏิบัติการ)

05506007 · เรียบเรียงจาก Presentation ของอาจารย์ (ภาพต้นฉบับ) + Handout + วิดีโอ + Quizizz ในห้องเรียน

เน้นละเอียดบทที่ 7–13 · พิมพ์เป็น A4 ได้ (Ctrl+P) · ภาพสไลด์ทุกภาพอ้างอิงเลขสไลด์จริงจาก presentation ของอาจารย์

🖨️ วิธีใช้ + พิมพ์

• เปิดหน้านี้ทิ้งไว้ กด `Ctrl+P` → เลือก Save as PDF → เปิด "Background graphics" เพื่อให้สีสไลด์/ตารางออกครบ

• กล่อง **แดง = จุดหลอกข้อสอบ** · **เขียว = เทคนิค/สูตรทำเร็ว** · **ม่วง = อุปมาช่วยจำ** · ภาพสไลด์ = ข้อมูลต้นฉบับของอาจารย์ (ตัวเลขในโจทย์ตัวอย่างตรงกับสไลด์)

สารบัญ

A. Process (กระบวนการ) & fork (แตกสำเนา process (กระบวนการ))/exec (เปลี่ยนโปรแกรมที่ process (กระบวนการ) รัน)/wait (รอ + หยิบสิทธิ์) 🔥

B. Thread (เธรด (สายงานย่อย)) & Multi-Threading

C. CPU Scheduling (การจัดคิว CPU) — Gantt/TAT/WT 🔥

D. Race Condition (สภาพแข่งขันของข้อมูล) & Critical Section (ช่วงวิกฤต (โค้ดแตะของแชร์))

E. Semaphore (ตัวนับควบคุมสิทธิ์เข้าถึง) · Monitor · Java (ภาษาจาวา) sync 🔥

F. Deadlock (การติดตาย (รอกันตลอดกาล)) + Banker's Algorithm (อัลกอริทึมนายธนาคาร) 🔥

G. Real Memory (หน่วยความจำ) Management

H. Paging (การแบ่งหน่วยความจำเป็นหน้า) & Address (ที่อยู่หน่วยความจำ) Translation 🔥🔥

I. Virtual Memory (หน่วยความจำ) & Page Replacement (การเลือกเพจออกเมื่อ RAM เต็ม) 🔥🔥

J. File System (ระบบไฟล์)

K. ความรู้จาก Quizizz ทั้ง 4 ชุด

L. แผ่นสูตรเร็วท้ายเล่ม (1 หน้า)

A. Process (กระบวนการ) & การสร้าง Process (กระบวนการ) ด้วย fork (แตกสำเนา process (กระบวนการ)) / exec (เปลี่ยนโปรแกรมที่ process (กระบวนการ) รัน) / wait (รอ + หยิบสิทธิ์) (OSNo03)

📖 รู้จักส่วนนี้ก่อนอ่าน

ความหมาย: Process (กระบวนการ) = โปรแกรมที่ "มีชีวิต" กำลังทำงานอยู่ใน RAM มีสถานะและทรัพยากรของตัวเอง · fork (แตกสำเนา process (กระบวนการ)) = คำสั่งแตกสำเนา process (กระบวนการ), exec (เปลี่ยนโปรแกรมที่ process (กระบวนการ) รัน) = เปลี่ยนโปรแกรมที่ process (กระบวนการ) กำลังรัน, wait (รอ + หยิบสิทธิ์) = ให้พ่อรอลูกจบ

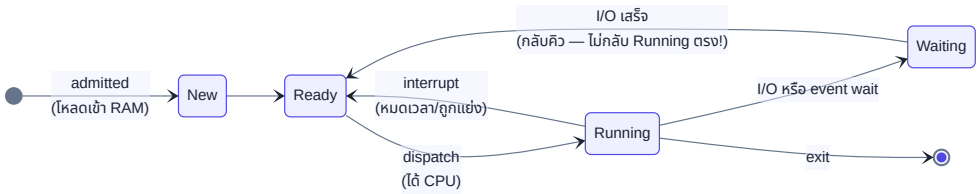
ความสำคัญกับข้อสอบ: ข้อสอบชอบให้ "ไล่โค้ด fork (แตกสำเนา process (กระบวนการ))" — นับว่ามี process (กระบวนการ) เกิดกี่ตัว เดาว่า output พิมพ์อะไร รุ่นพี่เตือนว่ามีจุดหลอกเยอะ และย้ำว่าโค้ดทุกข้อ compile ผ่านหมด ไม่มีคำตอบว่า Error

ใช้จริงในคอมพิวเตอร์: ทุกครั้งที่พิมพ์คำสั่งใน terminal หรือดับเบิลคลิกเปิดแอป คือ shell (โปรแกรมรับคำสั่งผู้ใช้) ทำ fork (แตกสำเนา process (กระบวนการ)) → exec (เปลี่ยนโปรแกรมที่ process (กระบวนการ) รัน) → wait (รอ + หยิบสิทธิ์) ให้เราทั้งหมด — เช่น พิมพ์ ls, เปิด Chrome, compile ไฟล์

A1. Program vs Process (กระบวนการ) · สถานะของ Process (กระบวนการ)

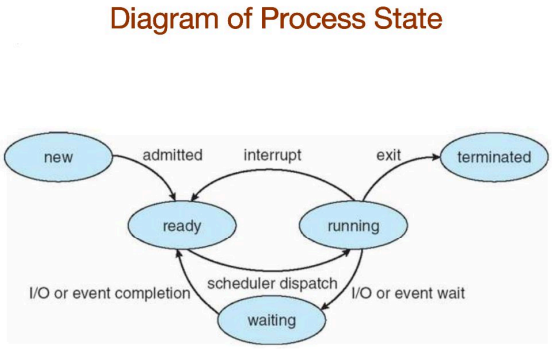
	Program	Process (กระบวนการ)
คือ	ไฟล์คำสั่งนิ่ง ๆ บนดิสก์ — **passive entity**	**program in execution** — active entity ต้องใช้ทรัพยากร
มีอะไร	แค่โค้ด	Program counter (PC), stack, data (ข้อมูล) section, heap, registers
ชื่อเรียกอื่น	—	Task (time-sharing) · Job (batch system)

สถานะของ process (state diagram):

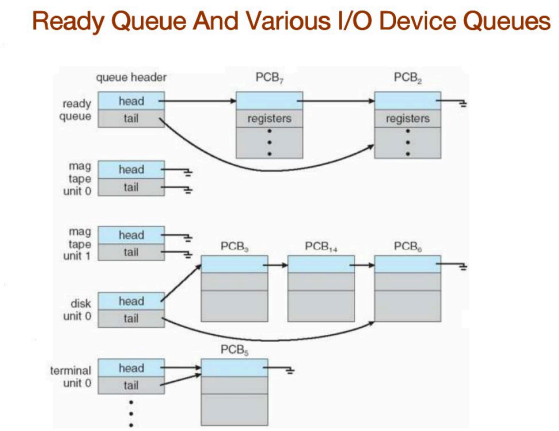


🎯 จุดหลอก

Waiting เสร็จแล้วไป **Ready ก่อนเสมอ** ไม่ใช่กลับ Running ทันที (ต้องรอ dispatcher เรียก) · Batch system เรียก process (กระบวนการ) ว่า Job (งาน (ในระบบ batch)), time-sharing เรียก Task



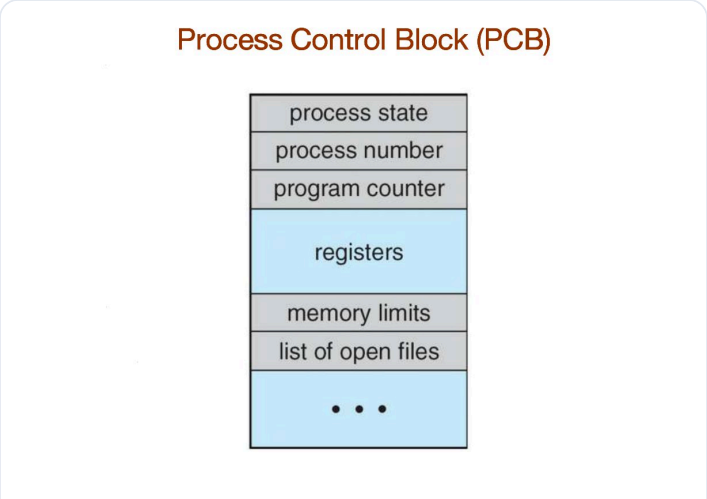
สไลด์ OSNo03 p.4 — Diagram of Process (กระบวนการ) State (ต้นฉบับอาจารย์)



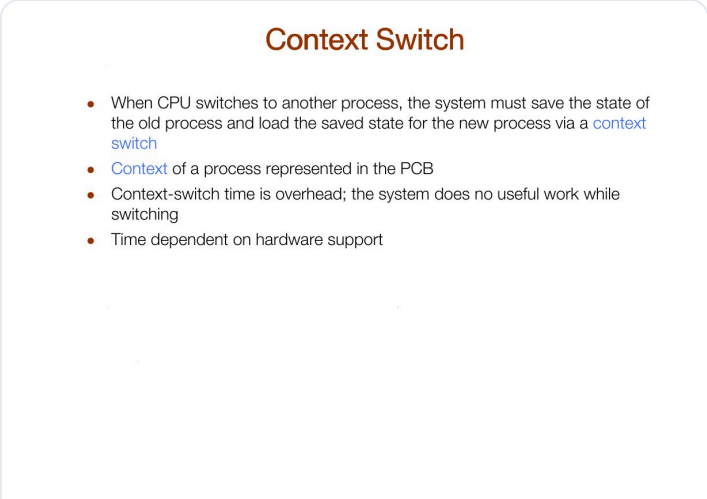
สไลด์ OSNo03 p.7 — Ready Queue (คิวงานที่พร้อมรัน) และ I/O Device Queues: process (กระบวนการ) วิ่งวน Ready → CPU → I/O คิว

A2. PCB — Process (กระบวนการ) Control Block (บล็อก (หน่วยจัดเก็บ))

PCB = "รหัสประชาชน" ของ process (กระบวนการ) เก็บทุกอย่างที่ OS ต้องรู้: **process (กระบวนการ) state, program counter, CPU registers, scheduling (การจัดคิว) info (priority (ลำดับสิทธิ์)/queue), memory (หน่วยความจำ) info (base/limit), accounting info, I/O status**



สไลด์ OSNo03 p.8 — Process (กระบวนการ) Control Block (บล็อก (หน่วยจัดเก็บ))



สไลด์ OSNo03 p.9 — Context Switch (การสลับงานบน CPU): เซฟ state ของ P0 ลง PCB0 → โหลด PCB1 → รัน P1 (ช่วงนี้ = pure overhead)

Dispatcher (ผู้ส่งงานเข้า CPU) ทำ 3 อย่าง

(1) switching context (2) switch to user mode (3) jump ไปตำแหน่งเดิมใน user program · **Dispatch latency** = เวลาที่ dispatcher ใช้หยุดตัวหนึ่ง เริ่มอีกตัวหนึ่ง

A3. fork() — สร้าง process (กระบวนการ) แบบโคลนตัวเอง

`pid_t fork(void)` สร้าง child โดย**ก๊อปปี้ parent** (address (ที่อยู่หน่วยความจำ) space, ตัวแปร, fd ทุกอย่าง) แล้วทั้งคู่รันต่อจากบรรทัดเดียวกัน (หลัง fork (แตกสำเนา process (กระบวนการ)))

ใครเรียก	ค่าที่ fork() คืน	เงื่อนไข if
ใน Parent	**PID ของลูก** (> 0)	`if (pid > 0)`
ใน Child	**0**	`else if (pid == 0)`
สร้างไม่สำเร็จ	**ค่าลบ**	`if (pid < 0)` → error

🪄 เข้าใจง่าย

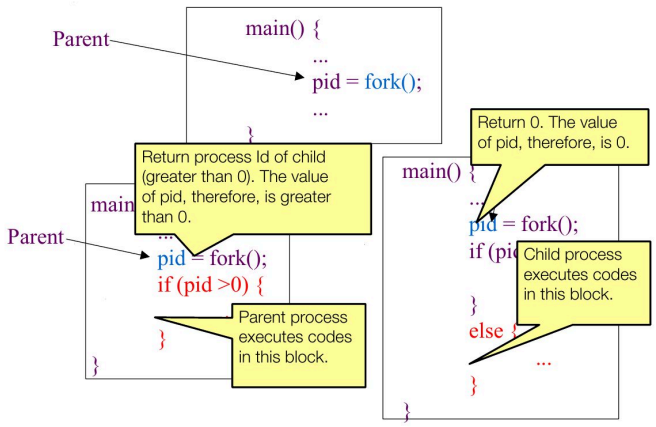
fork() เรียก**ครั้งเดียวแต่คืนค่า 2 ครั้ง** — ครั้งนึงใน "โลกพ่อ" ครั้งนึงใน "โลกลูก" ทั้งคู่ถือ copy ของตัวแปร pid ของตัวเอง แก้ฝั่งไหนไม่กระทบอีกฝั่ง

```c
/* fork_example.c — ตัวอย่างจากสไลด์/วิดีโอ: ผลลัพธ์พิมพ์ 2 บรรทัด */
#include <stdio.h>
#include <sys/types.h>   // pid_t
#include <unistd.h>      // fork()

int main(void) {
    pid_t pid;                  // เก็บค่า return ของ fork
    int i = 0;                  // ตัวแปรธรรมดา — จะถูก "ก๊อป" ไปให้ลูกด้วย

    pid = fork();               // ← จุดแตกหน่อ: หลังบรรทัดนี้มี 2 process รันโค้ดชุดเดียวกัน
    if (pid > 0) {              // โลกพ่อ: ได้ PID ของลูก (>0) เสมอ
        i = 1;                  // แก้ i ใน address space ของพ่อ — ลูกไม่เห็น
        printf("I am parent i = %d\n", i);
    }
    else {                      // โลกลูก: fork คืน 0
        i = 2;
        printf("I am child i = %d\n", i);
    }
    return 0;
}
/* Output: "I am parent i = 1" และ "I am child i = 2" (2 บรรทัดเสมอ)
   แต่ลำดับก่อน-หลัง non-deterministic — ขึ้นกับ scheduler ห้ามสรุปใครพิมพ์ก่อน */
```

Diagram for fork() example



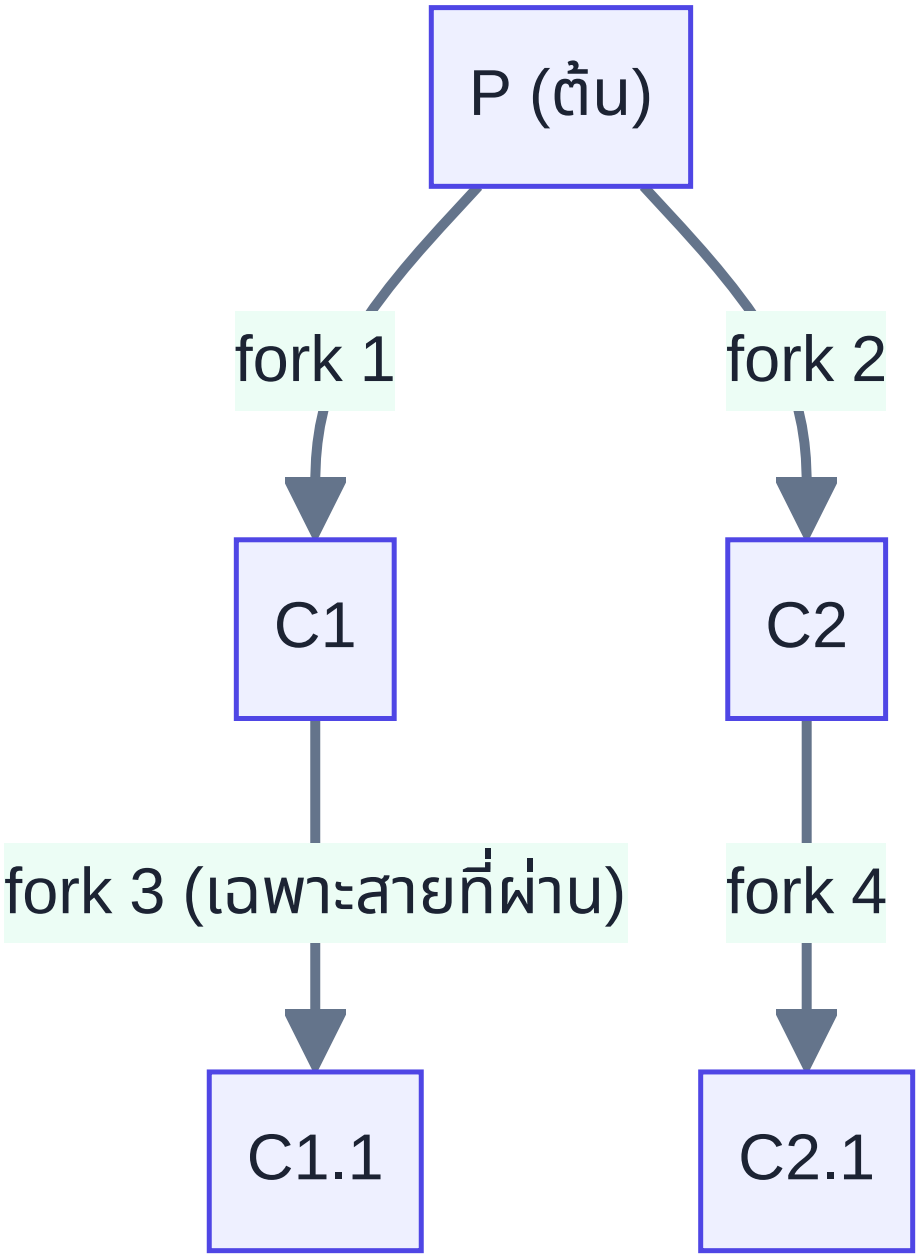
สไลด์ OSNo03 p.19 — Diagram for fork() example: พ่อ/ลูกรันต่อจากจุดเดียวกัน แยกคนละสาย

Process IDs (pids)

- pid = fork();
- In the child: pid == 0;
  In the parent: pid == the process ID of the child.
- If the value of pid is negative, it indicates that the creation of a child process was unsuccessful.
- The application can utilize this difference in the value of pid to execute distinct actions in the parent and child processes.

สไลด์ OSNo03 p.20 — Process (กระบวนการ) IDs: ลูกได้ 0 / พ่อได้ PID ลูก / ติดลบ = fail

A3.1 นับจำนวน process (กระบวนการ) — วาด Process Tree (แผนภาพต้นไม้ความสัมพันธ์พ่อ-ลูก)



`fork()` ติดกัน `n` ครั้ง ไม่มีเงื่อนไข → process (กระบวนการ) ทั้งหมด = 2^n (ลูกใหม่ = $2^n - 1$)

fork (แตกสำเนา process (กระบวนการ)) 3 ครั้ง = 8 process (กระบวนการ) · ถ้ามี if/exit ให้วาด tree แล้วนับ "สายที่ผ่าน fork (แตกสำเนา process (กระบวนการ)) จุดนั้น" ทีละจุด

A3.2 fork (แตกสำเนา process (กระบวนการ)) + loop จากวิดีโอ (จุดหลอก: ลูกต้อง exit)

```c
for (i = 0; i < 5; i++) {          // พ่อวน 5 รอบ
    pid = fork();
    if (pid == 0) {                 // ลูกเท่านั้นเข้า if
        if (i % 2 != 0) printf("I am happy\n");  // ลูกที่เกิดตอน i คี่
        else            printf("I am sad\n");    // ลูกที่เกิดตอน i คู่
        exit(0);                    // ← ถ้าลืม! ลูกจะหลุด if ไปวน for ต่อ
    }                               //   แล้ว fork ลูก/หลานของตัวเองเพิ่ม
}
while (wait(NULL) != -1);           // พ่อรอลูกทุกตัว (wait คืน -1 เมื่อไม่มีลูกเหลือ)
printf("I am parent goodbye\n");
```

> **🎯 จุดหลอก (อาจารย์สาธิตในวิดีโอ)**
> มี exit() → sad ×3 (i=0,2,4), happy ×2 (i=1,3), goodbye **1 ครั้งสุดท้าย** · ถ้าลูกเลขคี่**ลืม exit()** → ลูก C1, C3 กลายเป็น "พ่อ" สร้างหลาน → goodbye พิมพ์ **4 ครั้ง** (พ่อหลัก + C1 + C3 + หลาน $C1_{-3}$)

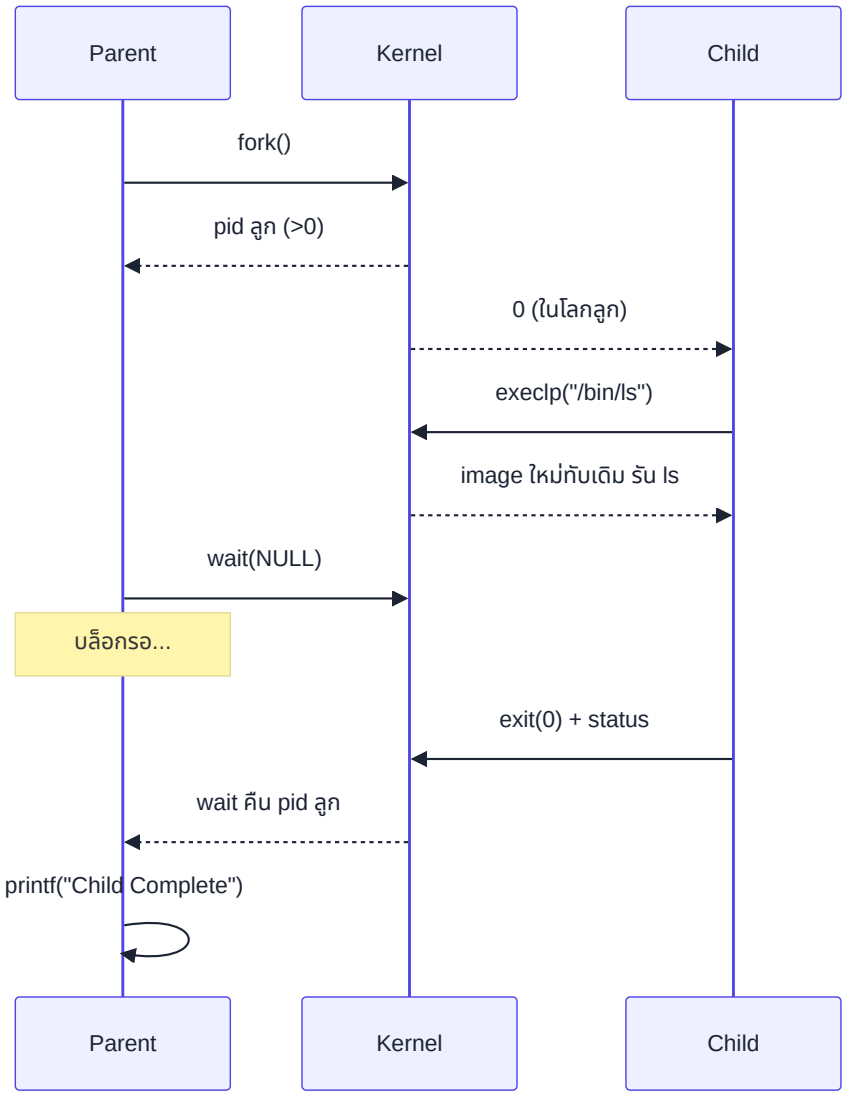
A4. exec() — เปลี่ยนร่าง ไม่สร้างคนใหม่

- ตระกูล exec (เปลี่ยนโปรแกรมที่ process (กระบวนการ) รัน) แทนที่ **process (กระบวนการ) image ทั้งหมด** (โค้ด+data (ข้อมูล)) ด้วยโปรแกรมใหม่ — PID คงเดิม
- `execlp(file, arg0, arg1, ..., NULL)` — **l** = list (ส่งทีละตัว) · **p** = หาโปรแกรมตาม PATH · **arg0** = ธรรมเนียมใส่ชื่อโปรแกรม (exec (เปลี่ยนโปรแกรมที่ process (กระบวนการ) รัน) ข้าม แต่ต้องใส่) · **NULL ปิดท้ายบังคับ**
- execv: ส่งเป็น array `char *argv[]`

> **🎯 จุดหลอก exec (เปลี่ยนโปรแกรมที่ process (กระบวนการ) รัน)**
> exec (เปลี่ยนโปรแกรมที่ process (กระบวนการ) รัน) **สำเร็จ** → โค้ดหลังบรรทัด exec (เปลี่ยนโปรแกรมที่ process (กระบวนการ) รัน) **ไม่มีวันรัน** (image โดนทับหมด) · exec (เปลี่ยนโปรแกรมที่ process (กระบวนการ) รัน) **ล้มเหลว** (ชื่อผิด เช่น "ly") → รันบรรทัดถัดไปต่อ · `exec("cd")` fail เสมอเพราะ **cd เป็น shell (โปรแกรมรับคำสั่งผู้ใช้) built-in** ไม่ใช่ไฟล์ (echo ได้เพราะมี /usr/bin/echo)

A5. fork (แตกสำเนา process (กระบวนการ)) + exec (เปลี่ยนโปรแกรมที่ process (กระบวนการ) รัน) + wait (รอ + หยิบสิทธิ์) — รูปแบบมาตรฐานของ shell (โปรแกรมรับคำสั่งผู้ใช้)

```c
int main() {
    pid_t pid = fork();
    if (pid < 0) {                          // สร้างลูกไม่สำเร็จ
        fprintf(stderr, "Fork Failed");
        exit(-1);
    }
    else if (pid == 0) {                    // ── ลูก: เปลี่ยนร่างเป็น ls ──
        execlp("/bin/ls", "ls", NULL);      // สำเร็จ = โค้ดลูกถูกแทนที่ทั้งหมด
    }
    else {                                  // ── พ่อ: รอลูก ──
        wait(NULL);                          // บล็อกรอลูกตัวใดตัวหนึ่งจบ (ได้ pid ลูกกลับ)
        printf("Child Complete");           // พิมพ์หลังลูกจบเสมอ
        exit(0);
    }
}
/* Output: ผล ls ก่อน ตามด้วย "Child Complete" — ลำดับแน่นอนเพราะ wait() */
```



A6. exit / wait (รอ + หยิบสิทธิ์) / waitpid (รอลูกที่ระบุตัวเจาะจง) + สถานะพิเศษ

Call	ทำอะไร	จุดสำคัญ
`exit(status)`	ขอ OS ทำลายตัวเอง ส่ง status ให้พ่อ	ส่งได้แค่ **8 บิต (0–255)** — exit(300) พ่ออ่านได้ 300 mod 256 = 44
`wait(&status)`	รอ*ลูกใดก็ได้*จบ	คืน pid ลูกที่จบ · ไม่มีลูก → คืน **-1** · ไม่สน status ใช้ wait(NULL)
`waitpid(pid, &status, 0)`	รอ*ลูกตัวเจาะจง*	พ่อรายงานผลเรียงลำดับได้ (ต่างจาก wait (รอ + หยิบสิทธิ์) ที่ได้ใครมาก่อน)
`WIFEXITED(status)`	เช็คลูกจบแบบปกติ?	พิมพ์ status ตรง ๆ ได้เลขแปลก (32768 ฯลฯ) เพราะ OS แปะข้อมูลอื่นปน — **ต้อง decode ด้วยแมโคร**
`WEXITSTATUS(status)`	ดึงค่า exit จริง	

🧟 Zombie (process (กระบวนการ) ตายแล้วแต่รายการค้าง) process (กระบวนการ)

ลูก**ตายแล้ว**แต่พ่อไม่ wait() → ทรัพยากรถูกคืนแต่รายการใน process (กระบวนการ) table (ตาราง) ค้าง สถานะ **Z / defunct** — ทางแก้: พ่อต้อง wait (รอ + หยิบสิทธิ์)

👶 Orphan (process (กระบวนการ) ที่พ่อตายก่อน) process (กระบวนการ)

พ่อ**ตายก่อนลูก** → OS ให้ init/systemd "รับเลี้ยง" (ps -l เห็น PPID เปลี่ยน) · **Cascading termination**: พ่อโดน terminate แล้ว OS บางระบบลบลูกหลานตามทั้งตระกูล

A6.1 ประสานพ่อ-ลูก: wait (รอ + หยิบสิทธิ์) vs waitpid (จากวิดีโอ parent.c/child.c)

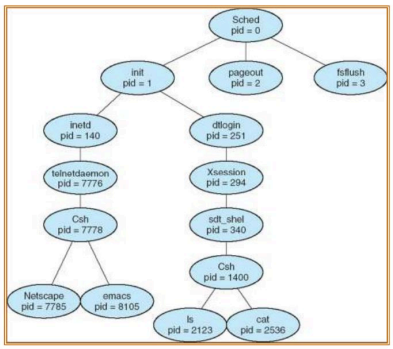
```c
/* child.c: รับ pid ตัวเอง สุ่ม sleep 0-4 วิ แล้ว exit(pid % 256)
   parent.c: fork 3 ลูก (pid[i]) แล้วเปรียบเทียบสองแบบ */
// แบบ 1: wait(&status) → ลูกตัวไหนจบก่อน พ่อรายงานตัวนั้นก่อน (ลำดับไม่แน่)
while ((pid = wait(&status)) > 0) printf("child %d done\n", pid);
// แบบ 2: waitpid(pid[i], &status, 0) → พ่อรอ "ตามลำดับที่กำหนด" เสมอ
for (i = 0; i < 3; i++) {
    waitpid(pid[i], &status, 0);   // นิ่งรอลูกตัวที่ 1 ก่อน แม้ตัว 3 จบก่อน
    printf("child %d done\n", pid[i]);
}
```

Fork and exec()

```c
int main()
{
    pid_t  pid;
    /* fork another process */
    pid = fork();
    if (pid < 0) { /* error occurred */
        fprintf(stderr, "Fork Failed");
        exit(-1);
    }
    else if (pid == 0) { /* child process */
        execlp("/bin/ls", "ls", NULL);
    }
    else { /* parent process */
        /* parent will wait for the child to complete */
        wait (NULL);
        printf ("Child Complete");
        exit(0);
    }
}
```

สไลด์ OSNo03 p.23 — Fork and exec(): โครงโค้ดมาตรฐานจากสไลด์อาจารย์

A tree of processes on a typical Solaris



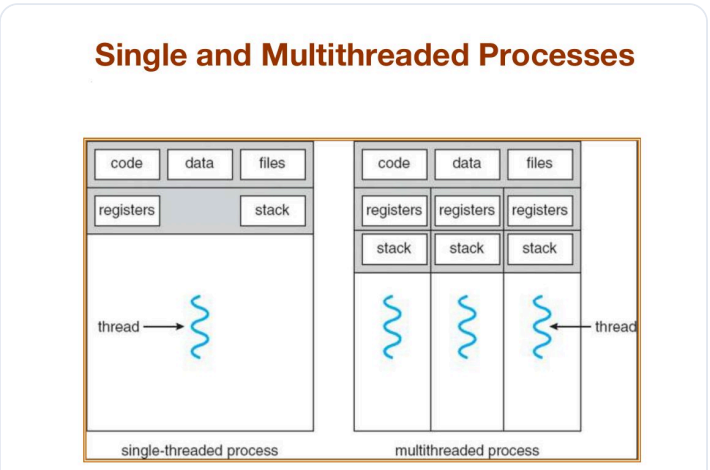
สไลด์ OSNo03 p.15 — Process (กระบวนการ) tree (login → sh → คำสั่งต่าง ๆ)

B. Thread (เธรด (สายงานย่อย)) & การเขียนโปรแกรม Multi-Threaded (OSNo04)

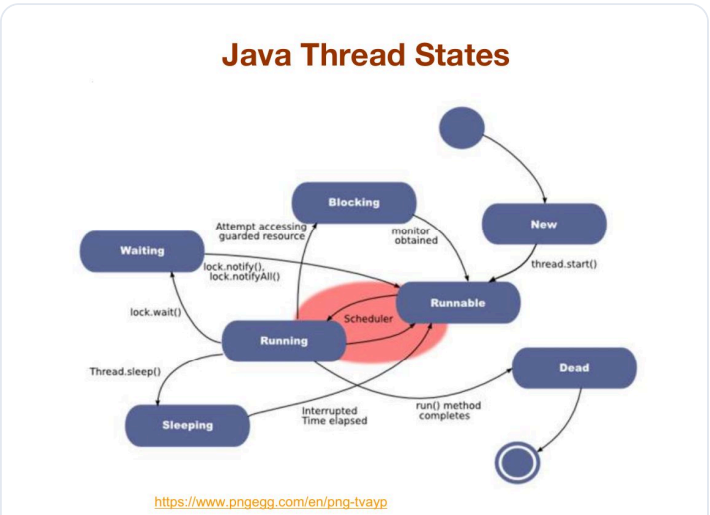
> 📖 **รู้จักส่วนนี้ก่อนอ่าน**
>
> **ความหมาย:** Thread (เธรด/สายงานย่อย) = หน่วยทำงานที่เล็กกว่า process (กระบวนการ) หลาย thread (เธรด (สายงานย่อย)) รันขนานกันภายใน process (กระบวนการ) เดียวกันได้ โดยแชร์หน่วยความจำร่วมกัน
>
> **ความสำคัญกับข้อสอบ:** ถามโมเดลการ map user thread (thread (เธรด (สายงานย่อย)) ฝั่งผู้ใช้) → kernel thread (Many-to-One / One-to-One / Many-to-Many) พร้อมตัวอย่าง OS, อ่านโค้ด Pthread/Java (ภาษาจาวา) ว่าอะไรแชร์กัน และเชื่อมไปบท Synchronization (การประสานงาน)
>
> **ใช้จริงในคอมพิวเตอร์:** เบราว์เซอร์ 1 แท็บใช้หลาย thread (โหลดหน้าเว็บ + เล่นวิดีโอ + รับการคลิก พร้อมกัน) · เกมออนไลน์ (เครือข่าย/กราฟิก/ฟิสิกส์แยกกัน) · Thread Pool (บ่อรวม thread (เธรด (สายงานย่อย)) เตรียมใช้) ใช้ในเว็บเซิร์ฟเวอร์รับคำขอนับพันพร้อมกัน

B1. Thread (เธรด (สายงานย่อย)) คืออะไร

Thread (เธรด (สายงานย่อย)) = ลำดับคำสั่งที่รันได้อิสระภายใน process (กระบวนการ) เดียวกัน (lightweight process (กระบวนการ)) · สิ่งที่แต่ละ thread (เธรด (สายงานย่อย)) มี**ส่วนตัว**: **program counter, registers, stack** · สิ่งที่**แชร์กัน**: code section, data (ข้อมูล) section, heap, ไฟล์ที่เปิด — thread (เธรด (สายงานย่อย)) คือ "หน่วยที่ OS จัดคิวเข้า CPU" ในระบบสมัยใหม่ และ**สลับ thread (เธรด (สายงานย่อย)) ถูกกว่าสลับ process (กระบวนการ)**



สไลด์ OSNo04 p.4 — Single vs Multithreaded: ขวาคือแชร์ code/data (ข้อมูล)/files ร่วมกัน แยกแค่ registers+stack ของแต่ละ thread (เธรด (สายงานย่อย))



สไลด์ OSNo04 p.25 — Java (ภาษาจาวา) Thread States (New/Runnable/Running/Blocked/Terminated)

OS แบบไหน	ตัวอย่าง
Single-process (กระบวนการ) single-threaded	**MS-DOS**
Single-process (กระบวนการ) multi-threaded	**JVM**
Multiprocessing single-threaded	Traditional UNIX (ระบบยูนิกซ์ (บรรพบุรุษของ Linux (OS ลินุกซ์)))
Multiprocessing multi-threaded	**Modern OS: Windows (OS วินโดวส์), Linux (OS ลินุกซ์), macOS**

B2. User Thread (เธรด (สายงานย่อย)) vs Kernel (แกนหลักของ OS) Thread (เธรด (สายงานย่อย)) + Multithreading Models

- **User-Level Thread (ULT)**: จัดการโดย thread (เธรด (สายงานย่อย)) library ฝั่งผู้ใช้ kernel ไม่รู้จัก — เร็วมาก แต่ถ้าตัวใด block (บล็อก (หน่วยจัดเก็บ)) ทั้ง process (กระบวนการ) หยุด
- **Kernel (แกนหลักของ OS)-Level Thread (KLT)**: kernel จัดการตรง — block (บล็อก (หน่วยจัดเก็บ)) ตัวเดียวตัวอื่นยังรัน ขนานจริงได้

Model	Map	จุดเด่น/จุดพัง	ตัวอย่าง
Many-to-One	ULT หลาย → KLT 1	เร็ว (user space) แต่ block (บล็อก (หน่วยจัดเก็บ)) ตัวเดียว = หยุดทั้ง process (กระบวนการ), ขนานบนหลาย CPU ไม่ได้	Early Unix (ระบบยูนิกซ์)
One-to-One	ULT 1 : 1 KLT	ขนานจริง ตัวใด block (บล็อก (หน่วยจัดเก็บ)) ตัวอื่นไม่พัง · สร้าง kernel thread (thread (เธรด (สายงานย่อย)) ที่ kernel รู้จัก) เยอะ = overhead	**Windows (OS วินโดวส์) NT+, Linux (OS ลินุกซ์), Solaris 9+**
Many-to-Many	ULT หลาย → KLT หลาย	OS สร้าง KLT ให้พอดีจำนวน	Solaris < 9, Tru64 UNIX (ระบบยูนิกซ์ (บรรพบุรุษของ Linux (OS ลินุกซ์)))

Many-to-One

- Many user-level threads mapped to single kernel thread
- Examples:
  - Early Unix System

p.13 Many-to-One

One-to-One

- Each user-level thread maps to kernel thread
- Examples
  - Windows starting from NT
  - Linux
  - Solaris 9 and later

p.15 One-to-One

Many-to-Many Model

- Allows many user level threads to be mapped to many kernel threads
- Allows the operating system to create a sufficient number of kernel threads
- Examples
  - Solaris prior to version 9
  - Tru64 UNIX

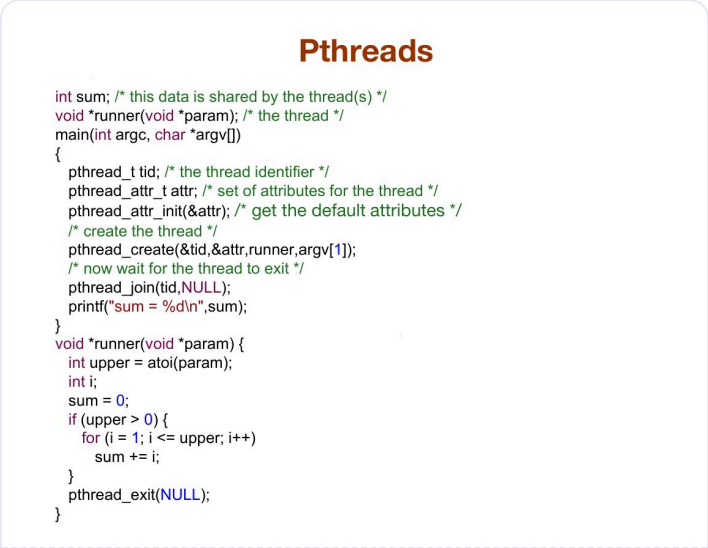
p.17 Many-to-Many

B3. Pthreads (C) — โค้ดตัวอย่างพร้อมคอมเมนต์

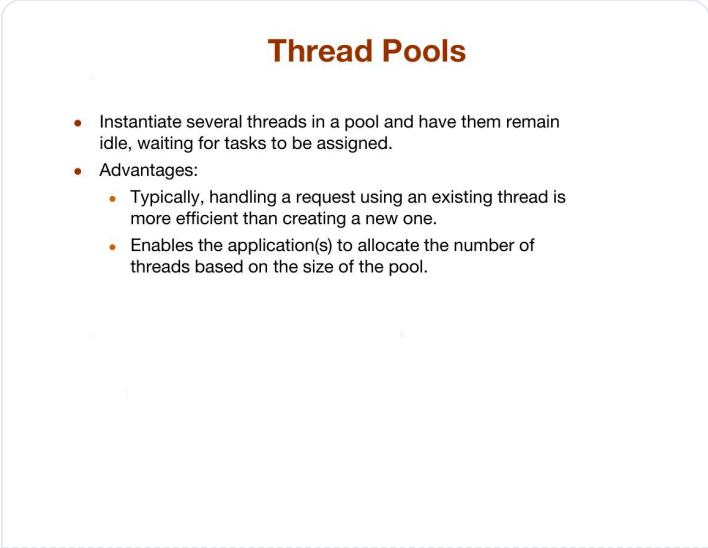
```c
int sum;  // ตัวแปร global — "แชร์" ระหว่าง thread (main กับ runner เห็นตัวเดียวกัน!)

/* ฟังก์ชันที่ thread จะรัน — signature ต้องเป็น void *f(void *) */
void *runner(void *param) {
    int upper = atoi(param);      // รับค่าจาก argv[1] ที่ส่งมาตอน create
    sum = 0;
    for (int i = 1; i <= upper; i++) sum += i;   // หาผลรวม 1..upper
    pthread_exit(NULL);           // จบ thread นี้ (≈ exit ของ process)
}

int main(int argc, char *argv[]) {
    pthread_t tid;                // เก็บ id ของ thread ที่สร้าง
    pthread_attr_t attr;
    pthread_attr_init(&attr);     // ตั้งค่า attribute = default
    pthread_create(&tid, &attr, runner, argv[1]);
    //  ↑ สร้าง thread ให้รันฟังก์ชัน runner โดยส่ง argv[1] เป็น param
    pthread_join(tid, NULL);      // หลักของ main: รอ thread จบก่อน (≈ wait ของ process)
    printf("sum = %d\n", sum);    // เห็นค่า sum ที่ runner แก้ — เพราะแชร์กันจริง
}
/* จับคู่: pthread_create ≈ fork · pthread_join ≈ wait · pthread_exit ≈ exit */
```



สไลด์ OSNo04 p.19 — Pthreads (โค้ดต้นฉบับ)



สไลด์ OSNo04 p.29 — Thread (เธรด (สายงานย่อย)) Pools: สร้าง thread (เธรด (สายงานย่อย)) เตรียมไว้ใน "บ่อ" รับงานเรื่อย ๆ

Thread Pool (Java (ภาษาจาวา) 3 แบบ)

Single thread (เธรด (สายงานย่อย)) executor (บ่อขนาด 1) · **Fixed thread (เธรด (สายงานย่อย)) executor** (ขนาดคงที่) · **Cached thread (เธรด (สายงานย่อย)) pool** (ขนาดไม่จำกัด สร้างเพิ่มเอง) — ประโยชน์: ใช้ thread (เธรด (สายงานย่อย)) เดิมเร็วกว่าสร้างใหม่ + คุมจำนวน thread (เธรด (สายงานย่อย)) ได้

B4. Java (ภาษาจาวา) Thread (เธรด (สายงานย่อย)) — 2 วิธีสร้าง + cancellation

วิธี	โค้ด	หมายเหตุ
extends Thread (เธรด (สายงานย่อย))	`class W extends Thread { public void run() {...} }` → `new W().start()`	เผา slot extends
implements Runnable	`class W implements Runnable { public void run() {...} }` → `new Thread(new W()).start()`	นิยมกว่า

- **Cancellation 2 แบบ: Asynchronous** = ยิงปิดทันที (เสี่ยงข้อมูลครึ่ง ๆ กลาง ๆ) · **Deferred** = บอกเป้าหมายให้เช็คเองเป็นระยะ (`Thread.currentThread().isInterrupted()`) แล้ว cleanup ก่อนจบ — ปลอดภัยกว่า
- `task.join()` = main รอ task จบก่อนเสมอ (เทียบ wait (รอ + หยิบสิทธิ์))

🎯 ประโยคทอง Java (ภาษาจาวา) scheduling (การจัดคิว)

Java (ภาษาจาวา): priority (ลำดับสิทธิ์) **1–10, default (ค่าเริ่มต้น) 5, มาก = สูง** (Unix (ระบบยูนิกซ์) ตรงข้าม: น้อย = สูง) · การตั้ง priority (ลำดับสิทธิ์) **ไม่การันตีลำดับ 100% — ห้ามฝากความถูกต้องของ logic ไว้กับ scheduler (ตัวจัดคิว CPU)** ต้องใช้ Synchronization (การประสานงานระหว่างงาน) เท่านั้น

C. CPU Scheduling (การจัดคิว CPU) — Gantt · TAT · WT (OSNo04 สไลด์ท้าย + วิดีโอ 6.x)

📖 รู้จักส่วนนี้ก่อนอ่าน

ความหมาย: CPU Scheduling (การจัดคิว CPU) = การตัดสินใจว่า process (กระบวนการ) ไหนจะได้ใช้ CPU "ตัวต่อไป" เมื่อหลายงานแย่งกัน — มีหลายอัลกอริทึม เช่น FCFS (มาก่อนได้ก่อน), SJF (งานสั้นก่อน), RR (วนแบ่งเวลา)

ความสำคัญกับข้อสอบ: โจทย์วาด Gantt (แผนภูมิช่วงเวลา) หา TAT/WT เป็นข้อคำนวณหลักประจำทุกปี — ฝึกให้ทำข้อละไม่เกิน 2 นาที ใช้เทคนิค "หา TAT ก่อนแล้วลบ BT เสมอ"

ใช้จริงในคอมพิวเตอร์: Windows (OS วินโดวส์)/Linux (OS ลินุกซ์)/macOS ใช้แนวคิด priority (ลำดับสิทธิ์) + time slice (โควตาเวลาแบบ RR (วนแบ่งเวลา (Round Robin (วนแบ่งเวลาทีละส่วน)))) จริง ๆ ในตัวจัดคิวของเคอร์เนล — เปิด Task Manager เห็น process (กระบวนการ) แย่ง CPU ด้วยกลไกพวกนี้ทั้งหมด

📖 รู้จักส่วนนี้ก่อนอ่าน

ความหมาย: CPU Scheduling (การจัดคิว CPU) = การตัดสินใจว่า process (กระบวนการ) ไหนจะได้ใช้ CPU "ตัวต่อไป" เมื่อมีหลายงานแย่งกัน — มีหลายอัลกอริทึม เช่น FCFS (มาก่อนได้ก่อน), SJF (งานสั้นก่อน), RR (วนแบ่งเวลา)

ความสำคัญกับข้อสอบ: โจทย์คำนวณวาด Gantt (แผนภูมิช่วงเวลา) แล้วหา TAT/WT เป็นข้อคำนวณหลักประจำทุกปี — ฝึกให้ทำข้อละไม่เกิน 2 นาที ใช้เทคนิค "หา TAT ก่อนแล้วลบ BT"

ใช้จริงในคอมพิวเตอร์: Windows (OS วินโดวส์)/Linux (OS ลินุกซ์)/macOS ใช้แนวคิด priority (ลำดับสิทธิ์) + time slice (โควตาเวลาแบบ RR (วนแบ่งเวลา (Round Robin (วนแบ่งเวลาทีละส่วน)))) จริง ๆ ในตัวจัดคิวของเคอร์เนล — เปิด Task Manager เห็น process (กระบวนการ) แย่ง CPU ตามกลไกพวกนี้ทั้งหมด

C1. พื้นฐาน + สูตรทุกตัวที่ใช้คิดเลข

- **CPU Scheduler (ตัวจัดคิว CPU)** เลือก process (กระบวนการ) จาก ready queue ให้ใช้ CPU · **Non-preemptive** = รันจนจบ/ขอออกเอง · **Preemptive (ขัดจังหวะ/แย่งได้)** = ขัดกลางคันได้ (หมดเวลา/มีตัวสำคัญกว่ามา)
- **CPU burst** = ช่วงที่ process (กระบวนการ) ใช้ CPU ติดกัน · I/O-bound = burst สั้นถี่ / CPU-bound = burst ยาว

`Turnaround Time (เวลารวมในระบบ): TAT = Completion − Arrival`

เวลารวมตั้งแต่งานมาถึง → เสร็จ

`Waiting Time (เวลารอคิว): WT = TAT − Burst Time (เวลาที่ต้องใช้ CPU)`

🟢 เทคนิคทำเร็ว: หา TAT ก่อนแล้วลบ BT เสมอ — โดยเฉพาะ preemptive ที่งานโดนแทรกหลายรอบ

`Response Time (เวลาตอบสนอง) = เวลาที่งานมาถึง → เริ่มได้ response ครั้งแรก`

เกณฑ์วัดอื่น: CPU utilization ↑, throughput ↑, TAT ↓, WT ↓, response ↓

C2. FCFS (มาก่อน-รันก่อน) — First-Come, First-Served (non-preemptive)

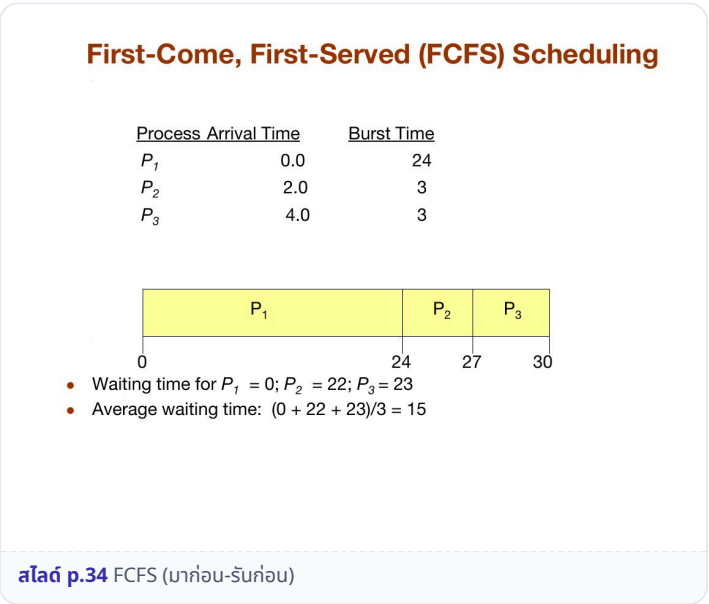
โจทย์ (ตรงจากสไลด์ p.34): P1 (Arrival 0, Burst 24) · P2 (2, 3) · P3 (4, 3)



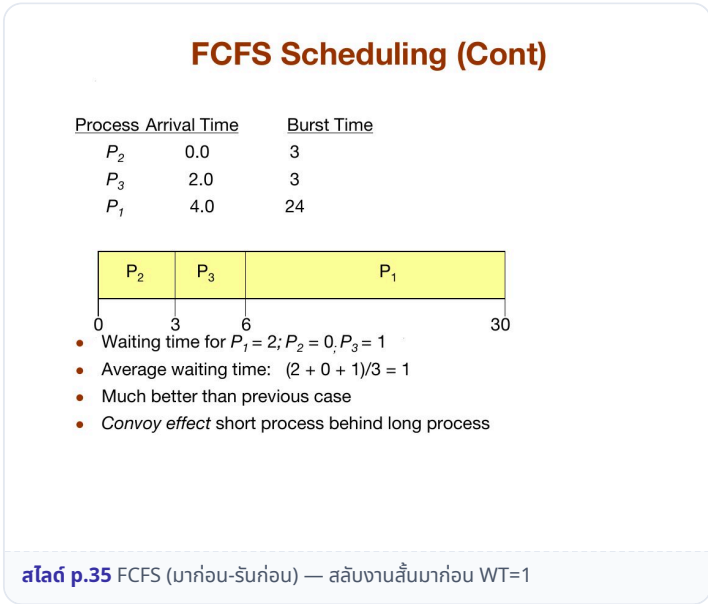
Process (กระบวนการ)	TAT = C − A	WT = TAT − BT
P1	24 − 0 = 24	24 − 24 = **0**
P2	27 − 2 = 25	25 − 3 = **22**
P3	30 − 4 = 26	26 − 3 = **23**
เฉลี่ย	**25**	**(0+22+23)/3 = 15**

🎯 Convoy Effect (อาการคอยตามงานใหญ่)

งานใหญ่ตัวเดียวบล็อกหน้า งานเล็กทั้งบ้านต่อแถวรอ — สไลด์ p.35 เปรียบเทียบ: ถ้างานสั้นมาก่อน (P2 มา 0, P3 มา 2, P1 มา 4) avg WT เหลือ **1** เท่านั้น!



สไลด์ p.34 FCFS (มาก่อน-รันก่อน)



สไลด์ p.35 FCFS (มาก่อน-รันก่อน) — สลับงานสั้นมาก่อน WT=1

C3. SJF (งานสั้นสุดทำก่อน) — Shortest Job (งาน (ในระบบ batch)) First (non-preemptive) — OPTIMAL

โจทย์ (p.36): P1 (0, 7) · P2 (2, 4) · P3 (4, 1) · P4 (5, 4) — เมื่อ CPU ว่าง เลือก BT สั้นสุดในคิว ณ ตอนนั้น (เท่ากันเลือกมาก่อน)

1. เวลา 0: มีแค่ P1 → รันจนจบที่ 7 (non-preemptive ใครมาทีหลังรอก่อน)
2. เวลา 7: คิว P2(4), P3(1), P4(4) → สั้นสุด P3 → 7→8
3. เวลา 8: P2(4), P4(4) เท่ากัน → มาก่อนชนะ: P2 → 8→12
4. เวลา 12: P4 → 12→16



Process (กระบวนการ)	Completion	TAT	WT = TAT − BT
P1	7	7	7−7 = **0**
P2	12	10	10−4 = **6**
P3	8	4	4−1 = **3**
P4	16	11	11−4 = **7**
เฉลี่ย WT	**—**		**(0+6+3+7)/4 = 4**

> 📌 **SJF (งานสั้นสุดทำก่อน) คือ optimal**
> ให้ avg WT ต่ำสุดจากชุดงานหนึ่ง ๆ แต่ใช้จริงยากเพราะ OS ต้อง**ทำนาย**ความยาว CPU burst ถัดไป

C4. SRTF (งานที่เหลือสั้นสุดทำก่อน) — Shortest Remaining Time First (Preemptive (ขัดจังหวะ/แย่งได้) SJF (งานสั้นสุดทำก่อน)) 🔥

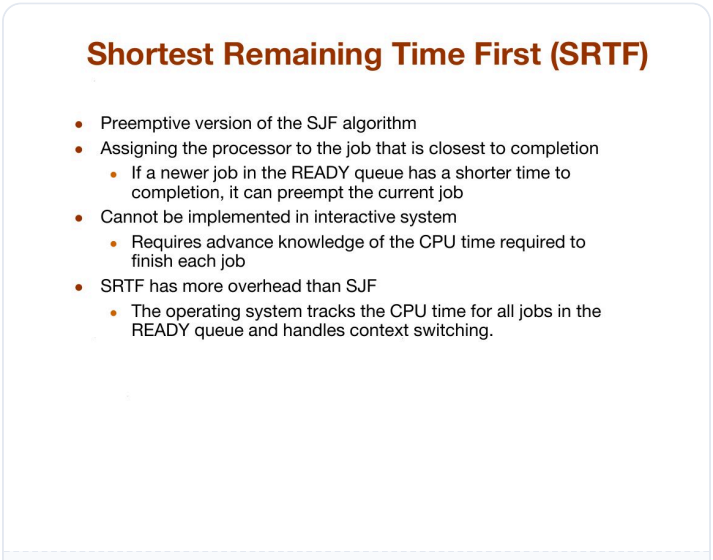
กติกา: **งานใหม่มาถึง → เทียบ BT ของงานใหม่ กับ Remaining Time (เวลาที่เหลือ) ของงานที่รันอยู่** — งานใหม่สั้นกว่า = ขัดทันที

1. 0–2: P1 รันไป 2 (เหลือ 5) — P2 มาถึง (BT=4) → เทียบ 4 < 5 → **ขัด P1**
2. 2–4: P2 รันไป 2 (เหลือ 2) — P3 มาถึง (BT=1) → 1 < 2 → **ขัด P2**
3. 4–5: P3 จบที่ 5 พอดี + P4 มาถึง (BT=4) → คิว: P1(5), P2(2), P4(4) → รัน P2 จบที่ 7
4. 7: เหลือ P1(5), P4(4) → P4 จบที่ 11 → P1 เศษ 5 จบที่ 16

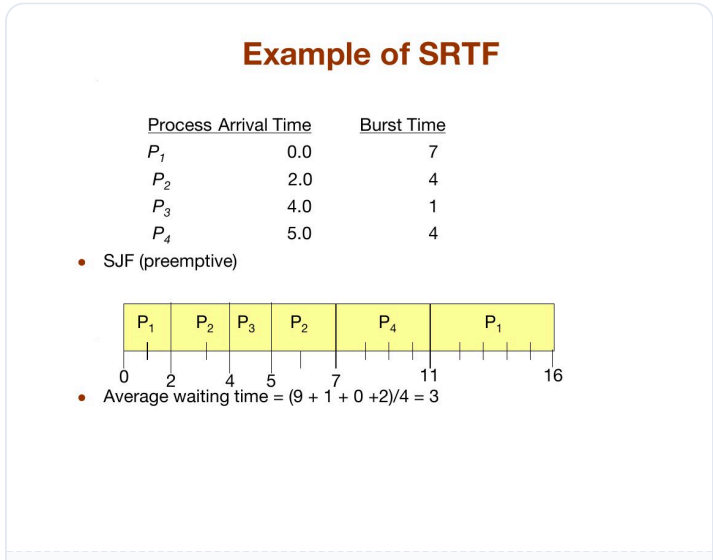


Process (กระบวนการ)	Completion	TAT = C − A	WT = TAT − BT
P1	16	16	16−7 = **9**
P2	7	5	5−4 = **1**
P3	5	1	1−1 = **0**
P4	11	6	6−4 = **2**
เฉลี่ย WT	**—**		**(9+1+0+2)/4 = 3**

> 🎯 **จุดหลอก SRTF (งานที่เหลือสั้นสุดทำก่อน)**
> เทียบด้วย **Remaining** เสมอ ไม่ใช่ BT ดั้งเดิม! (ตอนเวลา 4 ต้องเทียบ "P2 เหลือ 2" ไม่ใช่ "P2 = 4") · SRTF (งานที่เหลือสั้นสุดทำก่อน) overhead มากกว่า SJF (งานสั้นสุดทำก่อน) เพราะ context switch บ่อย



สไลด์ p.38 SRTF (งานที่เหลือสั้นสุดทำก่อน) นิยาม



สไลด์ p.39 SRTF (งานที่เหลือสั้นสุดทำก่อน) example (เฉลยตรงกับตารางข้างบน)

C5. Priority (ลำดับสิทธิ์) Scheduling (การจัดคิว)

- CPU ให้งาน priority (ลำดับสิทธิ์) สูงสุดก่อน (มีทั้ง preemptive/non-preemptive) · SJF (งานสั้นสุดทำก่อน) = priority (ลำดับสิทธิ์) ที่ใช้ "predicted next CPU burst" เป็น priority (ลำดับสิทธิ์)

- **ปัญหา Starvation (การอดตาย (ไม่ได้รันเลย))** (งานสิทธิต่ำไม่ได้รัน) → **แก้ด้วย Aging**: ยิ่งรอนานยิ่งเพิ่ม priority (ลำดับสิทธิ์)
- **ทิศทางตัวเลข:** Unix (ระบบยูนิกซ์)-based → **เลขน้อย = สิทธิสูง** / Java (ภาษาจาวา) Thread (เธรด (สายงานย่อย)) → **เลขมาก = สูง** (10 สูงสุด, default (ค่าเริ่มต้น) 5)

C6. Round Robin (RR (วนแบ่งเวลา (Round Robin (วนแบ่งเวลาทีละส่วน)))) — preemptive, ตอบสนองไว

แต่ละงานรันไม่เกิน **Time Quantum (q)** หมดเวลายังไม่เสร็จ → ถูกตัดไปต่อท้ายคิว · คิวมี n งาน → ไม่มีงานรอเกิน **(n−1)q**

โจทย์ (p.41–42): ข้อมูล FCFS (มาก่อน-รันก่อน) เดิม (P1: 0,24 · P2: 2,3 · P3: 4,3) q = 4



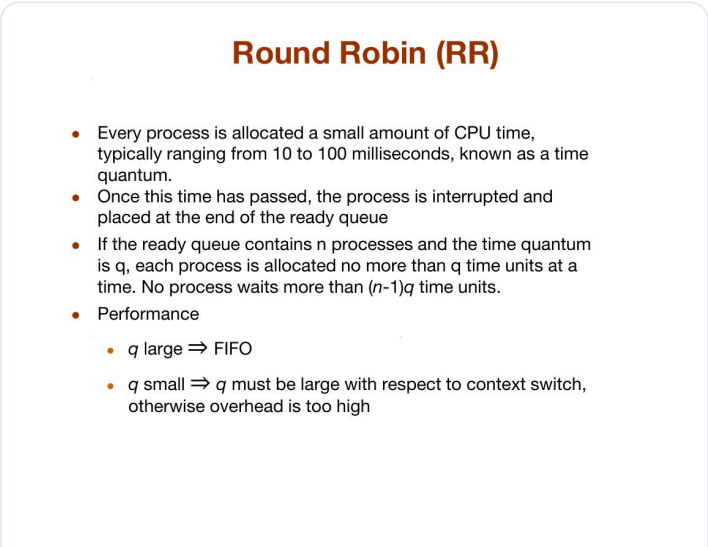
Process (กระบวนการ)	TAT = C − A	WT = TAT − BT
P1	30 − 0 = 30	30 − 24 = **6**
P2	7 − 2 = 5	5 − 3 = **2**
P3	10 − 4 = 6	6 − 3 = **3**
เฉลี่ย WT	**—**	**11/3 ≈ 3.67**

⚖️ ขนาด quantum

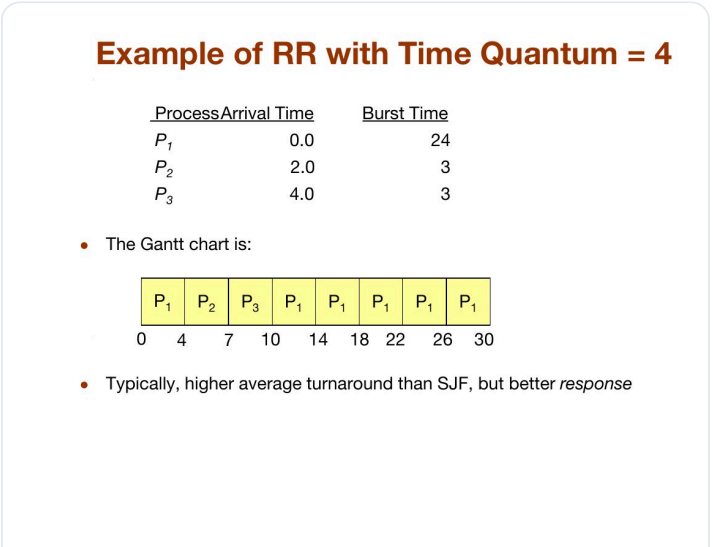
q ใหญ่มาก → RR (วนแบ่งเวลา (Round Robin (วนแบ่งเวลาทีละส่วน))) กลายเป็น **FCFS (มาก่อน-รันก่อน)** · q เล็กมาก → **context switch ถี่เกิน overhead โต** (ช่วง switch = pure overhead)

ไทม์ไลน์ RR (วนแบ่งเวลา (Round Robin (วนแบ่งเวลาทีละส่วน))) ทีละจังหวะ

0–4 P1(เหลือ 20) → 4–7 P2 จบ → 7–10 P3 จบ → 10–30 P1 ตัวเดียว รันรอบละ 4 จนครบ 24



สไลด์ p.41 Round Robin (วนแบ่งเวลาทีละส่วน)



สไลด์ p.42 Example RR (วนแบ่งเวลา (Round Robin (วนแบ่งเวลาทีละส่วน))) q=4 (เฉลยตรงกับตาราง)

C7. Multilevel Queue (คิวหลายชั้น) & Multilevel Feedback Queue (MLFQ)

- **Multilevel Queue (คิวหลายชั้น)**: แยก ready queue หลายคิว (เช่น foreground=RR (วนแบ่งเวลา (Round Robin (วนแบ่งเวลาทีละส่วน))) / background=FCFS (มาก่อน-รันก่อน)) — **งานอยู่คิวไหนตลอดชีวิต ย้ายไม่ได้** · ระหว่างคิว: fixed priority (คิวหน้าจบก่อน — เสี่ยง starvation) หรือแบ่งเวลา (80/20)
- **MLFQ**: **งานย้ายคิวได้** → ทำ aging ได้ จัดกลุ่มตามพฤติกรรมได้

โมเดลตัวอย่าง (p.46–47): Q0 = RR (วนแบ่งเวลา (Round Robin (วนแบ่งเวลาทีละส่วน))) q8ms → ไม่จบ ลง Q1 = RR (วนแบ่งเวลา (Round Robin (วนแบ่งเวลาทีละส่วน))) q16ms → ยังไม่จบ ลง Q2 = FCFS (มาก่อน-รันก่อน) จนจบ · คิวล่างได้รันเมื่อคิวบนว่างหมด

Multilevel Queue Scheduling

- Multilevel Queue Scheduling is a scheduling algorithm that partitions the ready queue into several separate queues.
- Each queue can have its own scheduling algorithm. For Examples
  - foreground – RR
  - background – FCFS
- This method is used when processes can be easily categorized into different classes with different scheduling needs.
- No Movement Between Queues: Processes do not move between queues. Once assigned to a queue, a process stays there.

สไลด์ p.44 Multilevel Queue (คิวหลายชั้น)

Multilevel Feedback Queue Scheduling

- A process can move between the various queues; aging can be implemented this way
- Multilevel-feedback-queue scheduler defined by the following parameters:
  - number of queues
  - scheduling algorithms for each queue
  - method used to determine when to upgrade a process
  - method used to determine when to demote a process
  - method used to determine which queue a process will enter when that process needs service

สไลด์ p.46 MLFQ นิยาม

Example of Multilevel Feedback Queue

- Three queues:
  - Q_0 – RR with time quantum 8 milliseconds
  - Q_1 – RR time quantum 16 milliseconds
  - Q_2 – FCFS
- Scheduling
  - A new job enters queue Q_0 which is served FCFS. When it gains CPU, job receives 8 milliseconds. If it does not finish in 8 milliseconds, job is moved to queue Q_1.
  - At Q_1 job is again served FCFS and receives 16 additional milliseconds. If it still does not complete, it is preempted and moved to queue Q_2.

สไลด์ p.47 Example MLFQ 3 คิว

🎯 สรุป 5 จุดพลาดของบทนี้

1) ลบ Arrival ไม่ครบ (เอาเวลาเริ่มรันตอบเป็น WT) 2) WT ใน preemptive ไล่นับมือผิด → ใช้ WT=TAT−BT 3) ตี priority (ลำดับสิทธิ์) ผิดฝั่ง 4) SRTF (งานที่เหลือสั้นสุดทำก่อน) ใช้ BT ดิบแทน remaining 5) เลือก quantum ผิด

D. Race Condition (สภาพแข่งขันของข้อมูล) & Critical Section (ช่วงวิกฤต (โค้ดแตะของแชร์)) (OSNo06 ต้น ๆ)

> 📖 **รู้จักส่วนนี้ก่อนอ่าน**
>
> **ความหมาย:** Race Condition (สภาพแข่งขันของข้อมูล) = ปัญหาที่ 2 process (กระบวนการ) แก้ข้อมูลแชร์พร้อมกันจนผลลัพธ์เพี้ยนตามใครแก้ก่อน-หลัง · Critical Section (ช่วงวิกฤต) = ช่วงโค้ดที่แตะข้อมูลแชร์ ห้ามใครเข้าพร้อมกัน
>
> **ความสำคัญกับข้อสอบ:** ไล่ timeline การแย่งตัวแปร count (S0–S5), ท่องเงื่อนไข 3 ข้อ (Mutual Exclusion (เข้าถึงได้คนเดียว) / Progress / Bounded Waiting (รอได้จำกัดจำนวนครั้ง)), อธิบาย Peterson's Solution (โซลูชัน 2 process (กระบวนการ) ของ Peterson) ว่าครบ 3 ข้อยังไง — ออกทั้งถูก/ผิดและไล่ค่า
>
> **ใช้จริงในคอมพิวเตอร์:** ระบบธนาคารที่โอนเงินพร้อมกัน 2 คน, ตั๋วหนังที่นั่งสุดท้ายถูกซื้อพร้อมกัน, ตัวนับยอดวิว — ถ้าไม่ล็อก ข้อมูลเสียแบบใน timeline เลย

D1. จุดเริ่ม: Bounded Buffer (บัฟเฟอร์ขนาดจำกัด) ยังไม่มี sync (Producer (ฝั่งผู้ผลิตข้อมูล)-Consumer (ฝั่งผู้บริโภคข้อมูล))

Producer (ฝั่งผู้ผลิตข้อมูล)-Consumer (ฝั่งผู้บริโภคข้อมูล) แชร์ buffer วงแหวนขนาด BUFFER_SIZE ช่อง + ตัวนับ `count` — producer เขียนแล้ว count (ตัวนับจำนวน)++ / consumer อ่านแล้ว count (ตัวนับจำนวน)−−

```
/* Producer (สไลด์ p.3)                    Consumer (สไลด์ p.4) */
while (true) {                          while (true) {
    while (count == BUFFER_SIZE);           while (count == 0);   // busy wait
    buffer[in] = nextProduced;              nextConsumed = buffer[out];
    in = (in + 1) % BUFFER_SIZE;            out = (out + 1) % BUFFER_SIZE;
    count++;                                count--;
}                                       }
/* ปัญหา: ข้อมูลหาย (producer เขียนทับที่ยังไม่อ่าน) / อ่านซ้ำ — และ count ไม่ atomic */
```

D2. Race Condition (สภาพแข่งขันของข้อมูล) — ไล่ S0–S5 ให้เห็นตา 🔥

`count++` ในระดับเครื่องแตกเป็น 3 คำสั่ง (ไม่ atomic!):

```
/* count++: */              /* count--: */
register1 = count;          register2 = count;
register1 = register1 + 1;  register2 = register2 - 1;
count = register1;          count = register2;
```

ไทม์ไลน์จากสไลด์ p.5 (เริ่ม count (ตัวนับจำนวน) = 5):

จังหวะ	ใครทำอะไร	ค่าที่เกี่ยวข้อง
S0	producer: register1 = count (ตัวนับจำนวน)	r1 = 5
S1	producer: r1 = r1 + 1	r1 = 6
S2	consumer: register2 = count (ตัวนับจำนวน)	r2 = **5** ⚠️ (ยังเก่า!)
S3	consumer: r2 = r2 − 1	r2 = 4
S4	producer: count (ตัวนับจำนวน) = r1	count (ตัวนับจำนวน) = 6
S5	consumer: count (ตัวนับจำนวน) = r2	count (ตัวนับจำนวน) = **4** ❌ (ควรได้ 5)

> 💳 **อุปมา**
> เงินในบัญชี 5,000 · คุณโอนเข้า +1,000 แฟนถอน −1,000 พร้อมกัน แอปให้ทั้งคู่เห็นยอด 5,000 พร้อมกันแล้วบันทึกทับ → เหลือ 4,000 เงินหายในอากาศ!

> 📖 **นิยามที่ต้องพิมพ์ได้**
> **Race condition**: หลาย process (กระบวนการ) เข้าถึง/จัดการข้อมูลร่วมกันพร้อมกัน แล้ว**ผลลัพธ์ขึ้นกับลำดับการเข้าถึง** · โค้ดช่วงที่แตะข้อมูลแชร์ = **critical section (critical region)**

D3. Critical Section (ช่วงวิกฤต (โค้ดแตะของแชร์)) — เงื่อนไข 3 ข้อที่โซลูชันต้องผ่าน 🔥

เงื่อนไข	ความหมาย (พิมพ์ได้ต้องได้)
1. Mutual Exclusion (เข้าถึงได้คนเดียว)	ถ้า P_i อยู่ใน CS (ช่วงวิกฤต) → **ห้ามมี process (กระบวนการ) อื่นอยู่ใน CS (ช่วงวิกฤต) ของมันพร้อมกัน**
2. Progress	ถ้า*ไม่มีใคร*อยู่ใน CS (ช่วงวิกฤต) และมีคนอยากเข้า → ต้องเลือกจาก*คนที่อยากเข้าเท่านั้น* และตัดสินไม่ล่าช้าไม่สิ้นสุด (กันคนไม่สนมาขวาง / กันไม่มีใครเข้าเลย)
3. Bounded Waiting (รอได้จำกัดจำนวนครั้ง)	มีขอบเขตว่า process (กระบวนการ) หนึ่งถูก "ตัดหน้า" ได้**จำกัดกี่ครั้ง**หลังยื่นขอ (กัน starvation ของคิว CS (ช่วงวิกฤต))

โครงโค้ด: ENTRY SECTION → CRITICAL SECTION → EXIT SECTION → REMAINDER SECTION (วน while(true)) · สมมติ: แต่ละ process (กระบวนการ) รันด้วยความเร็วไม่เป็นศูนย์ แต่**ไม่มีข้อสมมติเรื่องความเร็วสัมพัทธ์**

D4. Peterson's Solution (2 process (กระบวนการ))

แชร์ 2 ตัวแปร: `int turn` (ตาใคร) + `boolean flag[2]` (ใครอยากเข้า) — สมมติ LOAD/STORE **atomic**

```
/* โค้ดของ Pᵢ (อีกฝั่งคือ Pⱼ) — สไลด์ p.9–10 */
while (true) {
    flag[i] = TRUE;              // 1. ประกาศว่า "ฉันอยากเข้า!"
    turn = j;                    // 2. ยกสิทธิ์ให้เพื่อนก่อน (ความสุภาพ = หัวใจ)
    while (flag[j] && turn == j); // 3. รอเฉพาะเมื่อ: เพื่อนอยากเข้า "และ" ถึงตาเพื่อน

    CRITICAL SECTION

    flag[i] = FALSE;             // 4. ออกแล้ว — ปล่อยให้คนอื่นเข้าได้
    REMAINDER SECTION
}
```

เงื่อนไข	Peterson ทำไม่ผ่านทำไมยังไง
Mutual Exclusion (เข้าถึงได้คนเดียว) ✓	ทั้งคู่ตั้ง turn พร้อมกันไม่ได้ — คนที่ตั้ง turn เป็นฝ่ายอื่น**เป็นคนสุดท้าย** ต้องรอก่อน
Progress ✓	เงื่อนไขรอต้องเจอ flag[j] = true จริง — คนที่ไม่สนใจเข้าไม่ขวาง
Bounded Waiting (รอได้จำกัดจำนวนครั้ง) ✓	หลังออกจาก CS (ช่วงวิกฤต) อีกฝั่งได้เข้าภายใน 1 รอบ (turn สลับเสมอ)

> 📌 **ในโลกจริง**
> Peterson เวิร์คในทฤษฎี แต่ CPU สมัยใหม่ (compiler optimization, out-of-order) อาจพัง — ใช้จริงด้วยฮาร์ดแวร์/ไลบรารี แต่**ออกสอบเยอะ**เพราะสอนหลักคิด

D5. Bakery Algorithm (n process (กระบวนการ)) — จ่ายคิวร้านเบเกอรี่

- ก่อนเข้า CS (ช่วงวิกฤต) จ่ายเลขคิว = max(number[0..n−1]) + 1 → คนถือเลข**น้อยสุด**เข้าก่อน · เลขซ้ำเทียบ **PID** มาก่อนชนะ

- เทียบแบบ lexicographical: `(a,b) < (c,d)` ถ้า a < c หรือ (a = c และ b < d) — (เลขคิว, pid)
- แชร์: `boolean choosing[n]`, `int number[n]` (เริ่ม false/0)

```
do {
    choosing[i] = true;                          // กำลังจ่ายเลข (ยังไม่นับคิวจริง)
    number[i] = max(number[0..n-1]) + 1;          // จ่ายเลขใหญ่กว่าใคร ๆ +1 (อาจซ้ำ)
    choosing[i] = false;
    for (j = 0; j < n; j++) {
        while (choosing[j]);                     // รอคนที่กำลังจ่ายเลขจบก่อน
        while ((number[j] != 0) && ((number[j],j) < (number[i],i)));
        // รอถ้า: เพื่อนถือคิวอยู่ และ (เลขน้อยกว่า หรือ เลขเท่าแต่ pid น้อยกว่า)
    }
    CRITICAL SECTION
    number[i] = 0;                               // ออก → ยกเลิกคิวตัวเอง
    REMAINDER SECTION
} while (true);
```

เลขคิวอาจกระโดด (1,2,3,3,3,4) — ไม่เป็นไร lexicographical จัดการเคสซ้ำ · รับประกันครบ 3 เงื่อนไขสำหรับ n process (กระบวนการ)

E. Semaphore (ตัวนับควบคุมสิทธิ์เข้าถึง) · ปัญหาคลาสสิก · Monitor · Java (ภาษาจาวา) Synchronization (การประสานงานระหว่างงาน) (OSNo06 ท้าย ๆ) 🔥

📖 รู้จักส่วนนี้ก่อนอ่าน

ความหมาย: Semaphore (ตัวนับควบคุมคิว) = เครื่องมือที่ OS ให้มาจัดคิวเข้าถึงของแชร์อย่างปลอดภัย มีคำสั่ง wait (หยิบสิทธิ์) / signal (คืนสิทธิ์) · Monitor = กล่องควบคุมระดับสูงที่ล็อกให้อัตโนมัติ

ความสำคัญกับข้อสอบ: โค้ด Bounded Buffer (บัฟเฟอร์ขนาดจำกัด) / Readers-Writers (ปัญหาคนอ่าน-คนเขียน) / Dining Philosophers (ปัญหานักปราชญ์กินข้าว) ต้องจำค่าเริ่มต้นให้แม่น (mutex=1, empty=n, full=0) + จุดหลอก "สลับลำดับ wait (รอ + หยิบสิทธิ์) → deadlock (การติดตาย (รอกันตลอดกาล))" — ออกถี่มาก

ใช้จริงในคอมพิวเตอร์: คิวงานเครื่องปริ้นต์, connection pool ของฐานข้อมูล, ปุ่ม like ที่คนกดพร้อมกันหลายพันคน — ระบบจริงทั้งหมดใช้ semaphore (ตัวนับควบคุมสิทธิ์เข้าถึง)/mutex คุมจังหวะ

E1. Semaphore (ตัวนับควบคุมสิทธิ์เข้าถึง) — wait (รอ + หยิบสิทธิ์) / signal (อุปมาโหลหิน 🪨)

```
wait(S) {                // P — "หยิบหินจากโหล"
    while (S <= 0);      // โหลว่าง → ยืนรอ (busy wait)
    S--;
}
signal(S) {              // V — "คืนหินใส่โหล" แล้วปลุกคนรอ
    S++;
}
```

- **Binary semaphore (ตัวนับควบคุมสิทธิ์เข้าถึง)** = ค่า 0/1 เท่านั้น อีกชื่อคือ **mutex lock (การล็อก)** → ใช้กับ critical section (เข้าได้ทีละคน)
- **Counting semaphore (ตัวนับควบคุมสิทธิ์เข้าถึง)** = ค่า 0..k → ใช้กับ resource (ทรัพยากร) หลายหน่วย (k = จำนวน resource (ทรัพยากร))

🎯 กับดักจาก Quizizz ชุด 3

(1) CS (ช่วงวิกฤต) ใช้ **binary** ไม่ใช่ counting (2) Semaphore (ตัวนับควบคุมสิทธิ์เข้าถึง) การันตีได้แค่ **Mutual Exclusion (เข้าถึงได้คนเดียว)** — progress/bounded waiting ขึ้นกับ implementation

E2. Implement แบบไม่มี busy waiting (waiting queue + block (บล็อก (หน่วยจัดเก็บ))/wakeup)

```
/* แต่ละ semaphore มี waiting queue (แต่ละรายการเก็บ value + pointer รายการถัดไป) */
wait(S) {
    S.value--;                    // หยิบหินก่อนเลย
    if (S.value < 0) {            // ติดลบ = หินไม่พอ มีคนรอแล้ว
        เพิ่มตัวเองเข้า S.list;   // ต่อคิว
        block();                  // หลับ — ไม่เผา CPU
    }
}
signal(S) {
    S.value++;                    // คืนหิน
    if (S.value <= 0) {           // ยังมีคนรออยู่
        หยิบ P จาก S.list;
        wakeup(P);                // ปลุก 1 คนกลับ ready queue
    }
}
/* 💡 ค่า value ติดลบ = มี process รออยู่ |value| ตัว */
```

E3. ปัญหาคลาสสิก 1: Bounded Buffer (บัฟเฟอร์ขนาดจำกัด) 🔥 (semaphore (ตัวนับควบคุมสิทธิ์เข้าถึง) 3 ตัว)

Semaphore (ตัวนับควบคุมสิทธิ์เข้าถึง)	เริ่มต้น	ชนิด	หน้าที่
`mutex`	**1**	binary	ล็อก buffer เข้าแตะทีละคน
`empty`	**n**	counting	นับช่องว่าง
`full`	**0**	counting	นับช่องที่มีข้อมูล

```
/* Producer (สไลด์ p.20–21) */
while (true) {
    // produce an item
    wait(empty);   // ขอช่องว่าง
    wait(mutex);   // ล็อก buffer
    // เขียนลง buffer
    signal(mutex); // ปลดล็อก
    signal(full);  // บอกว่ามีของแล้ว
}
```

```
/* Consumer (สไลด์ p.21) */
while (true) {
    wait(full);    // มีของไหม?
    wait(mutex);
    // ดึงของออกจาก buffer
    signal(mutex);
    signal(empty); // คืนช่องว่าง
    // consume item
}
```

🎯 จุดหลอก: สลับ wait (รอ + หยิบสิทธิ์) → DEADLOCK

ถ้า producer เขียน `wait(mutex); wait(empty);` — buffer เต็ม → producer ครอง mutex ค้างรอ empty แต่ consumer เข้าอ่านไม่ได้เพราะ mutex ถูกครอง → รอกันตาย · **กฎเหล็ก: เช็ค resource (empty/full) ก่อนล็อก (mutex) เสมอ** · จำค่าเริ่มต้นให้แม่น: mutex=1, empty=n, full=0

E4. ปัญหาคลาสสิก 2: Readers-Writers (ปัญหาคนอ่าน-คนเขียน)

กติกา: **Reader หลายตัวอ่านพร้อมกันได้** · Writer ทำงานเมื่อไร้ห้ามใครยุ่งเด็ดขาด · แชร์: `wrt=1, mutex=1, readcount=0 (int ธรรมดา)`

```
/* Writer — ง่ายสุด */
wait(wrt);
// เขียนข้อมูล
signal(wrt);
```

```
/* Reader — "ตัวแรกล็อก ตัวสุดท้ายปลด" */
wait(mutex);              // แตะ readcount ทีละคน
readcount++;
if (readcount == 1)       // reader แรกล็อก writer
    wait(wrt);
signal(mutex);
// อ่านข้อมูล (พร้อมกันได้หลายคน)
wait(mutex);
readcount--;
if (readcount == 0)       // reader สุดท้ายปลดล็อก
    signal(wrt);
signal(mutex);
```

🎯 จาก Quizizz 3

semaphore (ตัวนับควบคุมสิทธิ์เข้าถึง) ที่ควบคุมการเพิ่ม/ลดจำนวน reader คือ **mutex** (ครอบ readcount) — **readcount เป็น int ธรรมดา** ไม่ใช่ semaphore (ตัวนับควบคุมสิทธิ์เข้าถึง) · ที่คุม writer คือ wrt

E5. ปัญหาคลาสสิก 3: Dining Philosophers (นักปราชญ์ 5 คน)

ตะเกียบ 5 อัน = **resource (ทรัพยากร)** ใช้ semaphore (ตัวนับควบคุมสิทธิ์เข้าถึง) 1 ตัวต่อ 1 ตะเกียบ เริ่มทุกตัว = 1 · กินได้ต้องมีครบ 2 ข้าง (ซ้าย i, ขวา (i+1)%5)

```
/* โค้ดนักปราชญ์ i (สไลด์ p.25–26) */
while (true) {
    wait(chopstick[i]);              // หยิบซ้าย
    wait(chopstick[(i + 1) % 5]);    // หยิบขวา
    // กินข้าว 🍚
    signal(chopstick[i]);            // วางซ้าย
    signal(chopstick[(i + 1) % 5]); // วางขวา
    // คิดปรัชญา
}
```

🎯 โซลูชัน naive นี้พัง

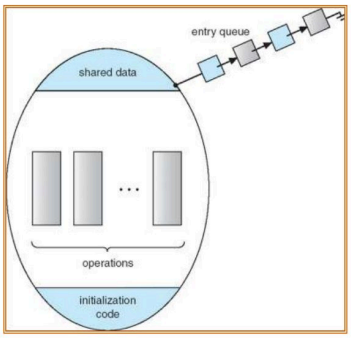
ทุกคน**หิวพร้อมกัน หยิบซ้ายพร้อมกัน** → ไม่เหลือตะเกียบว่าง → ทุกคนรอขวาเป็นวงกลม = **deadlock (การติดตาย (รอกันตลอดกาล))** (+ ไม่มี bounded waiting → starvation ได้) · แก้: จำกัดจำนวนคนนั่ง ≤ 4 / หยิบพร้อมกันเท่านั้น / ใช้ Monitor (ล่าง)

E6. Monitor — เกราะระดับสูง + Condition (เงื่อนไข) Variables

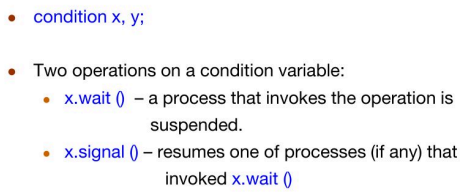
- Monitor = shared data (ข้อมูล) + ฟังก์ชันรวมในก้อนเดียว (คล้าย class) — **รับประกัน mutual exclusion อัตโนมัติ: 1 process (กระบวนการ) ใน monitor ต่อ 1 เวลา** (ลืมล็อกไม่ได้!)
- **Condition (เงื่อนไข) variable x:** `x.wait()` = สลัดล็อก หลับไปรอในคิวของ x · `x.signal()` = ปลุก 1 ตัวในคิว x

🎯 ต่างจาก semaphore (ตัวนับควบคุมสิทธิ์เข้าถึง) ยังไง

x.signal() ถ้าไม่มีใครนอน → ***สัญญาณหายไปเฉย ๆ*** (ไม่มี counter เก็บยอด) ต่างจาก semaphore (ตัวนับควบคุมสิทธิ์เข้าถึง) ที่ signal (สัญญาณแจ้งเหตุการณ์) สะสมค่า



สไลด์ p.29 Schematic view of a Monitor: คิว entry → ใน monitor มีได้ 1 คน



สไลด์ p.30 Condition (เงื่อนไข) Variables x.wait() / x.signal()

E6.1 แก้ Dining Philosophers (ปัญหานักปราชญ์กินข้าว) ด้วย Monitor (กัน deadlock (การติดตาย (รอกันตลอดกาล)) 100%)

```
monitor DP {
    enum {THINKING, HUNGRY, EATING} state[5];   // สถานะของแต่ละคน
    condition self[5];                          // คิวรอของแต่ละคน

    void pickup(int i) {
        state[i] = HUNGRY;
        test(i);                                // ลองดูว่ากินได้ไหม
        if (state[i] != EATING) self[i].wait;   // ยังไม่ได้ → หลับรอ
    }
    void putdown(int i) {
        state[i] = THINKING;
        test((i + 4) % 5);                      // เช็คเพื่อนบ้านซ้าย
        test((i + 1) % 5);                      // เช็คเพื่อนบ้านขวา
    }
    void test(int i) {
        // กินได้เมื่อ: เพื่อนซ้ายไม่กิน และ ฉันหิว และ เพื่อนขวาไม่กิน
        if ( (state[(i + 4) % 5] != EATING) &&
             (state[i] == HUNGRY) &&
             (state[(i + 1) % 5] != EATING) ) {
            state[i] = EATING;
            self[i].signal();
        }
    }
}
/* ใช้: dp.pickup(i) → EAT → dp.putdown(i)
   ไม่มีใคร "ถือตะเกียบเดี่ยวค้าง" → ไม่มี deadlock แน่นอน (แต่ starvation ยังเป็นไปได้) */
```

E7. Java (ภาษาจาวา) Synchronization (การประสานงานระหว่างงาน) — synchronized (ล็อกอัตโนมัติ (คีย์เวิร์ด Java (ภาษาจาวา))) · wait (รอ + หยิบสิทธิ์) · notify (ปลุก 1 thread (เธรด (สายงานย่อย)) ที่รออยู่)

- ทุก object มี **lock (การล็อก)** — เรียก `synchronized` method ต้องได้ lock (การล็อก) ก่อน · ตกคิวไปรอใน **Entry Set (คิวรอเข้าล็อก)** · ออกจาก method = คืน lock (การล็อก) อัตโนมัติ
- `wait()` ทำ 3 อย่าง: **(1) คืน object lock (2) สถานะ blocked (3) ย้ายเข้า Wait Set (คิวหลับรอเงื่อนไข)**
- `notify()` = **สุ่ม** 1 thread (เธรด (สายงานย่อย)) จาก Wait Set (คิวหลับรอเงื่อนไข) → ย้ายไป Entry Set (รอแย่ง lock (การล็อก) ใหม่ — **ไม่ได้รันทันที!** และเลือกตัวเองไม่ได้) · `notifyAll()` = ปลุก**ทุกตัว** (ปลอดภัยกว่าเมื่อ wait (รอ + หยิบสิทธิ์) set มีหลายตัว)
- Java (ภาษาจาวา) 5+ มี `Semaphore` class จริง: `acquire()` ≈ wait (รอ + หยิบสิทธิ์), `release()` ≈ signal (สัญญาณแจ้งเหตุการณ์)

```
/* insert() ด้วย wait/notify (สไลด์ p.41) */
public synchronized void insert(Object item) {
   while (count == BUFFER_SIZE) {
       try { wait(); }                  // เต็ม → คืนล็อก หลับรอ
       catch (InterruptedException e) { }
   }
   ++count;
   buffer[in] = item;
   in = (in + 1) % BUFFER_SIZE;
   notify();                            // ปลุกคนที่รออยู่ 1 ตัว
}
```

```
/* remove() (สไลด์ p.42) */
public synchronized Object remove() {
   Object item;
   while (count == 0) {
       try { wait(); }                  // ว่าง → หลับรอ
       catch (InterruptedException e) { }
   }
   --count;
   item = buffer[out];
   out = (out + 1) % BUFFER_SIZE;
   notify();
   return item;
}
```

🎯 ตารางแยกพวก "wait (รอ + หยิบสิทธิ์)" ที่หลอกบ่อยสุด

wait (รอ + หยิบสิทธิ์) ของใคร	ทำอะไร
`wait(S)` ของ Semaphore (ตัวนับควบคุมสิทธิ์เข้าถึง)	หยิบหิน — ลดค่า S
`wait()` ของ Java (ภาษาจาวา)/Monitor	สลัด lock (การล็อก) + หลับใน Wait Set (คิวหลับรอเงื่อนไข)
`wait()` ของ Process (บท A)	พ่อรอลูกจบ เก็บ exit status

แถม: `Thread.yield()` แค่ยอมออกจาก CPU — **ยังถือ lock (การล็อก) ค้างอยู่!** (เหมือนล็อกห้องแล้วปีนหน้าต่างหนี)

F. Deadlock (การติดตาย (รอกันตลอดกาล)) + Banker's Algorithm (อัลกอริทึมนายธนาคาร) (OSNo07) 🔥

> 📖 **รู้จักส่วนนี้ก่อนอ่าน**
>
> **ความหมาย:** Deadlock (การติดตาย) = กลุ่ม process (กระบวนการ) รอทรัพยากรกันเป็นวงกลม ไม่มีใครได้ทำงานต่ออีกเลย · Banker's Algorithm (อัลกอริทึมนายธนาคาร) = วิธี "เช็กก่อนให้" ว่าจัดสรรแล้วระบบยังปลอดภัยไหม
>
> **ความสำคัญกับข้อสอบ:** เงื่อนไข 4 ข้อ (ต้องครบพร้อมกัน), อ่าน Resource-Allocation Graph (กราฟแสดงการจัดสรรทรัพยากร), และไล่ Safety Algorithm (อัลกอริทึม (ขั้นตอนวิธี)) หา Safe Sequence (ลำดับการรันที่ปลอดภัย) — เป็นแนวคำนวณหลักของ Final
>
> **ใช้จริงในคอมพิวเตอร์:** ฐานข้อมูลล็อกแถวข้อมูล 2 ธุรกรรมรอกัน, เปิดไฟล์ Excel 2 โปรแกรมล็อกกัน, รถติด 4 แยกไม่มีใครยอมถอย — ระบบจริงเลือก "เมิน + ให้คนรีสตาร์ทเอง" เหมือนที่ OS ทำ

F1. Deadlock (การติดตาย (รอกันตลอดกาล)) คืออะไร + ตัวอย่างรากฐาน

Deadlock (การติดตาย (รอกันตลอดกาล)) = process (กระบวนการ) ≥ 2 ตัว**รอกันไม่สิ้นสุด** เพราะเหตุการณ์ที่จะปลุกนั้นอยู่ในมือกลุ่มผู้รอเอง

```
/* ตัวอย่างโค้ดจากสไลด์ p.4 — S, Q เป็น binary semaphore เริ่ม 1 ทั้งคู่ */
/* P0 */                    /* P1 */
wait(S);    ← S=0 ยึด S     wait(Q);    ← Q=0 ยึด Q
wait(Q);    รอ Q (P1 ถือ)    wait(S);    รอ S (P0 ถือ) → ตายล็อกทั้งคู่
...
signal(S); signal(Q);       signal(Q); signal(S);
```

- **Starvation (การอดตาย (ไม่ได้รันเลย))** = indefinite blocking — process (กระบวนการ) อาจไม่เคยถูกหยิบออกจากคิว semaphore (ตัวนับควบคุมสิทธิ์เข้าถึง) เลย (ต่างจาก deadlock (การติดตาย (รอกันตลอดกาล)) ที่รอกันเป็นวง)
- สไลด์ p.3 สะพานแคบ: รถ = process (กระบวนการ), ช่วงถนน = resource (ทรัพยากร), แก้โดยให้คันหนึ่ง**ถอย (preempt + rollback)**
- **OS ส่วนใหญ่ (รวม UNIX (ระบบยูนิกซ์ (บรรพบุรุษของ Linux (OS ลินุกซ์)))) ไม่ป้องกัน/ไม่จัดการ deadlock (การติดตาย (รอกันตลอดกาล)) เลย** — ปล่อยให้โปรแกรมเมอร์รับผิดชอบ

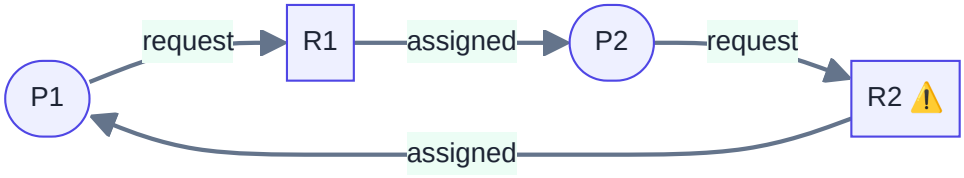
F2. เงื่อนไข 4 ข้อ — ต้องเกิด "พร้อมกัน" จึงเกิด deadlock (การติดตาย (รอกันตลอดกาล)) 🔥

เงื่อนไข	ความหมาย	Prevention ยังไง
1. Mutual Exclusion (เข้าถึงได้คนเดียว)	ทรัพยากร 1 ชิ้นมีได้ 1 เจ้าของ/เวลา	ทำให้แชร์ได้ — ทำไม่ได้กับทุกชนิด
2. Hold and Wait (ถือของเก่า รอของใหม่)	ถืออย่างน้อย 1 ชิ้น + รอชิ้นอื่นที่คนอื่นถือ	ขอให้**ครบทุกชิ้นก่อนเริ่มรัน** หรือขอเมื่อไม่ถืออะไรเลย → utilization ต่ำ + starvation ได้
3. No Preemption (ยึดคืนกลางคันไม่ได้)	ยึดคืนกลางคันไม่ได้ ปล่อยเมื่อเจ้าของใจดีเท่านั้น	ขอเพิ่มไม่ได้ = ถูกยึดของที่ถือทั้งหมด ไปให้คนรอ แล้วค่อยเริ่มใหม่ → ยุ่งยาก
4. Circular Wait (รอกันเป็นวงกลม)	P0 รอ P1, P1 รอ P2, ... วนกลับมา P0	ให้ทุกทรัพยากรมี**เลขประจำตัว** ขอเรียง**จากน้อยไปมากเท่านั้น** เช่น Disk=4, Printer=11 → ต้องได้ 4 ก่อน 11 ห้ามถือ 11 แล้วขอ 4

> 🎯 **จำแน่น**
>
> ต้องครบ **4 ข้อพร้อมกัน** → ตัดข้อเดียวก็ไม่เกิด · Prevention ทั้งหมด "ทฤษฎีดี แต่ปฏิบัติเสียประสิทธิภาพ" อาจารย์สรุปว่าไม่คุ้มใช้

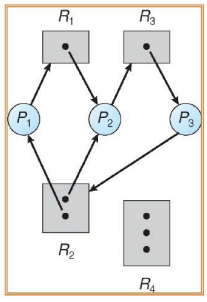
F3. Resource-Allocation Graph (RAG)

- **วงกลม** = Process (กระบวนการ) P_i · **สี่เหลี่ยม** = Resource (ทรัพยากร) type R_j (จุดในสี่เหลี่ยม = จำนวน instance)
- **Request edge** $P_i \rightarrow R_j$: "ขอ" · **Assignment edge** $R_j \rightarrow P_i$: "ได้ถืออยู่"



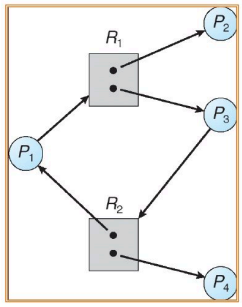
ไม่มี cycle (วงจร (วนกลับมาที่เดิม)) → ไม่เกิด deadlock (การติดตาย (รอกันตลอดกาล)) แน่ (100%)
มี cycle (วงจร (วนกลับมาที่เดิม)) → single instance/type = deadlock (การติดตาย (รอกันตลอดกาล)) แน่ · multiple instances = "อาจเกิด" เท่านั้น
slide p.11: มี cycle (วงจร (วนกลับมาที่เดิม)) แต่ R2 มีหลายชิ้น ชิ้นที่เหลือ (ถือโดย P4 นอกวง) มาช่วยได้ → ไม่ deadlock (การติดตาย (รอกันตลอดกาล))





สไลด์ p.10 RAG มี deadlock (การติดตาย (รอกันตลอดกาล))





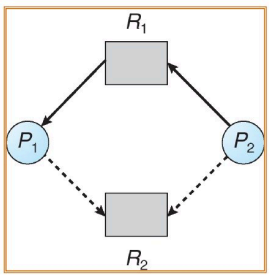
สไลด์ p.11 มี cycle (วงจร (วนกลับมาที่เดิม)) แต่ไม่ deadlock (multiple instance)

F4. กลยุทธ์จัดการ Deadlock (การติดตาย (รอกันตลอดกาล)) 4 แนวทาง

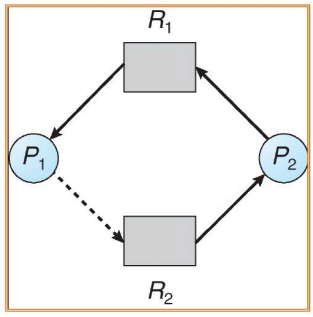
1. **Prevention** — ออกแบบให้เงื่อนไขข้อใดข้อหนึ่งเกิดไม่ได้เลย (ตาราง F2)
2. **Avoidance** — process (กระบวนการ) แจ้ง **Max demand ล่วงหน้า** OS เช็คทุกครั้งก่อนจัดสรรว่ายัง safe → Banker (อัลกอริทึมนายธนาคาร)'s / claim edge
3. **Detection + Recovery** — ปล่อยเกิด สแกนหา แล้วแก้
4. **เมิน (Ostrich)** — ที่ OS จริงใช้กัน

F4.1 Avoidance แบบ single instance — Claim Edge (เส้นประ)

- **Claim edge** $P_i \dashrightarrow R_j$ (เส้นประ) = "อาจขอในอนาคต" · ขอจริง → กลายเป็น request edge · ได้รับ → assignment edge · ปล่อย → กลับเป็น claim edge
- **ให้ทรัพยากรได้เมื่อ:** แปลง request เป็น assignment แล้ว**เกิด cycle (วงจร (วนกลับมาที่เดิม)) ไม่ได้** (มี cycle (วงจร (วนกลับมาที่เดิม)) = unsafe → ไม่ให้ ให้รอ)
- ข้อเสีย: สแกนหา cycle (วงจร (วนกลับมาที่เดิม)) ทุกครั้งที่ขอ = $O(n^2)$ แพง



สไลด์ p.20 Claim edges (เส้นประ)



สไลด์ p.21 Unsafe state: ถ้าให้ P2 ใช้ R2 จะเกิดวง → ไม่ให้

F5. Banker's Algorithm (multiple instances) 🔥🔥

ข้อมูลที่ OS ถือ (n = process (กระบวนการ), m = resource (ทรัพยากร) type): `Available[m]` · `Max[n×m]` · `Allocation[n×m]` · **Need = Max − Allocation**

```
Need[i,j] = Max[i,j] − Allocation[i,j]
```

หัวใจของทุกโจทย์ Banker (อัลกอริทึมนายธนาคาร)'s — คิด Need ให้ครบทุกช่องก่อนเสมอ

F5.1 Safety Algorithm (ตรวจว่า state ปลอดภัยไหม)

1. Work = Available · Finish[i] = false ทุกตัว
2. หา i ที่ **Finish[i] = false และ $Need_i \le Work$** (เทียบทีละช่องทุกชนิด) — ไม่มีใครผ่าน → ไปขั้น 4
3. สมมติ i รันจนจบแล้วคืนของ: **$Work = Work + Allocation_i$**, Finish[i] = true → กลับขั้น 2
4. ถ้า Finish = true **ทุกตัว** → **SAFE** (มี safe sequence) · ติดตรงไหน = UNSAFE

F5.2 โจทย์เต็มจากสไลด์ p.26–27 (5 process (กระบวนการ), A=10 B=5 C=7, Available = 3,3,2)

Process (กระบวนการ)	Allocation			Max			Need = Max − Alloc		
	A	B	C	A	B	C	A	B	C
P0	0	1	0	7	5	3	7	4	3
P1	2	0	0	3	2	2	1	2	2
P2	3	0	2	9	0	2	6	0	0
P3	2	1	1	2	2	2	0	1	1
P4	0	0	2	4	3	3	4	3	1

ไล่ Safety (Work เริ่ม = 3,3,2):

ขั้น	เลือกใคร (เหตุผล)	Work หลังคืนของ
1	P1 — Need (1,2,2) ≤ (3,3,2) ✓	(3,3,2)+(2,0,0) = **(5,3,2)**
2	P3 — Need (0,1,1) ≤ (5,3,2) ✓	(5,3,2)+(2,1,1) = **(7,4,3)**
3	P4 — Need (4,3,1) ≤ (7,4,3) ✓	(7,4,3)+(0,0,2) = **(7,4,5)**
4	P2 — Need (6,0,0) ≤ (7,4,5) ✓	(7,4,5)+(3,0,2) = **(10,4,7)**
5	P0 — Need (7,4,3) ≤ (10,4,7) ✓	(10,4,7)+(0,1,0) = (10,5,7) ครบทุกตัว ✅

`Safe sequence = <P1, P3, P4, P2, P0> → SAFE STATE`

มีได้หลายคำตอบ — ตอบลำดับไหนที่ไล่ได้จริงก็ถูก (สไลด์ให้ <P1,P3,P4,P2,P0>)

F5.3 Resource (ทรัพยากร)-Request Algorithm (อัลกอริทึม (ขั้นตอนวิธี)) — P1 ขอ (1,0,2) ให้ได้ไหม?

1. เช็ค Request ≤ Need: (1,0,2) ≤ (1,2,2) ✓ (ไม่ขอเกินที่ประกาศ)
2. เช็ค Request ≤ Available: (1,0,2) ≤ (3,3,2) ✓ (ของพอ)
3. **สมมติจัดให้ (pretend):** Available = (2,3,0) · P1 Alloc = (3,0,2) · P1 Need = (0,2,0)
4. รัน Safety ใหม่ → ได้ sequence <P1, P3, P4, P0, P2> → ยัง SAFE ✅ → **ให้ไปได้**

คำขออื่นในสไลด์ p.28	ผล	เหตุผล
P4 ขอ (3,3,0)	**ไม่ให้**	Request > Available (3,3,0) > (2,3,0) — ของไม่พอ P4 ต้อง wait (รอ + หยิบสิทธิ์)
P0 ขอ (0,2,0)	**ไม่ให้**	ผ่าน 2 เช็คแรก แต่ไล่ Safety แล้ว **UNSAFE** (มี deadlock (การติดตาย (รอกันตลอดกาล)) เป็นไปได้) → ให้รอ คืนสถานะเดิม

🟢 วิธีทำในห้องสอบ (เร็ว)

1) เติมตาราง Need = Max − Alloc ทุกช่อง 2) เขียน Work = Available กก.ลงข้าง ๆ 3) วนหา "Need ≤ Work" ใครผ่านให้ไปก่อน แล้ว **บวก Allocation แถวนั้นเข้า Work** 4) จนครบทุกตัว = safe · โจทย์ถาม "ขอได้ไหม" → เช็ค 2 ข้อแล้ว pretend แล้วไล่ safety ใหม่เสมอ

📌 ปัญหาของ Banker (อัลกอริทึมนายธนาคาร)'s (เคยออก)

ต้องประกาศ Max ล่วงหน้า / จำนวนทรัพยากรคงที่ / จำนวนงานคงที่ / overhead สูง / ใช้ทรัพยากรไม่คุ้มเพราะคิดแย่สุดเสมอ

F6. Detection (ตรวจจับ) & Recovery (กู้)

- **Single instance/type:** ยุบ RAG เป็น **Wait-For Graph (กราฟแสดงว่าใครรอใคร)** — ตัดโหนดทรัพยากรทิ้ง เหลือ $P_i \rightarrow P_j$ ("P_i รอ P_j") มี cycle (วงจร (วนกลับมาที่เดิม)) = deadlock (สแกนทุกระยะ, แพง $O(n^2)$)
- **Multiple instances:** Detection Algorithm (อัลกอริทึม (ขั้นตอนวิธี)) ทำงานเหมือน Safety แต่เทียบด้วย **Request** (ของที่ร้องขอค้างจริง) แทน Need · Finish เริ่มต้น: คนที่ Allocation ไม่เป็นศูนย์ = false

ตัวอย่าง Detection จากสไลด์ p.36–37 (A=7, B=2, C=6, Available = 0,0,0):

Process (กระบวนการ)	Allocation			Request		
	A	B	C	A	B	C
P0	0	1	0	0	0	0
P1	2	0	0	2	0	2
P2	3	0	3	0	0	0
P3	2	1	1	1	0	0
P4	0	0	2	0	0	2

ไล่: Work=(0,0,0) → P0 (Request 000 ≤ 000) จบ → Work=(0,1,0) → P2 (000 ≤ 010) จบ → Work=(3,1,3) → P3 จบ → (5,2,4) → P1 จบ → (7,2,4) → P4 จบ → **sequence <P0,P2,P3,P1,P4> ครบ → ไม่มี deadlock (การติดตาย (รอกันตลอดกาล))**

แต่ถ้า P2 ขอเพิ่ม C หนึ่งชิ้น (Request P2 = 0,0,1): Work=(0,0,0) → P0 จบ → (0,1,0) → ใครผ่าน? P1(2,0,2)✗ P2(0,0,1)✗ P3(1,0,0)✗ P4(0,0,2)✗ → ตัน → **deadlock (การติดตาย (รอกันตลอดกาล)): P1, P2, P3, P4**

Recovery 2 แนวทาง

- **Process (กระบวนการ) Termination:** จบทุกตัวที่ติด (เสียงานหมด) หรือจบทีละตัวจนวงแตก — เลือกจาก: priority (ลำดับสิทธิ์), รันมานานแค่ไหน, ใช้ทรัพยากรอะไรไปแล้ว, ต้องการอีกเท่าไร, ต้องจบกี่ตัว, interactive/batch?
- **Resource (ทรัพยากร) Preemption (การยึดทรัพยากรกลางคัน):** เลือกเหยื่อ (victim) → **Rollback** ไป safe state → เริ่มใหม่ · ระวัง **Starvation (การอดตาย (ไม่ได้รันเลย))** — ใส่ "จำนวน rollback ที่โดนมาแล้ว" ในตัวแปรต้นทุน

G. Real Memory (หน่วยความจำ) Management (OSNo08 ต้น)

📖 รู้จักส่วนนี้ก่อนอ่าน

ความหมาย: การจัดสรร RAM ให้ process (กระบวนการ) ตอนโหลด (ยุคก่อนมี virtual memory (หน่วยความจำ)) — จัดแบบช่องคงที่, ช่องแปรผัน, หรือแบ่งตามส่วน (Segmentation (การแบ่งหน่วยความจำตามส่วน)) พร้อมกลยุทธ์เลือกหลุม First/Best/Worst-fit (หยิบหลุมใหญ่ที่สุด)

ความสำคัญกับข้อสอบ: โจทย์ first/best/worst-fit คำนวณเร็วได้คะแนนง่าย + Segmentation (เช็ค offset (ตำแหน่งเลื่อนภายในเพจ) < limit ก่อน แล้วค่อย base + offset (ตำแหน่งเลื่อนภายในเพจ)) — และการแยก internal/external (ภายนอก (กระจายเป็นหลุม)) fragmentation (การแตกเป็นเศษพื้นที่) ให้ถูกตัว

ใช้จริงในคอมพิวเตอร์: เป็นต้นแบบ memory (หน่วยความจำ) allocator (malloc/free) ในทุกภาษา · ปัญหา fragmentation (การแตกเป็นเศษพื้นที่) ยังพบจริงในระบบฝังตัว (embedded) และการจัดการหน่วยความจำ GPU

G1. Logical (เชิงตรรกะ (มองจากโปรแกรม)) vs Physical Address (ที่อยู่จริงใน RAM) + MMU

	Logical (Virtual) Address (ที่อยู่หน่วยความจำ)	Physical Address (ที่อยู่จริงใน RAM)
สร้างโดย	CPU ปล่อยขณะรัน	ตำแหน่งจริงที่ memory (หน่วยความจำ) unit มองเห็น
จุดอ้างอิง	โปรแกรมเริ่มที่ byte 0 เสมอ	เทียบจุดเริ่ม RAM จริง

MMU = ฮาร์ดแวร์แปลง logical (เชิงตรรกะ (มองจากโปรแกรม)) → physical (เชิงกายภาพ (ตำแหน่งจริง)) แบบง่ายสุดคือ**บวก Relocation (Base) Register**

`Physical Address (ที่อยู่จริงใน RAM) = Relocation Register (รีจิสเตอร์เก็บตำแหน่งฐาน) + Logical Address (ที่อยู่เชิงตรรกะ (มองจากโปรแกรม))`

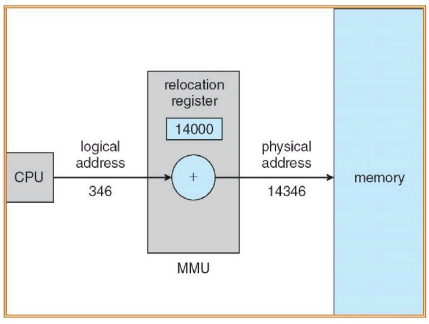
ตัวอย่างวิดีโอ: โปรแกรมโหลดที่ 14,000 · CPU ขอ byte 346 → PA = 14,000 + 346 = 14,346

Logical vs. Physical Address Space

- **Logical address** – generated by the CPU; also referred to as **virtual address**
- **Physical address** – address seen by the memory unit

สไลด์ p.3 Logical (เชิงตรรกะ (มองจากโปรแกรม)) vs Physical (เชิงกายภาพ (ตำแหน่งจริง))

Dynamic relocation using a relocation register



สไลด์ p.5 Dynamic relocation ด้วย relocation register

G2. Static vs Dynamic Linking (มีตัวเลขให้คำนวณ)

- **Static linking**: ก๊อปโค้ด library รวมใน executable ตั้งแต่ compile — lib ซ้ำทุกโปรแกรม
- **Dynamic linking**: เลื่อนไป link (ลิงก์ (ทางเชื่อม)) ตอน run — ใส่แค่ **stub** (โค้ดจิ๋วหา library / แทนตัวเองด้วย address (ที่อยู่หน่วยความจำ) routine) → library โหลดครั้งเดียวแชร์กันทั้งระบบ (shared libraries)

`ตัวอย่างวิดีโอ: A(10MB) + B(20MB) + Lib(20MB)`
`Static: A=30MB, B=40MB → RAM 30+40 = 70MB (lib ซ้ำ 2 ชุด)`
`Dynamic: A=10+2, B=20+2, lib 1 ชุด → 12+20+22 = 54MB ประหยัด 16MB`

G3. Fixed Partition — Internal Fragmentation (เศษภายในช่องที่จองแล้ว)

- แบ่ง RAM ส่วน user เป็นช่องคงที่ล่วงหน้า → **มีกี่ช่องรับ process (กระบวนการ) พร้อมกันได้เท่านั้น** · ป้องกันด้วย limit register (ขนาด) + relocation register (ตำแหน่ง): เช็ค LA < Limit ผ่านแล้วค่อย + Base
- **ตัวอย่างวิดีโอ:** Partitions P1=100K, P2=25K, P3=25K, P4=50K · Job1(30K)→P1 (เหลือ 70K ใช้ไม่ได้ = **internal frag**) · Job2(50K)→P4 พอดี · **Job3(30K) ต้องรอ!** เพราะ P2,P3 มีแค่ 25K แม้ว่างรวม 120K · Job4(25K)→P2 พอดี

🎯 คู่แฝด fragmentation (การแตกเป็นเศษพื้นที่)

Fixed = Internal (เศษในช่องจองแล้ว ใครใช้ต่อไม่ได้) · **Variable + Segmentation (การแบ่งหน่วยความจำตามส่วน) = External** (เศษกระจายนอก รวมว่างเยอะแต่ช่องเดียวไม่พอ) · **Paging (การแบ่งหน่วยความจำเป็นหน้า) = ไม่มี External แต่มี Internal** ที่เพจสุดท้าย — ออกถูก/ผิดวนไป

G4. Variable Partition — External Fragmentation (เศษกระจายนอก (หลุมเล็ก ๆ)) + วิธีรักษา

จัดสรรตามขนาดจริง ตอนแรกไม่มีของเสีย แต่งานจบทิ้งช่องว่างกระจาย · **ตัวอย่าง:** หลุมว่าง 5+2+1 = 8MB แต่ขอ 7MB โหลดไม่ได้ (ช่องเดียวใหญ่สุด 5MB)

วิธีรักษา	ทำอะไร	ข้อจำกัด
Coalescing (การรวมหลุมว่างที่ติดกัน)	รวมหลุมที่**ติดกัน**เป็นหลุมใหญ่ 1 หลุม	หลุมไม่ติดกันก็ช่วยไม่ได้ — มักไม่พอ
Compaction (การอัดข้อมูลให้ติดกัน)	ลาก process (กระบวนการ) มา**อัดติดกัน** รวมว่างปลายเดียว	overhead แพง และทำได้**เฉพาะ relocation แบบ dynamic (ตอน execution time)**

G5. First-fit (หยิบหลุมแรกที่พอ) · Best-fit (หยิบหลุมที่แน่นที่สุด) · Worst-fit (หยิบหลุมใหญ่ที่สุด) 🔥 (ตัวอย่างจากวิดีโอ)

โจทย์ (สไลด์ p.21–24): หลุมเรียงตามลำดับใน RAM: **16, 14, 5, 30 MB** · process (กระบวนการ) ใหม่ต้องการ **13 MB**

กลยุทธ์	หลักคิด	ลงหลุม	เศษเหลือ	จุดเด่น/ด้อย
First-fit (หยิบหลุมแรกที่พอ)	ไล่จากซ้าย เจอหลุมแรกที่ ≥ ขนาด เอาทันที	**16**	3	เร็วสุด (ไม่สแกนครบ) แต่เปลืองพื้นที่
Best-fit (หยิบหลุมที่แน่นที่สุด)	สแกนครบ เลือกหลุมที่ "ใหญ่พอ + เหลือน้อยสุด"	**14**	1	คุ้มสุด แต่ช้า + ทิ้งเศษจิ๋วใช้ต่อไม่ได้
Worst-fit (หยิบหลุมใหญ่ที่สุด)	สแกนครบ เลือกหลุมใหญ่สุด	**30**	17	ทิ้งชิ้นใหญ่ให้งานต่อไป (โดยรวมมักแพ้ best-fit)

🟢 เทคนิค 30 วินาที

เขียนเลขหลุมตามลำดับที่โจทย์ให้ (ห้ามจัดใหม่!) · First = หยุดหลุมแรกที่พอ · Best = เทียบ (หลุม−ขนาด) เลือกค่าน้อยสุด · Worst = หลุมใหญ่สุด · ถ้าโจทย์ให้จัดหลายงานต่อกัน อย่าลืมหักขนาดหลุมที่ใช้ไปแล้วด้วย

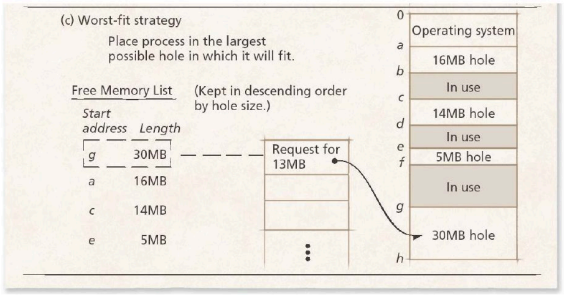
Memory Placement Strategies

- Where to put incoming processes
  - First-fit strategy
    - Process placed in first hole of sufficient size found
    - Simple, low execution-time overhead
  - Best-fit strategy
    - Process placed in hole that leaves least unused space around it
    - More execution-time overhead
  - Worst-fit strategy
    - Process placed in hole that leaves most unused space around it
    - Leaves another large hole, making it more likely that another process can fit in the hole

สไลด์ p.21 Memory (หน่วยความจำ) Placement Strategies

Memory Placement Strategies

First-fit, best-fit and worst-fit memory placement strategies (Part 3 of 3).



สไลด์ p.24 ผลการจัดทั้ง 3 กลยุทธ์ (ตรงกับตารางข้างบน)

G6. Segmentation (การแบ่งหน่วยความจำตามส่วน) 🔥 — แบ่งตามสายตาโปรแกรมเมอร์

- แบ่งเป็น logical (เชิงตรรกะ (มองจากโปรแกรม)) unit ขนาดไม่เท่ากัน: main, function, object, stack, global variables · Logical (เชิงตรรกะ (มองจากโปรแกรม)) address (ที่อยู่หน่วยความจำ) = คู่ `<segment s, offset d>`
- Segment table (ตาราง) แต่ละแถวมี **Base** (จุดเริ่มใน RAM) + **Limit** (ความยาว) · STBR ชี้ตาราง, STLR = จำนวน segment (s ใช้ได้ถ้า s < STLR)

```
ขั้นแรก: เช็ค d < Limit — ถ้า d ≥ Limit → TRAP (error!) ทันที
ขั้นต่อมา: Physical Address (ที่อยู่จริงใน RAM) = Base + Offset (ตำแหน่งเลื่อนภายในเพจ)
```

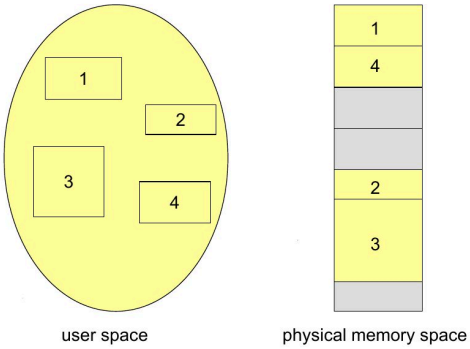
ตัวอย่างจากสไลด์ p.50: S1 (base 6300, limit 500) · S2 (base 4300, limit 400)

Logical (เชิงตรรกะ (มองจากโปรแกรม))	เช็ค	ผล
<1, 300>	300 < 500 ✓ → PA = 6300 + 300	**6600** ✓
<2, 400>	400 < 400 ✗ (400 ไม่ได้ "น้อยกว่า" 400)	**Trap / Error** ❌

🎯 จุดหลอก

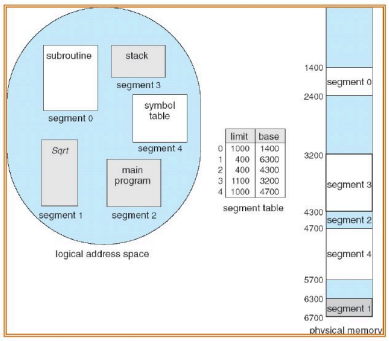
เช็ค **ก่อนเสมอ แล้วค่อยบวก** — เคร่งครัดเรื่อง < ไม่ใช่ ≤ · Segmentation (การแบ่งหน่วยความจำตามส่วน) ยังเจ็บกับ **external (ภายนอก (กระจายเป็นหลุม)) fragmentation (การแตกเป็นเศษพื้นที่)** (แต่ละ segment ต้องวางติดกันเป็นผืน) · สูตรอย่างสลับกับ paging (การแบ่งหน่วยความจำเป็นหน้า): Segment/Base = **Base + offset (ตำแหน่งเลื่อนภายในเพจ)** / Paging (การแบ่งหน่วยความจำเป็นหน้า) = **frame (เฟรม (ก้อน RAM)) × size (ขนาด) + offset (ตำแหน่งเลื่อนภายในเพจ)**

Logical View of Segmentation



สไลด์ p.47 Logical (เชิงตรรกะ (มองจากโปรแกรม)) view (user space) vs physical (เชิงกายภาพ (ตำแหน่งจริง)) memory (หน่วยความจำ)

Example of Segmentation



สไลด์ p.50 Example of Segmentation (ตัวเลขตรงกับตาราง)

H. Paging (การแบ่งหน่วยความจำเป็นหน้า) & Address (ที่อยู่หน่วยความจำ) Translation (OSNo08 ท้าย) 🔥🔥

> **📖 รู้จักส่วนนี้ก่อนอ่าน — รีวิวรุ่นพี่: "คนส่วนมากคำนวณ Paging (การแบ่งหน่วยความจำเป็นหน้า) กันไม่ทัน"**
>
> **ความหมาย:** Paging (การแบ่งหน่วยความจำเป็นหน้าเท่า ๆ กัน) = หั่น RAM เป็น frame (เฟรม (ก้อน RAM)) หั่นโปรแกรมเป็น page (ขนาดเท่ากัน) แล้วใช้ Page Table (ตารางจับคู่เพจ ↔ เฟรม) จับคู่ — "คำนวณ Paging (การแบ่งหน่วยความจำเป็นหน้า)" คือการแปลงที่อยู่ที่โปรแกรมเห็น (Logical (เชิงตรรกะ (มองจากโปรแกรม))) ให้เป็นที่อยู่จริงใน RAM (Physical (เชิงกายภาพ (ตำแหน่งจริง)))
>
> **ความสำคัญกับข้อสอบ:** แปลง address (ที่อยู่หน่วยความจำ) ทีละบิต, TLB (แคชตารางเพจความเร็วสูง) + สูตร EAT (เวลาเข้าถึงเฉลี่ย), Two-level paging (การแบ่งหน่วยความจำเป็นหน้า) — เป็นแนวคำนวณที่ตัดสินว่า "ทันสอบหรือไม่ทัน" ต้องฝึกจนข้อละไม่เกิน 1 นาที
>
> **ใช้จริงในคอมพิวเตอร์:** CPU ทุกตัวสมัยใหม่ (x86, ARM ในมือถือ) + OS ทุกตัว (Windows (OS วินโดวส์)/Linux (OS ลินุกซ์)/macOS/Android) ใช้ paging (การแบ่งหน่วยความจำเป็นหน้า) จริง 100% — แอปแต่ละตัวเห็น "RAM ของตัวเอง" ก็เพราะกลไกนี้

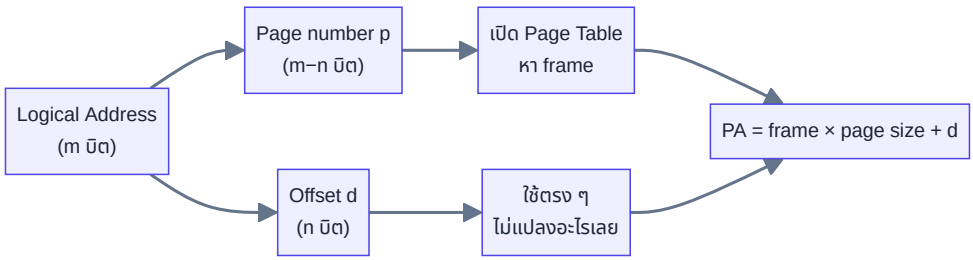
H1. แนวคิด Paging (การแบ่งหน่วยความจำเป็นหน้า)

- **Paging (การแบ่งหน่วยความจำเป็นหน้า) = จัดสรรแบบ non-contiguous** (ไม่ต้องติดกัน) — ตัด external (ภายนอก (กระจายเป็นหลุม)) fragmentation (การแตกเป็นเศษพื้นที่) ขาด
- **Frame (เฟรม (ก้อน RAM))**: แบ่ง RAM เป็นบล็อกขนาดคงที่ ขนาดเป็นกำลังสอง 2^n bytes (512–8,192)
- **Page (หน้า)**: แบ่งโปรแกรมเป็นบล็อก**ขนาดเท่า frame (เฟรม (ก้อน RAM)) เป๊ะ** · รันโปรแกรม n เพจ = หา frame (เฟรม (ก้อน RAM)) ว่าง n อันกระจายวาง
- **Page (หน้า) table (ตาราง)**: จับคู่ "เพจ i → frame (เฟรม (ก้อน RAM)) ไหน" ของแต่ละ process (กระบวนการ)

> **🎯 เช็คความเข้าใจ**
>
> Paging (การแบ่งหน่วยความจำเป็นหน้า) **ไม่มี External frag** แต่มี **Internal frag** — เกิดเฉพาะ**เพจสุดท้าย**ถ้าขนาดโปรแกรมไม่ลงตัว

H2. โครงสร้าง Logical Address (ที่อยู่เชิงตรรกะ (มองจากโปรแกรม)) — บิต p กับ d 🔥

Address (ที่อยู่หน่วยความจำ) ที่ CPU ปล่อย (ยาว **m บิต**) ซอยเป็น 2 ส่วนโดยฮาร์ดแวร์:



`Page (หน้า) size (ขนาด) = 2ⁿ bytes → n บิต offset (ตำแหน่งเลื่อนภายในเพจ) · m − n บิต page number (หมายเลขเพจ) → เพจได้สูงสุด 2^(m−n) เพจ`

ตัวอย่างวิดีโอ: m = 5 บิต (2^5 = 32 bytes), page (หน้า) = 4 bytes (2^2) → offset (ตำแหน่งเลื่อนภายในเพจ) 2 บิต, page (หน้า)# 3 บิต → 8 เพจ (0–7 นับจากศูนย์!)

> **🪑 อุปมาอาจารย์**
>
> เก้าอี้ 32 ตัว จัด 8 ห้อง ห้องละ 4 — อ้างอิงว่า "เก้าอี้ตัวที่ 3 ในห้องเบอร์ 1" = (Page (หน้า) 1, Offset (ตำแหน่งเลื่อนภายในเพจ) 3) สองเลขนี้คือทั้งชีวิตของ paging (การแบ่งหน่วยความจำเป็นหน้า)

H3. แปลง Logical (เชิงตรรกะ (มองจากโปรแกรม)) → Physical (เชิงกายภาพ (ตำแหน่งจริง)) ทีละขั้น 🔥

Physical Address (ที่อยู่จริงใน RAM) = (Frame (เฟรม (ก้อน RAM)) Number × Page Size (ขนาดของเพจ)) + Offset (ตำแหน่งเลื่อนภายในเพจ)

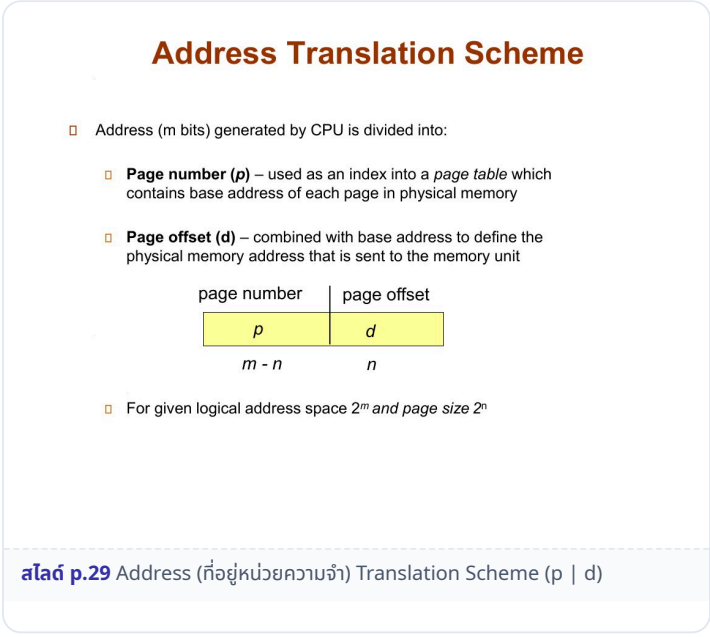
เปิดตารางด้วย p → ได้ frame (เฟรม (ก้อน RAM)) → คูณขนาด frame (เฟรม (ก้อน RAM)) = จุดเริ่ม → บวก offset (ตำแหน่งเลื่อนภายในเพจ)

โจทย์จากสไลด์ p.32 (Paging (การแบ่งหน่วยความจำเป็นหน้า) Example): page size (ขนาดของเพจ) = 4 bytes · Page (หน้า) table (ตาราง): 0→5, 1→4, 2→1, 3→7

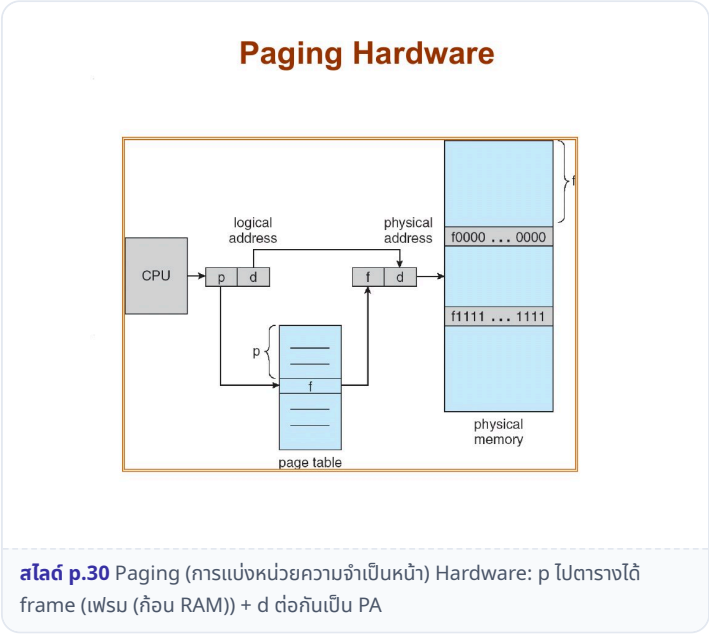
หาอะไร	Logical (เชิงตรรกะ (มองจากโปรแกรม))	ไล่ทีละขั้น	PA
ตัวอักษร 'C'	(p=0, d=2)	เพจ 0 → frame (เฟรม (ก้อน RAM)) 5 → (5 × 4) + 2 = 20 + 2	**22**
ตัวอักษร 'K'	(p=2, d=2)	เพจ 2 → frame (เฟรม (ก้อน RAM)) 1 → (1 × 4) + 2 = 4 + 2	**6**

🎯 3 จุดพลาดคลาสสิก

1) **บวก frame (เฟรม (ก้อน RAM))+offset (ตำแหน่งเลื่อนภายในเพจ) ตรง ๆ** (5+2=7 ❌) ต้องคูณขนาดก่อน 2) นับเพจเริ่ม 1 → ต้องเริ่ม **Page (หน้า) 0** 3) ปัดจำนวนเพจลง — 42 bytes / เพจ 4 = 10.5 → **ปัดขึ้น 11 เพจ** (10 เพจเก็บได้ 40 bytes ข้อมูลขาด!) เพจสุดท้ายใช้ 2 bytes → internal frag = 4−2 = **2 bytes**

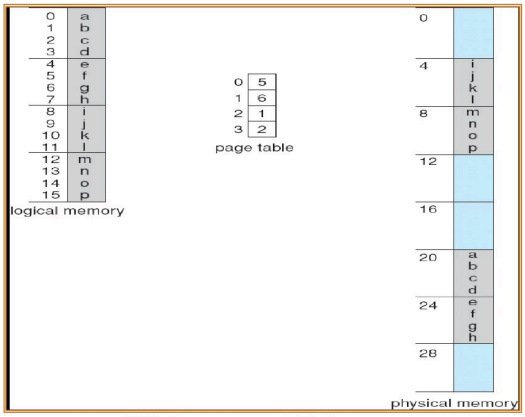


สไลด์ p.29 Address (ที่อยู่หน่วยความจำ) Translation Scheme (p | d)



สไลด์ p.30 Paging (การแบ่งหน่วยความจำเป็นหน้า) Hardware: p ไปตารางได้ frame (เฟรม (ก้อน RAM)) + d ต่อกันเป็น PA

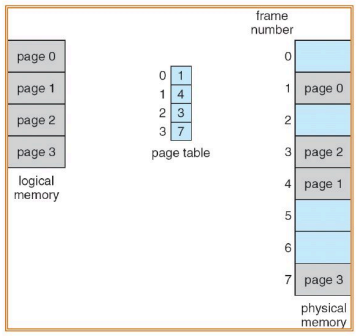
Paging Example



32-byte memory and 4-byte pages

สไลด์ p.32 Paging (การแบ่งหน่วยความจำเป็นหน้า) Example — หน่วยความจำ 32 bytes, เพจ 4 bytes (โจทย์ 'C' และ 'K' อยู่นี่)

Paging Model of Logical and Physical Memory



สไลด์ p.31 Paging (การแบ่งหน่วยความจำเป็นหน้า) Model: เพจกระจาย frame (เฟรม (ก้อน RAM)) ไม่ติดกันก็ได้

H4. TLB (แคชตารางเพจความเร็วสูง) + Effective Access Time 🔥

- Page (หน้า) table (ตาราง) อยู่ใน RAM → อ่านข้อมูล 1 ครั้ง = ไป RAM **2 ครั้ง** (1: อ่านตารางหา frame (เฟรม (ก้อน RAM)), 2: อ่านข้อมูลจริง)
- **TLB (แคชตารางเพจความเร็วสูง)** = แคชฮาร์ดแวร์ความเร็วสูง เก็บคู่ (page (หน้า) # → frame (เฟรม (ก้อน RAM)) #) ล่าสุด ค้นแบบขนาน (associative memory (หน่วยความจำ))

กรณี	ไป RAM	เวลา (memory (หน่วยความจำ) = 1 µs, TLB (แคชตารางเพจความเร็วสูง) search = ε)
TLB (แคชตารางเพจความเร็วสูง) Hit (โอกาส α)	1 ครั้ง	ε + 1 = 1 + ε
TLB (แคชตารางเพจความเร็วสูง) Miss (โอกาส 1−α)	2 ครั้ง	ε + 1 + 1 = 2 + ε

EAT (เวลาเข้าถึงเฉลี่ย) = α(1 + ε) + (1 − α)(2 + ε) = 2 + ε − α

โจทย์วิดีโอ: ε = 0.1 µs, α = 80% → (0.8×1.1) + (0.2×2.1) = 0.88 + 0.42 = 1.3 µs (สูตรลัด: 2 + 0.1 − 0.8 = 1.3 ✓)

Associative Memory

- Associative memory – parallel search

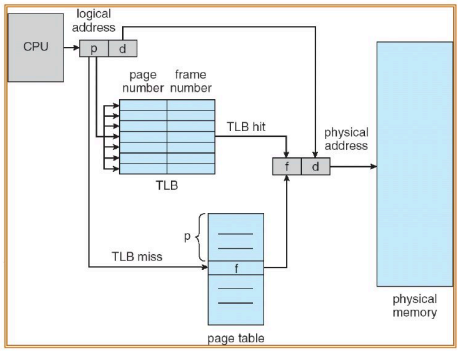
Page #	Frame #

Address translation (p, d)

- If p is in associative register, get frame # out
- Otherwise get frame # from page table in memory

สไลด์ p.35 Associative Memory (TLB (แคชตารางเพจความเร็วสูง)): ค้นขนาน

Paging Hardware With TLB



สไลด์ p.36 Paging (การแบ่งหน่วยความจำเป็นหน้า) Hardware With TLB (แคชตารางเพจความเร็วสูง)

Effective Access Time

- Associative Lookup = ε time unit
- Assume memory cycle time is 1 microsecond
- Hit ratio – percentage of times that a page number is found in the associative registers; ratio related to number of associative registers
- Hit ratio = α
- **Effective Access Time** (EAT)

$$EAT = (1 + \varepsilon)\,\alpha + (2 + \varepsilon)(1 - \alpha)$$
$$= 2 + \varepsilon - \alpha$$

สไลด์ p.37 Effective Access Time สูตร (ตรงกับข้างบน)

H5. Protection — Valid / Invalid bit

- แต่ละแถว page table (ตารางจับคู่เพจ ↔ เฟรม) มี **valid–invalid bit**: **v** = เพจนี้อยู่ใน address (ที่อยู่หน่วยความจำ) space ของ process (ใช้ได้) · **i** = ห้ามอ้างอิง
- **PTBR** ชี้ตำแหน่ง page table (ตารางจับคู่เพจ ↔ เฟรม) ใน RAM · **PTLR** = ความยาวตาราง (เช็คเบื้องต้นว่าไม่อ้างเพจเกินขอบเขต)

🎯 จุดหลอก: valid ดู "ทั้งเพจ" ไม่ได้ดู "ทีละไบต์"

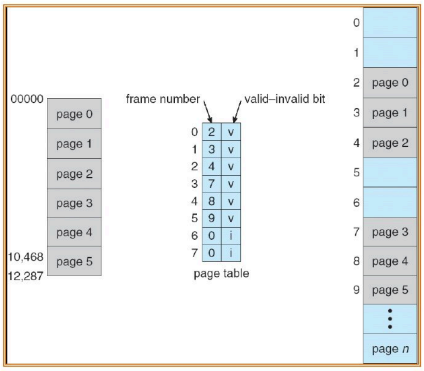
เพจสุดท้าย valid = v → ระบบยอมให้อ้าง offset (ตำแหน่งเลื่อนภายในเพจ) *ใด ๆ* ในเพจนั้น (แม้เกินขนาดโปรแกรมจริง) — นี่คือเหตุผลที่ internal fragmentation (การแตกเป็นเศษพื้นที่) ยังอยู่รอดใน paging (การแบ่งหน่วยความจำเป็นหน้า)

Memory Protection

- Memory protection implemented by associating protection bit with each frame
- **Valid-invalid** bit attached to each entry in the page table:
  - "valid" indicates that the associated page is in the process' logical address space, and is thus a legal page
  - "invalid" indicates that the page is not in the process' logical address space

สไลด์ p.38 Memory (หน่วยความจำ) Protection

Valid (v) or Invalid (i) Bit In A Page Table

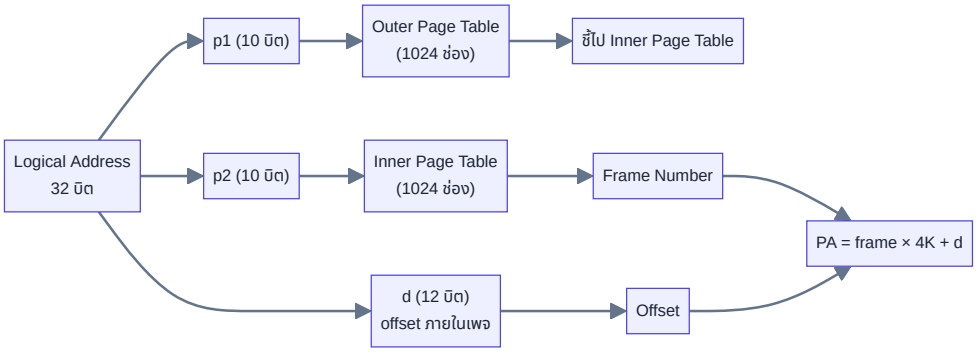


สไลด์ p.39 Valid (v) / Invalid (i) Bit

H6. Two-Level Paging (การแบ่งหน่วยความจำเป็นหน้า) — ทำไม + คำนวณ 🔥

ปัญหา single-level ในระบบ 32-bit: page (หน้า) 4KB (offset (ตำแหน่งเลื่อนภายในเพจ) 12 บิต) → page (หน้า)# 20 บิต → 2^{20} = 1,048,576 ช่อง × 4B = **4MB ต่อ process (กระบวนการ) ทุกตัว!** แม้โปรแกรมเล็กแค่ 1MB (ตารางต้องอยู่ต่อเนื่องด้วย)

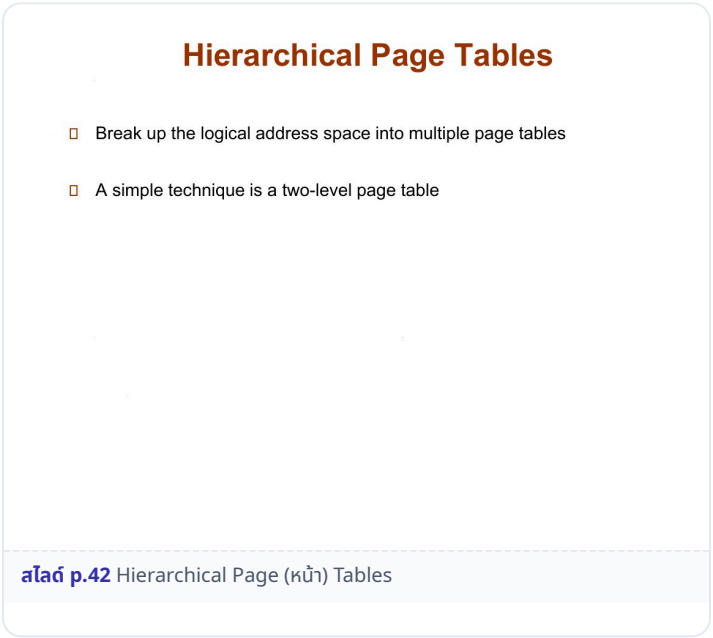
ทางแก้: ซอยบิต 32 ตัวเป็น **p_1 = 10 | p_2 = 10 | d = 12** — p_1 เปิด outer table (ตาราง) หา inner table (ตาราง), p_2 เปิด inner table (ตาราง) หา frame (เฟรม (ก้อน RAM))



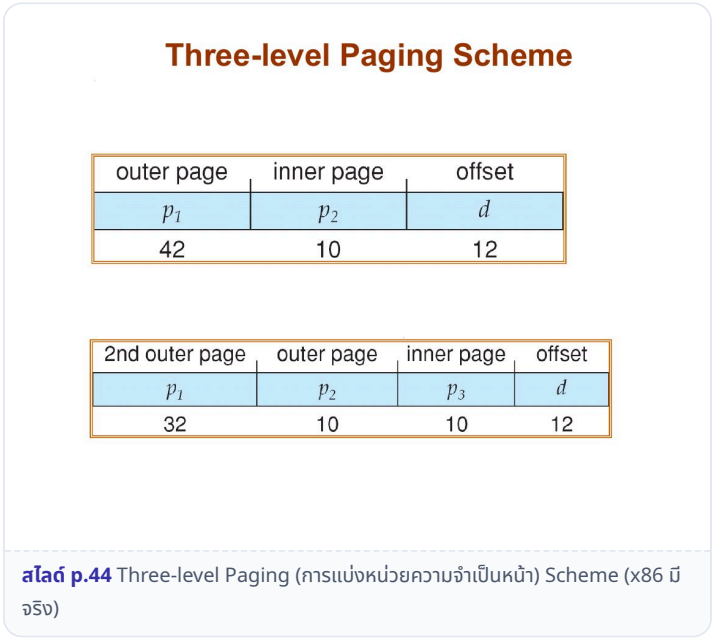
โจทย์ (จากวิดีโอ)	จำนวนเพจ	Inner tables ที่ต้องสร้าง	พื้นที่ page table (ตารางจับคู่เพจ ↔ เฟรม) รวม
โปรแกรม 1 MB	1024KB ÷ 4KB = **256 เพจ**	256 ≤ 1024 → **1 ตาราง**	outer 4KB + inner 4KB = **8 KB** (แทน 4MB!)
โปรแกรม 8 MB	8192KB ÷ 4KB = **2048 เพจ**	2048 ÷ 1024 = **2 ตาราง**	4KB + 2×4KB = **12 KB**

🟢 สูตรรวมทั้งบท

บิต offset (ตำแหน่งเลื่อนภายในเพจ) = $\log_2$(page size (ขนาดของเพจ)) · บิต page (หน้า)# = บิต address (ที่อยู่หน่วยความจำ) − บิต offset (ตำแหน่งเลื่อนภายในเพจ) · เพจ = ceil(ขนาดโปรแกรม ÷ page size (ขนาดของเพจ)) · PA = frame (เฟรม (ก้อน RAM)) × pageSize + offset (ตำแหน่งเลื่อนภายในเพจ) · EAT (เวลาเข้าถึงเฉลี่ย) = 2 + ε − α · inner tables = ceil(จำนวนเพจ ÷ 1024)



สไลด์ p.42 Hierarchical Page (หน้า) Tables



สไลด์ p.44 Three-level Paging (การแบ่งหน่วยความจำเป็นหน้า) Scheme (x86 มีจริง)

หมายเหตุ: ยังมี Hashed Page Table (ตารางจับคู่เพจ ↔ เฟรม) และ Inverted Page Table (ตารางจับคู่เพจ ↔ เฟรม) — รู้จักชื่อพอ (inverted: 1 แถวต่อ 1 frame (เฟรม (ก้อน RAM)) จริง ค้นด้วย (pid, page (หน้า)))

I. Virtual Memory (หน่วยความจำ) & Page Replacement (การเลือกเพจออกเมื่อ RAM เต็ม) (OSNo09) 🔥🔥

📖 รู้จักส่วนนี้ก่อนอ่าน

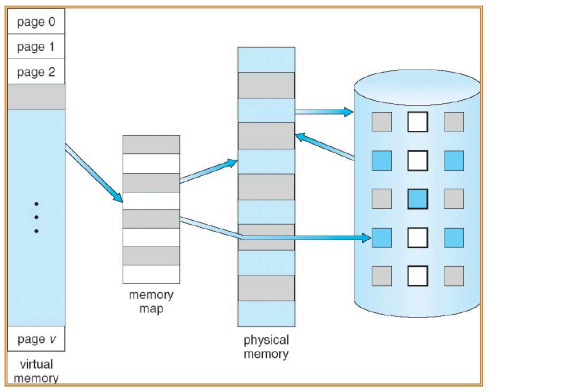
ความหมาย: Virtual Memory (หน่วยความจำเสมือน) = โปรแกรมไม่ต้องอยู่ใน RAM ครบทั้งตัว โหลดเฉพาะเพจที่กำลังใช้ (demand paging (การแบ่งหน่วยความจำเป็นหน้า)) ทำให้ RAM จริงเหมือนใหญ่กว่าจริง · Page Replacement (การเลือกเพจออกเมื่อ RAM เต็ม) = เมื่อ RAM เต็ม ต้องเลือกเพจ "ตัวปลอด" ออก

ความสำคัญกับข้อสอบ: นับ page fault (เพจยังไม่อยู่ใน RAM) ทั้ง FIFO (เข้าก่อน-ออกก่อน) / Optimal (ดีที่สุดเชิงทฤษฎี) / LRU (รู้ผล 15/9/12 ของ string หลัก), Belady's Anomaly (ปรากฏการณ์เพิ่ม RAM แล้ว fault เพิ่ม), Thrashing (สลับเพจหนักจนระบบแย่) — เป็นแนวคำนวณหลักอีกแนวของ Final

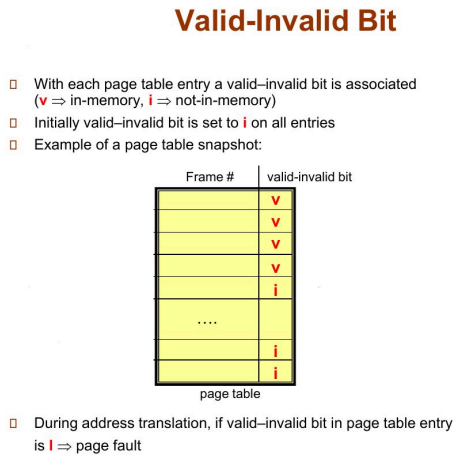
ใช้จริงในคอมพิวเตอร์: เปิดเกมใหญ่กว่า RAM เครื่องได้, สลับแอปเร็วโดยไม่โหลดใหม่, ขนาด "หน่วยความจำเสมือน" ที่เห็นใน Task Manager คือกลไกนี้ — ข้อมูลที่ไม่ได้ใช้ถูกส่งไปพักบน disk

I1. Demand Paging (โหลดเพจเฉพาะเมื่อถูกใช้) — ขอเมื่อไรค่อยเอามา

- **Virtual memory (หน่วยความจำ)** = แยก logical (เชิงตรรกะ (มองจากโปรแกรม)) memory (หน่วยความจำ) ออกจาก physical (เชิงกายภาพ (ตำแหน่งจริง)) → address (ที่อยู่หน่วยความจำ) space ใหญ่กว่า RAM จริงก็ได้ · implement ผ่าน **demand paging (การแบ่งหน่วยความจำเป็นหน้า)** หรือ demand segmentation
- **Demand paging (การแบ่งหน่วยความจำเป็นหน้า)**: โหลดเพจเฉพาะเมื่อถูกเรียกใช้ — หลักขี้เกียจของ **lazy swapper** (ตัวทำงานกับเพจเรียก **pager**) · ผลดี: I/O น้อย, RAM น้อย, ตอบเร็ว, รองรับ user มาก
- เพจที่ยังไม่โหลด → bit = **i (invalid)** → แปลง address (ที่อยู่หน่วยความจำ) เจอ i → **Page Fault (เพจยังไม่อยู่ใน RAM)** — **invalid ที่นี่ส่วนใหญ่ไม่ใช่ error** แค่ "ยังไม่มา"

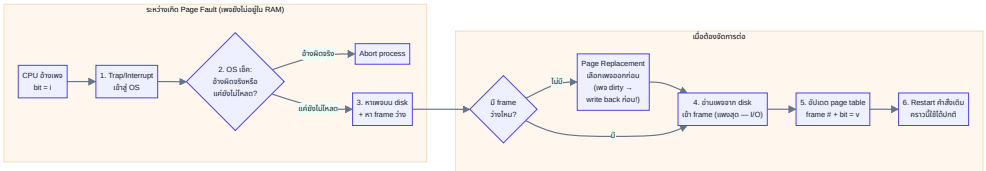


สไลด์ p.4 Virtual Memory (หน่วยความจำ) ใหญ่กว่า Physical (เชิงกายภาพ (ตำแหน่งจริง))



สไลด์ p.6 Valid–Invalid Bit ใน page table (ตารางจับคู่เพจ ↔ เฟรม)

I2. Page Fault (เพจยังไม่อยู่ใน RAM) — ขั้นตอนจัดการ 6 ขั้น (เรียงได้ = ได้คะแนน)



⚠️ Dirty page (หน้า)

เพจที่ถูกแก้ไข (dirty (เพจที่ถูกแก้ไขข้อมูลแล้ว)/modified = 1) ต้อง**เขียนกลับ disk ก่อน** (write back (เขียนกลับลงดิสก์ก่อน)) ค่อยแทนที่ — เพจ clean ทับได้เลยเพราะ disk มีสำเนา

I3. Page Replacement (การเลือกเพจออกเมื่อ RAM เต็ม) — นับ fault 3 อัลกอริทึม 🔥🔥

โจทย์หลัก (สไลด์ใช้ string เดียวกัน, 3 frames (เฟรมหลายก้อน) เริ่มว่าง):

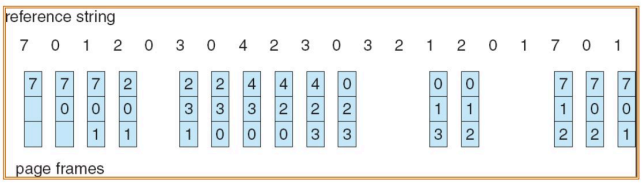
```
Reference String (ลำดับการอ้างเพจ): 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1
```

อัลกอริทึม	มองไหน	หลักเลือก victim	Faults (3 frames (เฟรมหลายก้อน))	ใช้จริง?
FIFO (เข้าก่อน-ออกก่อน)	คิวเข้า	เข้ามาก่อน ออกก่อน (ไม่สนว่าใครฮิต)	**15**	ง่ายแต่โง่ · เจอ Belady
Optimal (OPT)	อนาคต 👁️	ตัวที่**จะไม่ถูกใช้อีกนานสุด** ในอนาคต	**9**	**ใช้จริงไม่ได้** (รู้อนาคตไม่ได้) — ไว้เทียบ benchmark
LRU (แทนตัวที่ไม่ได้ใช้นานสุด)	อดีต 🔙	ตัวที่**ไม่ได้ใช้มานานสุด**	**12**	ใช้จริงได้ (เก็บประวัติย้อนหลัง) ดีกว่า FIFO (เข้าก่อน-ออกก่อน)

👕 ตู้เสื้อผ้า 3 ชั้น

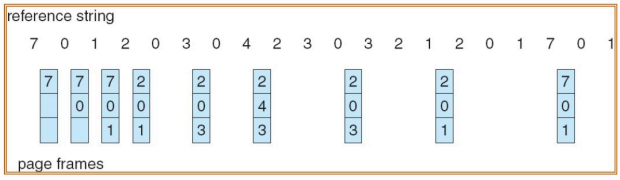
FIFO (เข้าก่อน-ออกก่อน) = รีดตัวใส่นานสุดก่อน · Optimal (ดีที่สุดเชิงทฤษฎี) = หมอดู รีดตัวที่จะไม่ใส่อีกนานสุด · LRU (แทนตัวที่ไม่ได้ใช้นานสุด) = จำได้ว่าใส่ตัวไหนล่าสุดเมื่อไร รีดตัวที่ทิ้งไว้นานสุด

FIFO Page Replacement

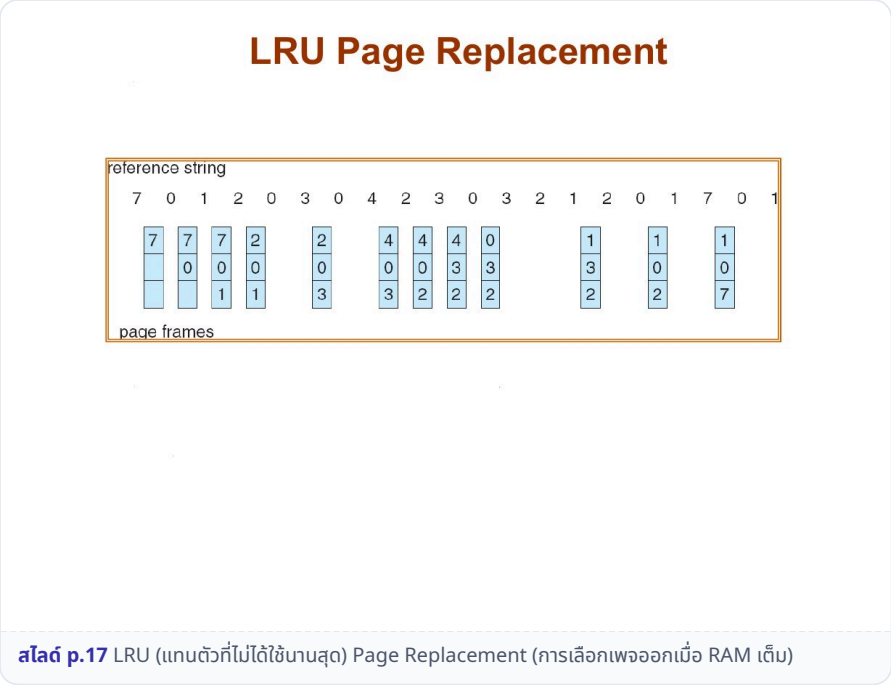


สไลด์ p.13 FIFO (เข้าก่อน-ออกก่อน) Page Replacement (การเลือกเพจออกเมื่อ RAM เต็ม)

Optimal Page Replacement



สไลด์ p.16 Optimal (ดีที่สุดเชิงทฤษฎี) Page Replacement (การเลือกเพจออกเมื่อ RAM เต็ม)



สไลด์ p.17 LRU (แทนตัวที่ไม่ได้ใช้นานสุด) Page Replacement (การเลือกเพจออกเมื่อ RAM เต็ม)

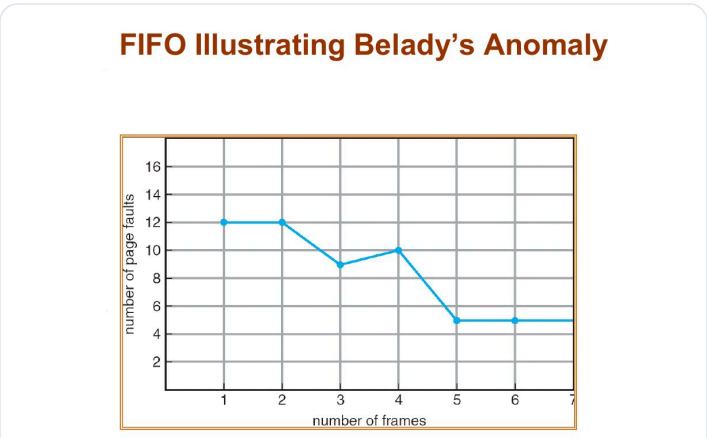
💡 เทคนิคไล่มือ: เขียน 3 ช่อง frame (เฟรม (ก้อน RAM)) ใต้ string ทีละตัวอักษร · FIFO (เข้าก่อน-ออกก่อน) บอกตัวเลือก "เก่าสุด" ไว้ในหัว · LRU (แทนตัวที่ไม่ได้ใช้นานสุด) เขียนเลขกำกับ "ใช้ล่าสุดเมื่อไร" ใต้ช่อง · OPT กลับไปดูขวาของ string

I4. Belady's Anomaly (ปรากฏการณ์เพิ่ม RAM แล้ว fault เพิ่ม) — เพิ่ม frame (เฟรม (ก้อน RAM)) แล้ว fault กลับโต! 🤯

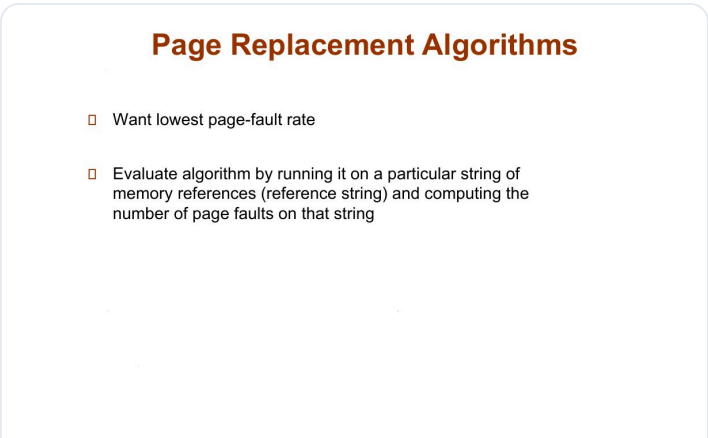
String: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5 (FIFO (เข้าก่อน-ออกก่อน)):

จำนวน frame (เฟรม (ก้อน RAM))	3 frames (เฟรมหลายก้อน)	4 frames (เฟรมหลายก้อน)
Page (หน้า) faults (เหตุการณ์เพจไม่อยู่ใน RAM)	**9**	**10** 😱

เพิ่ม RAM fault กลับเพิ่ม — เกิดได้กับ **FIFO (เข้าก่อน-ออกก่อน)** (และ non-stack algorithm) แต่**ไม่เกิดกับ LRU (แทนตัวที่ไม่ได้ใช้นานสุด) / Optimal (ดีที่สุดเชิงทฤษฎี)**



สไลด์ p.14 FIFO (เข้าก่อน-ออกก่อน) Illustrating Belady's Anomaly (ปรากฏการณ์เพิ่ม RAM แล้ว fault เพิ่ม)



สไลด์ p.11 Page Replacement (การเลือกเพจออกเมื่อ RAM เต็ม) Algorithms (กราฟ fault vs frames (เฟรมหลายก้อน))

I5. EAT (เวลาเข้าถึงเฉลี่ย) เมื่อมี Page Fault (เพจยังไม่อยู่ใน RAM) Rate (p)

```
EAT (เวลาเข้าถึงเฉลี่ย) = (1 − p) × memory (หน่วยความจำ) access time + p × page fault (เพจยังไม่อยู่ใน RAM) service time
```

ตัวเลขชวนคิด: mat = 200ns, fault service = 8ms → ที่ p = 0.001 (fault แค่ 0.1%) EAT (เวลาเข้าถึงเฉลี่ย) = 0.999×200 + 0.001×8,000,000 ≈ 8,200ns = 8.2µs — ช้าขึ้น 40 เท่า! ห้ามให้ p โต

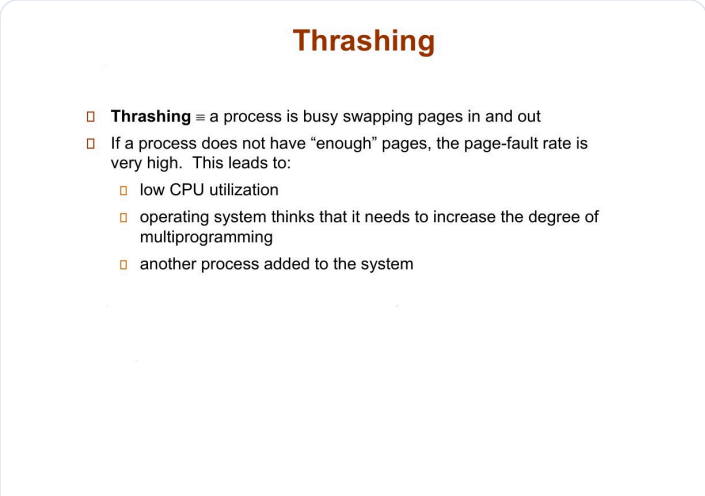
I6. Global vs Local Allocation

- **Global replacement**: แทนที่ได้จาก **frame (เฟรม (ก้อน RAM)) ใดก็ได้ในระบบ** — แย่งข้าม process (กระบวนการ) ได้
- **Local replacement**: เลือกจาก**เฉพาะ frame (เฟรม (ก้อน RAM)) ของตัวเอง** (คนละบ้านกับคนละบ้าน)

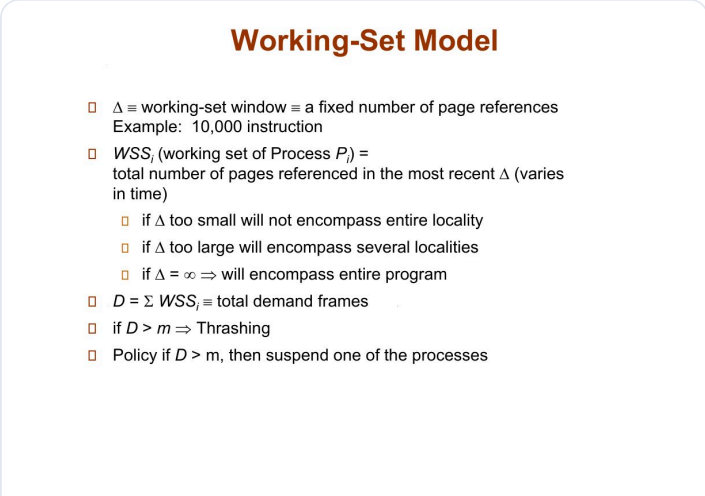
I7. Thrashing (สลับเพจหนักจนระบบแย่) · Locality · Working Set (ชุดเพจที่ต้องมีใน RAM) 🔥

- **Thrashing (สลับเพจหนักจนระบบแย่)**: process (กระบวนการ) มีเพจไม่พอ → สลับเพจเข้า-ออก disk ตลอดเวลา → **CPU utilization ดิ่ง** · OS เข้าใจผิดว่า "งานน้อย" เลยเพิ่ม process (กระบวนการ) → thrashing (สลับเพจหนักจนระบบแย่) ยิ่งหนัก (วงจรนรก) · เกิดทั้งข้ามงานและในงานเดียว (ลูปข้ามเพจ)
- **Locality of reference**: โปรแกรมมักอ้างกลุ่มเพจเดิมซ้ำ ๆ ในช่วงสั้น ๆ (อยู่ในฟังก์ชัน/ลูปเดิม)
- **Working set**: กลุ่มเพจที่ process (กระบวนการ) ต้องมีใน RAM เพื่อไม่ให้ fault ในช่วง locality นั้น — มองย้อนหลัง Δ (working-set window เช่น 10,000 instruction) · **WSS_i** = จำนวนเพจที่ P_i อ้างใน Δ · Δ เล็กไปครอบ locality ไม่หมด / ใหญ่ไปกินหลาย locality / Δ = ∞ คือทั้งโปรแกรม
- **D = Σ WSS_i** = ความต้องการ frame (เฟรม (ก้อน RAM)) รวม · ถ้า **D > m (frame (เฟรม (ก้อน RAM)) จริง) → thrashing (สลับเพจหนักจนระบบแย่)!** → OS ต้อง **suspend บาง process (กระบวนการ)** คืน frame (เฟรม (ก้อน RAM)) ให้คนที่เหลือ

> 🍳 **อุปมา**
> Working set = ของที่ต้องมีบนเคาน์เตอร์เพื่อทำเมนูให้จบ ถ้าวางไม่ครบ คุณวิ่งไปตู้เย็น (disk) ทุก 30 วินาที = ไม่มีอะไรเสร็จ → ทางแก้: ลดจำนวนเมนูที่ทำพร้อมกัน (suspend process (กระบวนการ))



สไลด์ p.19 Thrashing (สลับเพจหนักจนระบบแย่)



สไลด์ p.22 Working-Set Model (D = ΣWSSᵢ, D > m → suspend)

I8. โจทย์สุดคลาสสิก: Array 128×128 🔥 (สไลด์ p.24)

`int data[128][128]` · **1 แถว = 1 เพจ** · เก็บแบบ **row-major** (เรียงตามแถว)

	ลูป	พฤติกรรม	Page (หน้า) faults (เหตุการณ์เพจไม่อยู่ใน RAM)
❌ แย่	`for (j...) for (i...) data[i][j]=0;` (column นอก, row ใน)	ทุกครั้งกระโดดข้ามแถว = เปลี่ยนเพจทุกครั้ง	**128 × 128 = 16,384**
✅ ดี	`for (i...) for (j...) data[i][j]=0;` (row นอก, column ใน)	อยู่เพจเดียวกันทั้งแถว fault เฉพาะตอนเปลี่ยนแถว	**128**

โค้ดเหมือนกัน สลับลูปเท่านั้น → `fault` ต่างกัน 128 เท่า (16,384 : 128)

บทเรียน: เขียนลูปให้เข้าถึงข้อมูลต่อเนื่องตาม row-major = locality ดี

J. File System (ระบบไฟล์) (OSNo10)

📖 รู้จักส่วนนี้ก่อนอ่าน

ความหมาย: File System (ระบบไฟล์) = ผู้จัดการไฟล์บนดิสก์ — ตั้งชื่อ เก็บ ดึง แก้ไข จัด metadata และกำหนดสิทธิ์การเข้าถึง โดยที่เราไม่ต้องรู้เลยว่าข้อมูลอยู่เซกเตอร์ไหนของฮาร์ดดิสก์

ความสำคัญกับข้อสอบ: เทียบ FAT32 (ระบบไฟล์เก่า ไฟล์เกิน 4GB ไม่ได้)/NTFS (ระบบไฟล์ของ Windows (OS วินโดวส์))/ext4/exFAT (FAT32 (ระบบไฟล์เก่า ไฟล์เกิน 4GB ไม่ได้) ตัวใหม่ ไฟล์ใหญ่ได้)/APFS (ระบบไฟล์ของ Apple), วิธีจัดสรรบล็อก 3 แบบ, คำนวณ inode (โครงสร้างเก็บข้อมูลไฟล์ของ UNIX (ระบบยูนิกซ์ (บรรพบุรุษของ Linux (OS ลินุกซ์)))), hard link (ลิงก์ (ทางเชื่อม)) vs symbolic link (ลิงก์ (ทางเชื่อม)), สิทธิ์ rwx → ส่วนใหญ่ออกถูก/ผิด + โจทย์คำนวณสั้น ๆ

ใช้จริงในคอมพิวเตอร์: เลือกฟอร์แมตแฟลช (FAT32 (ระบบไฟล์เก่า ไฟล์เกิน 4GB ไม่ได้) เก็บไฟล์เกิน 4GB ไม่ได้!), คำสั่ง chmod/ls -l ที่เจอทุกวันใน Linux (OS ลินุกซ์), สร้าง shortcut ไฟล์ = symbolic link (ลิงก์ (ทางเชื่อม)) — ใช้ในชีวิตประจำวันจริง

J1. File System (ระบบไฟล์) รับผิดชอบอะไร

จัดการเก็บ/ดึง/จัดระเบียบข้อมูลบน storage — หน้าที่: **ตั้งชื่อ+โครงสร้างไฟล์ · เก็บข้อมูล · ดึง/แก้ไข · Metadata · ความปลอดภัย/สิทธิ์เข้าถึง** · หัวใจคือ **abstraction**: interface เข้าถึงไฟล์เหมือนกันไม่ว่าข้อมูลอยู่บน HDD/SSD/USB (device independence — จาก Quizizz 4 ข้อ 2)

J2. 5 ระบบไฟล์ยอดนิยม

ระบบไฟล์	ไฟล์สูงสุด	Volume สูงสุด	Journaling (สมุดบันทึกเพื่อกู้คืนหลังพัง)	ใช้ที่ไหน
FAT32 (ระบบไฟล์เก่า ไฟล์เกิน 4GB ไม่ได้) (1996)	**4 GB** 😅	8 TB	ไม่มี	USB/การ์ด — เข้ากันได้*ทุก*เครื่อง
NTFS (ระบบไฟล์ของ Windows (OS วินโดวส์)) (1993)	16 TB	256 TB	✅	Windows (OS วินโดวส์) — สิทธิ์/เข้ารหัส (EFS)/compression/quota
ext4 (2008)	16 TB	1 EB	✅	Linux (OS ลินุกซ์) หลัก (ต่อจาก ext3)
exFAT (FAT32 (ระบบไฟล์เก่า ไฟล์เกิน 4GB ไม่ได้) ตัวใหม่ ไฟล์ใหญ่ได้) (2006)	16 EB	128 PB	ไม่มี	USB/SD ใหญ่ข้าม OS — แก้ขีดจำกัด 4GB
APFS (ระบบไฟล์ของ Apple) (2017)	8 EB	8 EB	✅ + Snapshots	Apple ทั้งหมด — เข้ารหัสทั้งดิสก์

🎯 จาก Quizizz 4

• USB/external (ภายนอก (กระจายเป็นหลุม)) drive = **ExFAT** · • **Journaling (สมุดบันทึกเพื่อกู้คืนหลังพัง) ≠ เข้ารหัส!** — journaling คือบันทึก log ก่อนเขียนจริงเพื่อ**กู้คืนหลัง crash** · เข้ารหัสของ NTFS (ระบบไฟล์ของ Windows (OS วินโดวส์)) คือ **EFS** แยกกัน · • FAT32 (ระบบไฟล์เก่า ไฟล์เกิน 4GB ไม่ได้) ไฟล์เดี่ยว**ต้น 4GB เด็ดขาด** (ฟิลด์ 32 บิต) ดิสก์ว่างเยอะก็ไม่ช่วย

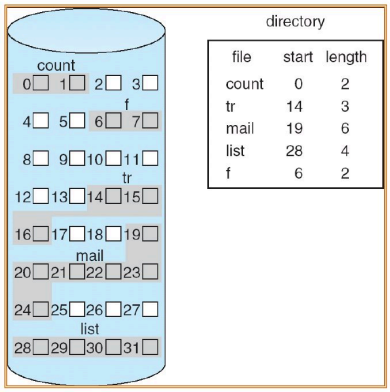
J3. File (ไฟล์) Allocation Methods 🔥 (วิธีจัดสรรบล็อกให้ไฟล์)

วิธี	หลักการ	ข้อดี	ข้อเสีย
Contiguous	บล็อก**ติดกันเป็นผืน** · directory เก็บ start + length	อ่านเร็วมาก (ไม่ต้อง seek)	**External fragmentation (การแตกเป็นเศษพื้นที่)** + ขยายไฟล์ยาก (บล็อกถัดไปถูกจอง = ต้องย้ายทั้งไฟล์)
Linked	บล็อกกระจาย + pointer (ตัวชี้ตำแหน่ง) ชี้บล็อกถัดไปเป็นโซ่ · directory เก็บ start + end	ไม่มี external (ภายนอก (กระจายเป็นหลุม)) frag · ขยายง่าย	**Random access ช้ามาก** (ไล่โซ่) · pointer (ตัวชี้ตำแหน่ง) เปลืองพื้นที่ · pointer (ตัวชี้ตำแหน่ง) พัง = ท่อนหลังหาย
Indexed	รวม pointer (ตัวชี้ตำแหน่ง) ทุกตัวไว้ใน **Index Block (บล็อก (หน่วยจัดเก็บ))** · directory ชี้มาจุดเดียว	**Random access เร็ว** · ไม่มี external (ภายนอก (กระจายเป็นหลุม)) frag	เปลืองบล็อก index (ไฟล์จิ๋วก็เสีย 1 บล็อก) · ไฟล์ใหญ่ต้อง multi-level

🎯 FAT (File (ไฟล์) Allocation Table (ตาราง))

FAT = ดึง pointer (ตัวชี้ตำแหน่ง) ทั้งหมดออกจาก data (ข้อมูล) block (บล็อก (หน่วยจัดเก็บ)) มารวมไว้ใน**ตารางเดียว (อยู่ใน RAM)** → ไล่โซ่ได้เร็วโดยไม่ต้องเปิดดิสก์ทีละบล็อก — นี่คือที่มาของชื่อ FAT filesystem

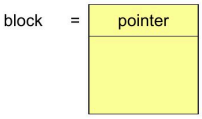
Contiguous Allocation of Disk Space



สไลด์ p.19 Contiguous Allocation (การจัดสรรบล็อกติดกัน)

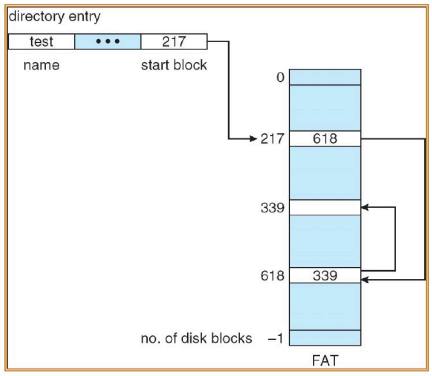
Linked Allocation

- Each file is a linked list of disk blocks: blocks may be scattered anywhere on the disk.



สไลด์ p.20 Linked Allocation (การจัดสรรแบบโซ่ pointer (ตัวชี้ตำแหน่ง))

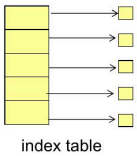
File-Allocation Table



สไลด์ p.24 File (ไฟล์)-Allocation Table (ตาราง)

Indexed Allocation

- Brings all pointers together into the *index block*.
- Logical view.



สไลด์ p.25 Indexed Allocation (การจัดสรรแบบสารบัญ)

J3.1 คำนวณ index (สารบัญ (ตัวชี้ตำแหน่ง)) / inode (โครงสร้างเก็บข้อมูลไฟล์ของ UNIX (ระบบยูนิกซ์ (บรรพบุรุษของ Linux (OS ลินุกซ์)))) 🔥 (สูตรเดียวใช้ทุกโจทย์)

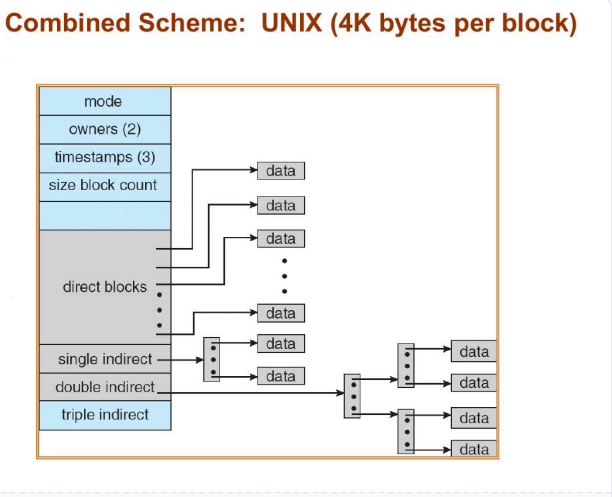
`Entries ต่อ 1 index (สารบัญ (ตัวชี้ตำแหน่ง)) block (บล็อก (หน่วยจัดเก็บ)) = Block (บล็อก (หน่วยจัดเก็บ)) Size ÷ Pointer Size`

จาก Quizizz 4: บล็อก 2KB ÷ pointer (ตัวชี้ตำแหน่ง) 4B = **512 entries** → ไฟล์ใหญ่สุด single-level = 512 × 2KB = 1MB

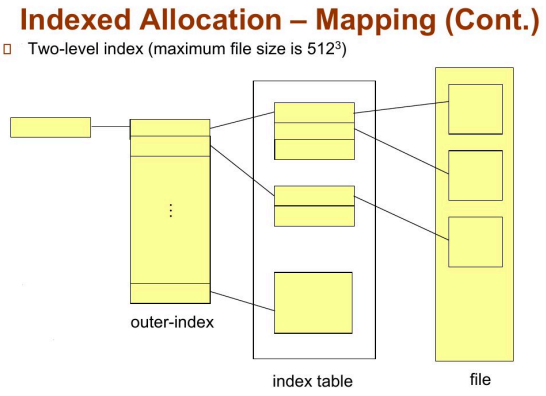
UNIX (ระบบยูนิกซ์ (บรรพบุรุษของ Linux (OS ลินุกซ์))) Combined Scheme — inode (สไลด์ p.29, block (บล็อก (หน่วยจัดเก็บ)) 4KB, pointer (ตัวชี้ตำแหน่ง) 4B): มี pointer (ตัวชี้ตำแหน่ง) 15 ตัว

Pointer	จำนวน	คำนวณ	ขนาด
Direct ชี้ data (ข้อมูล) ตรง ๆ	12	12 × 4KB	**48 KB**
Single indirect	1	(4KB÷4B) × 4KB = 1024 × 4KB	**4 MB**
Double indirect	1	1024^2 × 4KB	**4 GB**
Triple indirect	1	1024^3 × 4KB	**4 TB**
ไฟล์ใหญ่สุด ≈ 48KB + 4MB + 4GB + 4TB			**≈ 4 TB**

> **🎯 จุดพลาด**
> ลืมหา "1024 ช่องต่อบล็อก" ก่อน / ลืมบวก direct 48KB / บวกเลขผิดหลัก (4MB ≠ 4×1024KB=4096KB ก็คือ 4MB ตรงกัน แต่ double = 1024×1024×4KB = 4GB)



สไลด์ p.29 Combined Scheme: UNIX (4K/block (บล็อก (หน่วยจัดเก็บ))) — 15 pointers (ตัวชี้ตำแหน่ง)



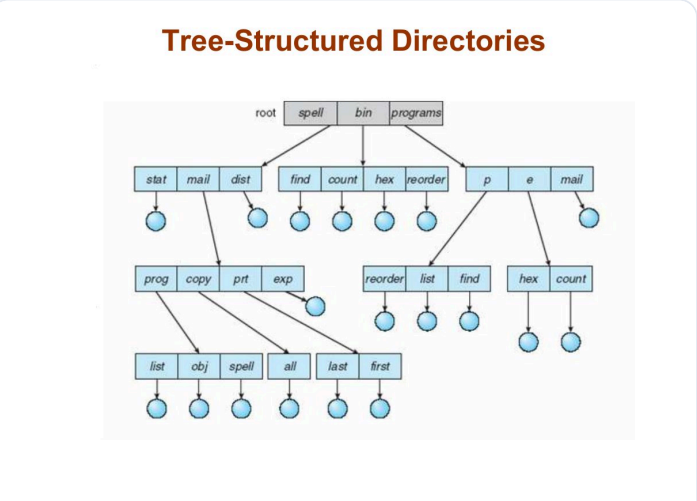
สไลด์ p.28 Two-level index (512^3 ในสไลด์ = block (บล็อก (หน่วยจัดเก็บ)) 512B)

J4. Directory (โฟลเดอร์/สารบัญไฟล์) · Tree vs Acyclic Graph

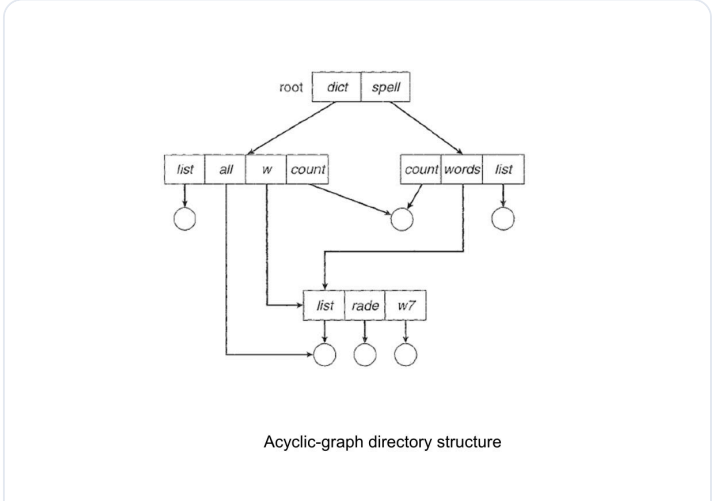
- **Directory (โฟลเดอร์/สารบัญไฟล์)** = symbol table (ตาราง) แปลง "ชื่อไฟล์ → ข้อมูลไฟล์" · operations: search, create, delete, list, rename, traverse
- **Tree**: ไฟล์มี parent เดียว — ปลอดภัยแต่**แชร์ไม่ได้**
- **Acyclic graph** (ไม่มีวงจร): แชร์ได้ผ่าน **link (ลิงก์ (ทางเชื่อม))** — ปัญหา: aliasing (ไฟล์เดียวหลายชื่อ), การลบไฟล์แชร์

การลบไฟล์แชร์ 2 แนวทาง

แนวทาง	ทำอย่างไร	ข้อควรระวัง
ลบเมื่อไรก็ได้	ลบไฟล์ทันทีที่ใครสั่ง	**Dangling pointer (ตัวชี้ตำแหน่ง)** — link (ลิงก์ (ทางเชื่อม)) ที่เหลือชี้หาอะไรที่ไม่มีแล้ว (แนวทาง symbolic link (ลิงก์ (ทางเชื่อม)))
เก็บจน reference หมด	นับ **reference count (ตัวนับจำนวน)**: เพิ่ม link (ลิงก์ (ทางเชื่อม)) → count (ตัวนับจำนวน)+1, ลบ link (ลิงก์ (ทางเชื่อม)) → count (ตัวนับจำนวน)−1, **count (ตัวนับจำนวน) = 0 ค่อยปล่อยพื้นที่**	ต้องตรวจว่ากราฟไม่มีวงวน (cycle (วงจร (วนกลับมาที่เดิม))) — แนวทาง hard link (ลิงก์ (ทางเชื่อม)) ของ UNIX (ระบบยูนิกซ์ (บรรพบุรุษของ Linux (OS ลินุกซ์)))



สไลด์ p.32 Tree-Structured Directories



สไลด์ p.33 Acyclic-graph Directory (โฟลเดอร์/สารบัญไฟล์)

J5. Hard Link (ลิงก์ชี้ inode (โครงสร้างเก็บข้อมูลไฟล์ของ UNIX (ระบบยูนิกซ์ (บรรพบุรุษของ Linux))) เดียวกัน) vs Symbolic (Soft) Link (ลิงก์ (ทางเชื่อม)) 🔥

	Hard Link (ลิงก์ชี้ inode (โครงสร้างเก็บข้อมูลไฟล์ของ UNIX (ระบบยูนิกซ์ (บรรพบุรุษของ Linux))) เดียวกัน) `ln original new`	**Symbolic Link (ลิงก์แบบเก็บ path (shortcut))** `ln -s original new`
ชี้ไปที่	directory entry ชี้**inode (โครงสร้างเก็บข้อมูลไฟล์ของ UNIX (ระบบยูนิกซ์ (บรรพบุรุษของ Linux (OS ลินุกซ์)))) เดียวกัน**โดยตรง + เพิ่ม link (ลิงก์ (ทางเชื่อม)) count (ตัวนับจำนวน)	**ไฟล์ใหม่แยกต่างหาก** เก็บ "path" ของต้นทาง (shortcut)
ลบต้นฉบับแล้ว	ข้อมูลยังอยู่ — ปล่อยจริงเมื่อ **link (ลิงก์ (ทางเชื่อม)) count (ตัวนับจำนวน) = 0**	กลายเป็น **broken/dangling link (ลิงก์ (ทางเชื่อม))**
ข้าม partition/FS	❌ ห้าม	✅ ได้
ชี้ directory	❌ ห้าม (กัน cycle (วงจร (วนกลับมาที่เดิม)))	✅ ได้

🏠 อุปมา

Hard link (ลิงก์ (ทางเชื่อม)) = ลงทะเบียนบ้านเพิ่มอีกชื่อ — บ้านหลังเดิม อยู่ต่อจนคนทะเบียนหมด · Soft link (ลิงก์ (ทางเชื่อม)) = กระดาษโน้ตเขียนว่า "บ้านอยู่ซอย 5" — โยนบ้านทิ้ง กระดาษก็ชี้อากาศ

🎯 จาก Quizizz 4

• Symbolic link (ลิงก์ (ทางเชื่อม)) อีกชื่อคือ **soft link (ลิงก์ (ทางเชื่อม))** ไม่ใช่ hard link (ลิงก์ (ทางเชื่อม)) · • Hard link (ลิงก์ (ทางเชื่อม)) จัดการด้วย **Reference Counter** ✓ · • **ไม่มีคำสั่ง hardln!** — จริงคือ `ln` (hard) / `ln -s` (symbolic)

Hard Link

- A hard link is a direct reference to the inode of a file.
- Each hard link behaves like an independent file, but in reality, it is pointing to the same data (inode) on the disk.
- In essence, multiple filenames point to the same physical data, making it possible to access the file via any of those names.

สไลด์ p.42 Hard Link (ลิงก์ชี้ inode (โครงสร้างเก็บข้อมูลไฟล์ของ UNIX (ระบบยูนิกซ์ (บรรพบุรุษของ Linux))) เดียวกัน)

Symbolic Link (Soft Link)

- A symbolic link (or soft link) is a file that contains a path to another file or directory.
- Instead of pointing directly to the inode, a symbolic link acts as a shortcut or reference to the original file.

สไลด์ p.44 Symbolic Link (Soft Link (ลิงก์ (ทางเชื่อม)))

J6. Mounting (การติดตั้งดิสก์เข้าระบบไฟล์)

- **Mounting (การติดตั้งดิสก์เข้าระบบไฟล์)** = ติด filesystem ของอุปกรณ์ (partition/USB/network) เข้า directory tree ที่ **mount (ติดตั้งเข้าระบบไฟล์) point** เช่น `mount /dev/sdb1 /mnt/usb`
- ก่อนถอดต้อง `umount` — ให้การเขียนค้าง ๆ เสร็จก่อน ไม่งั้นข้อมูลพัง
- จาก Quizizz 4: อุปกรณ์เดียว **mount (ติดตั้งเข้าระบบไฟล์) ได้หลาย mount (ติดตั้งเข้าระบบไฟล์) point** (ข้อความว่า "ไม่สามารถ" = เท็จ)

File System Mounting

- Mounting a file system refers to making a file system accessible for users and programs on a computer.
- In practical terms, it means attaching a storage device (such as a hard drive, SSD, USB drive, or network file system) to a specific location (or directory) in the system's directory tree so that its contents can be accessed.

สไลด์ p.46 File System (ระบบไฟล์) Mounting (การติดตั้งดิสก์เข้าระบบไฟล์)

How Mounting Works?

- Device: A physical device, like a hard disk or a partition on that disk, must be mounted to access its contents.
- The device typically has an identifier like /dev/sda1 for a specific partition of the hard disk.
- Mounting Command: In Linux/Unix systems, the mount command is used. The basic syntax is:
  - mount [OPTIONS] DEVICE MOUNT_POINT
  - Example: *mount /dev/sdb1 /mnt/usb*
    - This command mounts the device /dev/sdb1 (a USB drive in this case) to the directory /mnt/usb.
- Unmounting Command: When you're done with the device, you unmount it with the umount command:
  - *umount /mnt/usb*

สไลด์ p.48 How Mounting (การติดตั้งดิสก์เข้าระบบไฟล์) Works? (mount (ติดตั้งเข้าระบบไฟล์)/umount command)

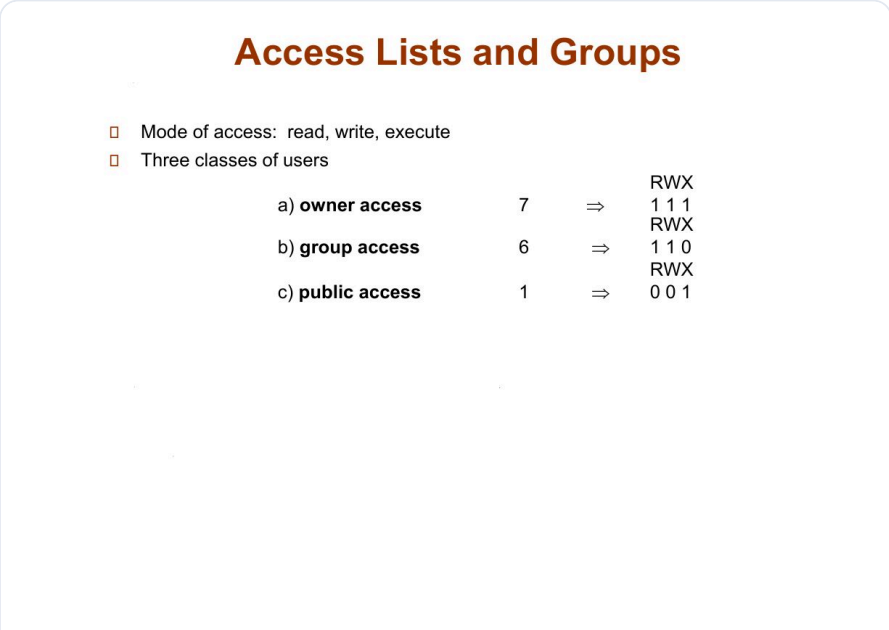
J7. Protection — สิทธิ์ RWX 🔥

เจ้าของไฟล์คุมได้ 2 อย่าง: **what can be done** (read/write/execute) กับ **by whom** (owner/group/others)

$$R = 2^2 = 4 \cdot W = 2^1 = 2 \cdot X = 2^0 = 1$$

3 กลุ่ม: owner, group, other (public) — ตัวอย่างสไลด์ p.50: owner=7 (rwx), group=6 (rw-), public=1 (--x) = 761

เลข	บวกได้จาก	แปลง
7	4+2+1	rwx — ทุกสิทธิ์
6	4+2	rw-
5	4+1	r-x (ไม่ใช่ 6! จาก Quizizz)
4	4	r--
0	—	--- ไม่มีสิทธิ์



สไลด์ p.50 Access Lists and Groups (RWX 7 6 1)

K. ความรู้จาก Quizizz ในห้องเรียน (4 ชุด) — จัดเรียงตามชุด

📖 รู้จักส่วนนี้ก่อนอ่าน

ความหมาย: ความรู้ที่สกัดจากแบบควิซในห้องเรียนจริง 4 ชุด (Signal/Pipe, Shared Memory (หน่วยความจำที่แชร์กัน), Semaphore (ตัวนับควบคุมสิทธิ์เข้าถึง), File (ไฟล์) Management) — ทุกข้อมี "ความรู้สำคัญ" ที่ตอบโจทย์แบบเดิมได้ทันที

ความสำคัญกับข้อสอบ: นี่คือธนาคารข้อสอบย่อที่ตรงสไตล์อาจารย์ที่สุด — หลายข้อมีกับดักที่อาจารย์ตั้งใจหลอก (เช่น ข้อ 8 ชุด 4 มีคีย์ขัดตำรา — ต้องตอบ False ตามคีย์!) อ่านก่อนสอบคุ้มที่สุด

ใช้จริงในคอมพิวเตอร์: ประเด็นในควิซคือข้อผิดพลาดจริงที่โปรแกรมเมอร์พลาดบ่อย เช่น เข้าใจว่า signal() ใช้ส่งสัญญาณ, ลืมปิดปลาย pipe (ท่อส่งข้อมูลทางเดียว) — รู้ไว้เขียนโค้ดจริงไม่พัง

ทุกข้อเป็นความรู้ที่ตอบโจทย์แบบเดิมได้ทันที — อ่านกล่อง "ความรู้สำคัญ" ต่อข้อ

K1. Quiz 1 — IPC & Unix (ระบบยูนิกซ์) Signal

ข้อ 2 **False**

"Signal ใช้ส่งข้อความระหว่าง process (กระบวนการ)"

ความรู้: Signal = **event notification** เท่านั้น — ไม่มี payload แนบข้อมูลไม่ได้ ผู้รับรู้แค่ว่า "signal (สัญญาณแจ้งเหตุการณ์) ชนิด X มา" · ส่งข้อมูลจริงใช้ Message Passing (การส่งข้อความผ่าน OS) / Shared Memory (หน่วยความจำที่แชร์กัน)

ข้อ 3 **Message Passing (การส่งข้อความผ่าน OS)**

"IPC ที่ OS จัดจังหวะการรับส่งให้"

ความรู้: Message passing ให้ OS จัดทั้ง send/receive และ**จังหวะ** (blocking/non-blocking) · Shared Memory (หน่วยความจำที่แชร์กัน) จัดเตรียมแค่พื้นที่ — **ซิงก์จังหวะเป็นหน้าที่ของ process (กระบวนการ) เอง** (เช่นใช้ semaphore (ตัวนับควบคุมสิทธิ์เข้าถึง))

ข้อ 4 **SIGINT**

"Ctrl-C ส่ง signal (สัญญาณแจ้งเหตุการณ์) อะไร"

ความรู้: Ctrl-C → **SIGINT** (INT = INTerrupt) · คู่เทียบ: SIGFPE = หารศูนย์, SIGTERM = ขอจบสุภาพ (default (ค่าเริ่มต้น) ของ kill), SIGQUIT = Ctrl-\, SIGKILL = ฆ่าทันทีจับไม่ได้ (9)

ข้อ 5 **False**

"signal() ใช้ส่ง signal (สัญญาณแจ้งเหตุการณ์) ระหว่าง process (กระบวนการ)"

ความรู้: `signal()` = **ลงทะเบียน handler (ฟังก์ชันรับมือเหตุการณ์)** ("โดน X แล้วทำอะไร") · ตัวส่งคือ `kill(pid, sig)` (ส่งข้าม process (กระบวนการ)) และ `raise(sig)` (ส่งให้ตัวเอง)

ข้อ 6 **False**

"1 process (กระบวนการ) จัดการ signal (สัญญาณแจ้งเหตุการณ์) ได้แค่ 1 ตัว"

ความรู้: จัดการได้**หลาย signal (สัญญาณแจ้งเหตุการณ์)** — แต่ละชนิดมี handler (ฟังก์ชันรับมือเหตุการณ์) ของตัวเอง · ข้อจำกัดจริง: 1 ชนิดมี 1 handler (ลงทะเบียนซ้ำ = ทับตัวเดิม)

ข้อ 7 **False**

"kill() ใช้ส่งได้แค่ SIGKILL"

ความรู้: System (ระบบ) call `kill(pid, sig)` ส่งได้**ทุกชนิด** — SIGKILL แค่ 1 ตัวเลือก (ชื่อมาจากประวัติศาสตร์ ทำให้เข้าใจผิด)

ข้อ 8 **True**

"คำสั่ง kill ใน UNIX (ระบบยูนิกซ์ (บรรพบุรุษของ Linux (OS ลินุกซ์))) ใช้ส่ง signal (สัญญาณแจ้งเหตุการณ์) ให้ process (กระบวนการ)"

ความรู้: หน้าที่จริงของคำสั่ง kill = **ส่ง signal (สัญญาณแจ้งเหตุการณ์)** (default (ค่าเริ่มต้น) SIGTERM) · `kill -9 pid` = SIGKILL · ⚠️ แยกจากข้อ 7: system call ส่งได้ทุกชนิด (โจทย์ใส่ "เท่านั้น" มาหลอก)

K2. Quiz 2 — Shared Memory (หน่วยความจำที่แชร์กัน) & Message Passing (Pipe)

ข้อ 2 False

"Message passing รองรับทิศทางเดียว"

ความรู้: ทิศเดียวจริง ๆ คือ **pipe (ท่อส่งข้อมูลทางเดียว) ธรรมดา (anonymous)** — สองทิศต้อง 2 pipe (ท่อส่งข้อมูลทางเดียว) · Message passing แต่ละ process (กระบวนการ) มีทั้ง send/receive = **สองทิศได้**

ข้อ 3 อาเรย์ int 2 ตัว

"พารามิเตอร์ของ pipe()"

ความรู้: `int pfd[2]; pipe(pfd);` — OS ใส่ค่า 2 ช่อง: **pfd[0] = ปลายอ่าน, pfd[1] = ปลายเขียน** (file (ไฟล์) descriptor ไม่ใช่ชื่อไฟล์ — pipe (ท่อส่งข้อมูลทางเดียว) ธรรมดาไม่มีชื่อ)

ข้อ 4 write(pfd[1], data (ข้อมูล), length)

"ประโยคส่งข้อมูลเข้า pipe (ท่อส่งข้อมูลทางเดียว)"

ความรู้: pipe (ท่อส่งข้อมูลทางเดียว) ใช้ system call ไฟล์มาตรฐาน: **write()** ส่ง / **read()** รับ · เขียนต้องลงปลายเขียน **pfd[1]** · จำ: "write to 1, read from 0"

ข้อ 5 dup2(pfd[1], 1)

"เปลี่ยนทิศ stdout ไปปลายเขียน pipe (ท่อส่งข้อมูลทางเดียว)"

ความรู้: `dup2(oldfd, newfd)` = ทำให้ **newfd** ชี้ที่เดียวกับ oldfd · fd 0=stdin, 1=stdout, 2=stderr → stdout(1) ไหลเข้าปลายเขียน = `dup2(pfd[1],1)` · pattern มาตรฐาน: ฝั่งเขียน dup2(pfd[1],1) แล้ว exec (เปลี่ยนโปรแกรมที่ process (กระบวนการ) รัน) / ฝั่งอ่าน dup2(pfd[0],0) · ปิดปลายที่ไม่ใช้ก่อน

ข้อ 6 shmget()

"System (ระบบ) call ขอสร้าง shared memory (หน่วยความจำ)"

ความรู้: shm**get** = get — ขอสร้าง/อ้างถึง segment ได้ **shmid** · (shmat=attach, shmdt=detach, shmctl=control/ลบ)

ข้อ 7 ftok()

"ฟังก์ชันสร้าง key ของ shared memory (หน่วยความจำ)"

ความรู้: `key_t ftok(path, id)` — path + id เดียวกัน → key เดียวกัน → อ้าง shm ก้อนเดียวกันได้ (นิยม path = ".")

ข้อ 8 0764

"rwxrw-r-- เป็นเลขฐานแปดอะไร"

ความรู้: r=4 w=2 x=1 → rwx=7, rw-=6, r--=4 → **0764** (ใช้กับ shmget flag เช่น IPC_CREAT | 0666 ด้วย)

ข้อ 9 IPC_PRIVATE

"shmget ที่การันตีไม่ซ้ำกับใครแน่นอน"

ความรู้: IPC_PRIVATE = key พิเศษ OS สร้าง segment ใหม่ไม่ซ้ำใครเสมอ — นิยมใช้ parent→child (shmid ส่งต่อได้ในตระกูล) · IPC_EXCL แค่ flag กันชน ไม่การันตี

ข้อ 10 shared memory (หน่วยความจำ) id

"shmget() คืนอะไร"

ความรู้: คืน **shmid** (เลขจำนวนเต็ม) — เอาไปใช้กับ shmat/shmctl/shmdt · flow: key → shmget → **shmid** → shmat → pointer (ตัวชี้ตำแหน่ง)

ข้อ 11 shmat()

"ฟังก์ชันให้ตัวแปรอ้างถึงตำแหน่ง shared memory (หน่วยความจำ)"

ความรู้: shm**at** = attach — ผูก segment เข้า address (ที่อยู่หน่วยความจำ) space แล้ว**คืน pointer (ตัวชี้ตำแหน่ง):** `shm_ptr = shmat(shmid, NULL, 0)`

ข้อ 12 False

"ตัวแปรเก็บตำแหน่ง shm ต้องเป็น pointer (ตัวชี้ตำแหน่ง) ไป int เท่านั้น"

ความรู้: shmat คืน **void*** — cast เป็นชนิดไหนก็ได้ (char*, struct data (ข้อมูล)* ...) ข้อจำกัดเดียวคือต้องเป็น pointer (ตัวชี้ตำแหน่ง)

ข้อ 13 struct data (ข้อมูล) *shm_ptr;

"shmget(..., sizeof(struct data (ข้อมูล)), ...) ตัวแปรชี้ต้องประกาศยังไง"

ความรู้: จองขนาด sizeof(struct data (ข้อมูล)) → pointer (ตัวชี้ตำแหน่ง) ต้องตรงชนิด: `struct data *p; p = (struct data *) shmat(shmid, NULL, 0);` ใช้ผ่าน `p->field`

🟢 Shared Memory (หน่วยความจำที่แชร์กัน) ครบวงจร (จำ 6 ขั้น)

```
key_t key = ftok(".", 'a');                    // 1. สร้าง key
int shmid = shmget(key, sizeof(struct data), IPC_CREAT | 0666); // 2. สร้าง segment
struct data *p = (struct data *) shmat(shmid, NULL, 0);   // 3. attach
/* 4. อ่าน/เขียนผ่าน p */
shmdt(p);                                      // 5. detach
shmctl(shmid, IPC_RMID, NULL);                 // 6. ลบ (คนสุดท้าย)
```

K3. Quiz 3 — Semaphore (ตัวนับควบคุมสิทธิ์เข้าถึง)

ข้อ 2 False

"Semaphore (ตัวนับควบคุมสิทธิ์เข้าถึง) ที่ใช้กับ critical section คือ counting"

ความรู้: CS (ช่วงวิกฤต) ใช้ **binary semaphore (mutex)** เริ่ม 1: wait (รอ + หยิบสิทธิ์) → CS (ช่วงวิกฤต) → signal (สัญญาณแจ้งเหตุการณ์) · counting (เริ่ม = จำนวน resource (ทรัพยากร)) ใช้กับ resource (ทรัพยากร) หลายหน่วย เช่น bounded buffer

ข้อ 3 False

"Semaphore (ตัวนับควบคุมสิทธิ์เข้าถึง) การันตีผ่านเงื่อนไข 3 ข้อของ CS (ช่วงวิกฤต)"

ความรู้: 3 เงื่อนไข = Mutual Exclusion (เข้าถึงได้คนเดียว) / Progress / Bounded Waiting (รอได้จำกัดจำนวนครั้ง) — semaphore (ตัวนับควบคุมสิทธิ์เข้าถึง) การันตีได้แค่ **ข้อ 1** · progress/bounded waiting ขึ้นกับ implementation (classic อาจ starvation)

ข้อ 4 2 ตัว

"bounded buffer ใช้ counting semaphore (ตัวนับควบคุมสิทธิ์เข้าถึง) กี่ตัว"

ความรู้: ใช้รวม 3 ตัว: empty(n), full(0), mutex(1) — แต่**counting จริงมี 2** (empty, full) เพราะ mutex เป็น binary

ข้อ 5 empty

"producer ตรวจช่องว่างด้วยตัวไหน"

ความรู้: Producer (ฝั่งผู้ผลิตข้อมูล): `wait(empty)` ขอช่องว่างก่อน · Consumer (ฝั่งผู้บริโภคข้อมูล): `wait(full)` ก่อนอ่าน · mutex ใช้ตอนแตะ buffer จริง (ลำดับ: resource (ทรัพยากร) ก่อน mutex!)

ข้อ 6 mutex

"readers-writers คุมจำนวน reader ด้วย semaphore (ตัวนับควบคุมสิทธิ์เข้าถึง) ไหน"

ความรู้: mutex ครอบช่วง readcount++/-- · **readcount เป็น int ธรรมดา** ไม่ใช่ semaphore (ตัวนับควบคุมสิทธิ์เข้าถึง) · wrt คุม writer / reader แรกล็อก-ตัวสุดท้ายปลด

ข้อ 7 resource (ทรัพยากร)

"ตะเกียบใน Dining philosopher แทนอะไร"

ความรู้: ตะเกียบ = **shared resource (ทรัพยากร)** (นักปราชญ์ = process (กระบวนการ)) · implement ใช้ semaphore (ตัวนับควบคุมสิทธิ์เข้าถึง) 1 ตัวต่อ 1 ตะเกียบ แต่ตัวตนของมันคือ resource (ทรัพยากร)

ข้อ 8 False

"semaphore (ตัวนับควบคุมสิทธิ์เข้าถึง) การันตีนักปราชญ์ทุกคนได้กิน"

ความรู้: หยิบซ้ายพร้อมกัน → รอขวาเป็นวง = **deadlock (การติดตาย (รอกันตลอดกาล))** + ไม่มี bounded waiting → starvation · แก้: จำกัด ≤ 4 คน / คนนึงหยิบขวาก่อน / ใช้ Monitor

ข้อ 9 sem_post()

"PThread semaphore (ตัวนับควบคุมสิทธิ์เข้าถึง) ตัวไหน = signal()"

ความรู้: PThread: `sem_init(&s,0,val)` สร้าง · **sem_wait = P (ลด)** · **sem_post = V (เพิ่ม/ปลุก)** · sem_destroy เก็บกวาด · **ไม่มี sem_signal ใน POSIX!**

K4. Quiz 4 — Introduction to File (ไฟล์) Management

ข้อ 2 True

"เข้าถึงไฟล์เหมือนกันไม่ว่าอุปกรณ์อะไร"

ความรู้: ระบบไฟล์ = ชั้น abstraction → open/read/write ชุดเดียวกันทุกอุปกรณ์ (**device independence**)

ข้อ 3 False

"Journaling (สมุดบันทึกเพื่อกู้คืนหลังพัง) = การเข้ารหัสไฟล์"

ความรู้: Journaling (สมุดบันทึกเพื่อกู้คืนหลังพัง) = write-ahead log เพื่อ**กู้คืนหลัง crash** · การเข้ารหัสเป็นฟีเจอร์แยก (NTFS (ระบบไฟล์ของ Windows (OS วินโดวส์)) ใช้ **EFS**)

ข้อ 4 **ExFAT**

"ระบบไฟล์สำหรับ USB drive"

ความรู้: USB/external (ภายนอก (กระจายเป็นหลุม)) = **ExFAT** (ใหญ่กว่า 4GB ข้าม Windows (OS วินโดวส์)-macOS) · NTFS (ระบบไฟล์ของ Windows (OS วินโดวส์))=Windows (OS วินโดวส์), ext4=Linux (OS ลินุกซ์), APFS (ระบบไฟล์ของ Apple)=Apple

ข้อ 5 **False**

"contiguous ปัญหาคือ internal fragmentation (การแตกเป็นเศษพื้นที่)"

ความรู้: contiguous → **external (ภายนอก (กระจายเป็นหลุม))** fragmentation (ต้องกันบล็อกติดกัน พื้นที่แตกเป็นหลุม) · internal = เศษในบล็อกสุดท้ายที่ไม่เต็ม

ข้อ 6 **False**

"linked allocation เป็นการจัดสรรแบบต่อเนื่อง"

ความรู้: linked = บล็อกกระจาย + pointer (ตัวชี้ตำแหน่ง) โซ่ = **non-contiguous** โดยนิยาม (FAT อยู่ตระกูลนี้)

ข้อ 7 **False**

"บล็อกใหญ่ → external (ภายนอก (กระจายเป็นหลุม)) fragmentation (การแตกเป็นเศษพื้นที่)"

ความรู้: บล็อกใหญ่ → เศษในบล็อกสุดท้ายเยอะ = **internal fragmentation (การแตกเป็นเศษพื้นที่)** (ไฟล์ 1B ในบล็อก 4KB = เสีย ~4KB)

ข้อ 8 **False (ตามคีย์!)**

"single-level index (สารบัญ (ตัวชี้ตำแหน่ง)): ขนาดไฟล์สูงสุดถูกจำกัดด้วยจำนวนช่องใน index (สารบัญ (ตัวชี้ตำแหน่ง)) block (บล็อก (หน่วยจัดเก็บ))"

ความรู้: ⚠️**คีย์ผู้สอน = False** (แม้ตามตำราน่าจะ True) — ตอบ False ตามคีย์ · ความรู้จริง: ไฟล์ใหญ่สุด = entries × block (บล็อก (หน่วยจัดเก็บ)) size (ขนาด)

ข้อ 9 **512**

"บล็อก 2KB, pointer (ตัวชี้ตำแหน่ง) 4B → 1 index (สารบัญ (ตัวชี้ตำแหน่ง)) block (บล็อก (หน่วยจัดเก็บ)) มีกี่ช่อง"

ความรู้: entries = 2048 ÷ 4 = **512** → single-level ใหญ่สุด 512 × 2KB = 1MB · สูตรเดียวกับ inode (โครงสร้างเก็บข้อมูลไฟล์ของ UNIX (ระบบยูนิกซ์ (บรรพบุรุษของ Linux (OS ลินุกซ์)))) ของ J3.1

ข้อ 10 **False**

"Symbolic link (ลิงก์ (ทางเชื่อม)) เรียกอีกอย่างว่า hard link (ลิงก์ (ทางเชื่อม))"

ความรู้: symbolic = **soft link (ลิงก์ (ทางเชื่อม))** (ไฟล์แยกเก็บ path) · hard link (ลิงก์ (ทางเชื่อม)) = entry ชี้ inode (โครงสร้างเก็บข้อมูลไฟล์ของ UNIX (ระบบยูนิกซ์ (บรรพบุรุษของ Linux (OS ลินุกซ์)))) เดียวกัน

ข้อ 11 **True**

"Hard link (ลิงก์ (ทางเชื่อม)) จัดการด้วย Reference Counter"

ความรู้: inode (โครงสร้างเก็บข้อมูลไฟล์ของ UNIX (ระบบยูนิกซ์ (บรรพบุรุษของ Linux (OS ลินุกซ์)))) เก็บ **link (ลิงก์ (ทางเชื่อม)) count (ตัวนับจำนวน)** — rm แค่ลด counter ปล่อยพื้นที่เมื่อ = 0

ข้อ 12 **False**

"คำสั่ง hardln ใช้สร้าง Hard Link (ลิงก์ชี้ inode (โครงสร้างเก็บข้อมูลไฟล์ของ UNIX (ระบบยูนิกซ์ (บรรพบุรุษของ Linux))) เดียวกัน)"

ความรู้: ไม่มี hardln! จริงคือ `ln target link` (hard) / `ln -s target link` (symbolic)

ข้อ 13 **False**

"mount (ติดตั้งเข้าระบบไฟล์) อุปกรณ์เดียวกันได้แค่ 1 mount (ติดตั้งเข้าระบบไฟล์) point"

ความรู้: mount (ติดตั้งเข้าระบบไฟล์) อุปกรณ์เดียวได้**หลาย mount (ติดตั้งเข้าระบบไฟล์) point** (รองรับ bind mount (ติดตั้งเข้าระบบไฟล์)) — "ไม่สามารถ" เป็นเท็จ

L. แผ่นสูตรเร็วท้ายเล่ม (อ่าน 3 นาทีก่อนเข้าห้อง)

📖 รู้จักส่วนนี้ก่อนอ่าน

ความหมาย: สรุปสูตรคำนวณทุกบทมารวมใน 1 หน้า (TAT/WT, Paging (การแบ่งหน่วยความจำเป็นหน้า), EAT (เวลาเข้าถึงเฉลี่ย), นับ fault, Banker (อัลกอริทึมนายธนาคาร)'s, inode (โครงสร้างเก็บข้อมูลไฟล์ของ UNIX (ระบบยูนิกซ์ (บรรพบุรุษของ Linux (OS ลินุกซ์)))), rwx)

ความสำคัญกับข้อสอบ: เป็น index (สารบัญ (ตัวชี้ตำแหน่ง)) ระหว่างทำข้อสอบ — ตันหัวข้อไหนกลับมาเปิดหน้านี้ก่อน แล้วค่อยไล่ไปส่วนเต็มที่อยู่หน้าก่อน ๆ

ใช้จริงในคอมพิวเตอร์: สูตรพวกนี้คือสิ่งที่ OS คำนวณจริงทุกมิลลิวินาที (แปลง address (ที่อยู่หน่วยความจำ) ทุกครั้งที่เข้าถึง RAM) — เข้าใจสูตร = เข้าใจการทำงานเครื่อง

⏱️ CPU Scheduling (การจัดคิว CPU)

TAT	Completion − Arrival
WT	TAT − Burst (หา TAT ก่อนเสมอ!)
RR (วนแบ่งเวลา (Round Robin (วนแบ่งเวลาทีละส่วน)))	รอไม่เกิน (n−1)q · q ใหญ่=FCFS (มาก่อน-รันก่อน)
เท่ากัน	เลือกมาถึงก่อน (FCFS (มาก่อน-รันก่อน) tiebreak)

🧮 Paging (การแบ่งหน่วยความจำเป็นหน้า)

offset (ตำแหน่งเลื่อนภายในเพจ)	$\log_2$(page size (ขนาดของเพจ)) บิต
page (หน้า)#	บิต address (ที่อยู่หน่วยความจำ) − บิต offset (ตำแหน่งเลื่อนภายในเพจ)
เพจ	ceil(ขนาด ÷ page size (ขนาดของเพจ)) — เริ่มนับ 0
PA	(frame (เฟรม (ก้อน RAM)) × pageSize) + offset (ตำแหน่งเลื่อนภายในเพจ)
EAT (เวลาเข้าถึงเฉลี่ย)	2 + ε − α (ตย. ε=0.1, α=0.8 → 1.3µs)
2-level	inner = ceil(เพจ ÷ 1024) × 4KB + 4KB

💾 Page Replacement (7,0,1,2,0,3,0,4,2,3,0,3,2,1,2,0,1,7,0,1 / 3 frames (เฟรมหลายก้อน))

FIFO (เข้าก่อน-ออกก่อน)	15	เข้าก่อนออกก่อน · มี Belady (3f=9, 4f=10)
OPT	9	มองอนาคต ใช้จริงไม่ได้
LRU (แทนตัวที่ไม่ได้ใช้นานสุด)	12	มองอดีต ใช้จริงได้
EAT(p)	(1−p)×mat + p×service	
128×128	row นอก = 128 · col นอก = 16,384	

🏦 Banker (อัลกอริทึมนายธนาคาร)'s · 📁 File (ไฟล์)

Need	Max − Allocation
ให้ได้ไหม	Req ≤ Need และ ≤ Avail แล้ว safety ใหม่
seg	d < Limit → PA = Base + d (≥ = trap)
fit	First=หลุมแรก Best=เหลือน้อยสุด Worst=ใหญ่สุด
inode (โครงสร้างเก็บข้อมูลไฟล์ของ UNIX (ระบบยูนิกซ์ (บรรพบุรุษของ Linux (OS ลินุกซ์))))	entries = block (บล็อก (หน่วยจัดเก็บ))÷ptr (4K/4B=1024) · 48KB+4MB+4GB+4TB
rwx	r=4 w=2 x=1 · 761 = rwx rw- --x

🚨 กลยุทธ์ห้องสอบ (จากรีวิวรุ่นพี่)

• ถูก/ผิด: ถูก +2 ผิด −1 ไม่ตอบ 0 — **ไม่แน่จริงอย่าตอบ** · • ทำโจทย์คำนวณ (Paging (การแบ่งหน่วยความจำเป็นหน้า)/Replacement (การแทนที่)/Scheduling (การจัดคิว)) **ก่อนข้อทฤษฎี** คะแนน/นาทีสูงกว่า · • จับเวลาข้อคำนวณ ≤ 2 นาที ติดให้ข้าม · • เขียนขั้นตอนสั้น ๆ ข้างคำตอบ · • โค้ดทุกข้อ compile ผ่านหมด — ไม่มีตอบ Error!